Fusion Engine Design Document

Le Hoang Minh Phu

June 1, 2026

Contents

Chapter 1

Introduction

1.1 Overview of the engine

This book will talks about the architecture, mathematics and physics behind the game engine that I made called Fusion Engine. This engine is designed based on two popular game engines that I have used to make games in the past which is Unity and Godot game engine. Other than making games, I was interested in developing reinforcement learning agents to play games or for robotics. Both Unity and Godot offer solution to do this, however they require external extensions which can have some instability and conflict with the engine, it is also difficult to use many python libraries for these engines since they don't support python as their default programming languages. Nvidia Isaac Sim solves this exact problem by providing an integrated environment for creating RL training environments and deploying custom models on them, however the program mainly supports 3D simulation and require high-end hardware.

For these reasons, I have taken on the task of creating my own custom 2D game engine that is designed for creating custom light-weight RL training environments. These small environments can be used for prototyping, testing training algorithm before committing to one for a full training using a more extensive environment like Isaac Sim. It could also be a place for new programmers to learn and get accustomed with training RL agents without the unnecessary bloat or overhead that comes with larger engines. The engine core is coded in C++ for its speed and low-level control, however its functionalities are all exposed to python which allows the user to easily create and train RL models using existing python libraries like open AI gym or stable-baseline.

The engine's design philosophy is based on the Entity Component System present in Unity but also takes some inspiration from the modular scene node system of Godot. Each object in a scene is considered to be an entity, an entity do not perform any functions on its own, components are added to an entity to modify or add a behaviour of the entity. Multiple objects can be combined together into a scene, a scene can contain another scene, this allow the main scene to be concise while still having all the features. The core of the engine is divided into a few main parts:

- Rendering Engine: Handles all the rendering of the engine, for my engine, I used OpenGL as the base for my renderer.
- Physics Engine: Handles all the physics processing, constraint resolution, collision detection and handling of the objects

- Object and Component Manager: Managers components and objects
- Editor: Handles the editor UI, gizmos, I used ImGUI for the entirety of the editor UI
- File and Project Manager: Handles interacting with the file system, saving, loading projects and resources
- Script Manager: Handles communication between C++ and python, provide python bindings, stub for the code editor.

At the start I will go over basic object rendering, transformation. The majority of this book will be spent on how I made my physics engine, going step by step building up from basic force application to more advance topics like constraint resolution, position based dynamics,.. The rest of the books will go over how I implement scripting, combining python and C++ as well as integrating reinforcement learning. In this chapter, I will now go over the general architecture of each of the core systems

1.2 Rendering Engine

The underlying graphics API for my rendering engine is OpenGL which uses a rasterization approach to rendering objects. The rendering pipeline that OpenGL uses is displayed in *Figure 1.1*

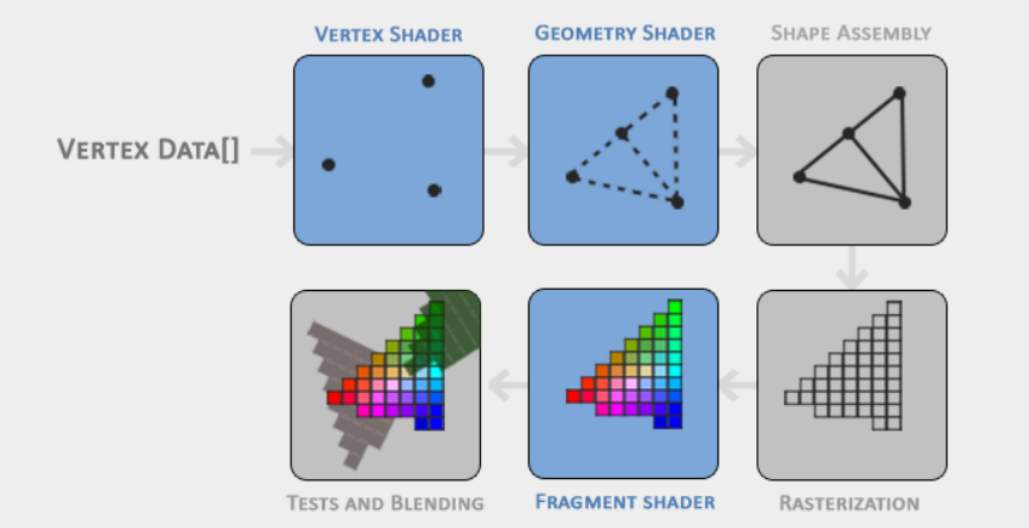

Figure 1.1: OpenGL render pipeline

In my engine, for an object to be able to be rendered, it needs to have a render component which stores data about the vertices, color, texture and shaders. The rendering engine loop through each object in the scene and perform draw calls on each of them. Before drawing the object, the background is draw, the default background is a grid, after drawing an object, the gizmos and optional debug objects are drawn. *Figure 1.2* shows this draw loop, the loop is called once every single frame

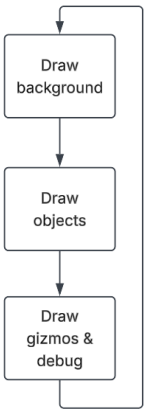

Figure 1.2: Renderer draw loop

Since the engine is in 2D, beyond the draw loop, the rest of the rendering is quite basic so I will not talk about it in further details. However, things like transformation from model space, to world space, to screen space will still be discussed in the later chapters.

1.3 Physics Engine

The physics engine is the core component of the engine and will be the main topic of this book. In this chapter, I will only provide an overview of the entire physics engine architecture. Unlike Unity or Godot physics engine which mostly simulates rigid body, my physics engine is a multi-physics engine that simulates rigid body, soft body and fluids. All of these can work as standalone simulations but can also be combined together into a unified simulation. Cases where this is useful can be simulating a boat on top of water, simulating compression of soft body under water or under heavy rigid body.

For my physics engine, I used a different solver for rigid body, soft body and fluid dynamics and combine them together using an impulse bridge. The details of the combination will be discussed further in the book. To simulation rigid body, I use a Projected Gauss Seidel (PGS) solver with Jacobian matrices to describe constraints, this is also the solution used in the popular physics engine Box2D. For the soft body simulation, I use Extended Position Based Dynamics (XPBD) because of its stability compared to a pure force-based approach. For fluid simulation, I use Position Based Fluids (PBF) since it integrate well with soft body, both working in position space as well as being quite fast.

One of the most important part of a physics engine is the collision detection and resolution. For the broad phase collision checking, I use a Bounding Area Hierarchy (BAH) approach and for the narrow phase I use a combination of different approaches. For rigid body collision, Separating axis theorem (SAT) is used for narrow phase checking and Sutherland Hodgeman line clipping algorithm is used for contact points generation. For rigid - soft body and soft - soft body collision, a ray casting approach is used. The ray casting approach is also used to detect collision between

fluid and rigid, soft bodies. For collision resolution, impulse are applied to both objects to correct their velocities. For soft - soft body collision, direct positional correction is applied, and for collision involving multiple solvers a mixed approach is used.

The physics processing loop for my engine is displayed in *Figure 1.3*

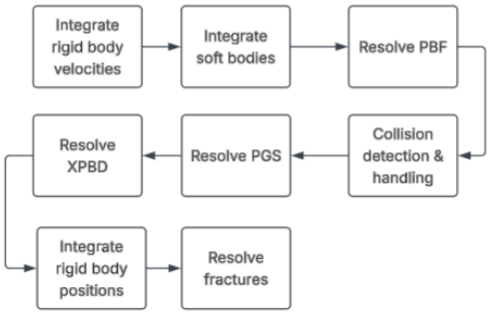

Figure 1.3: Physics process loop

Figure 1.4 shows the algorithm I use for each case in the narrow phase collision detection part. The details of each algorithm will be discussed further in the later chapters.

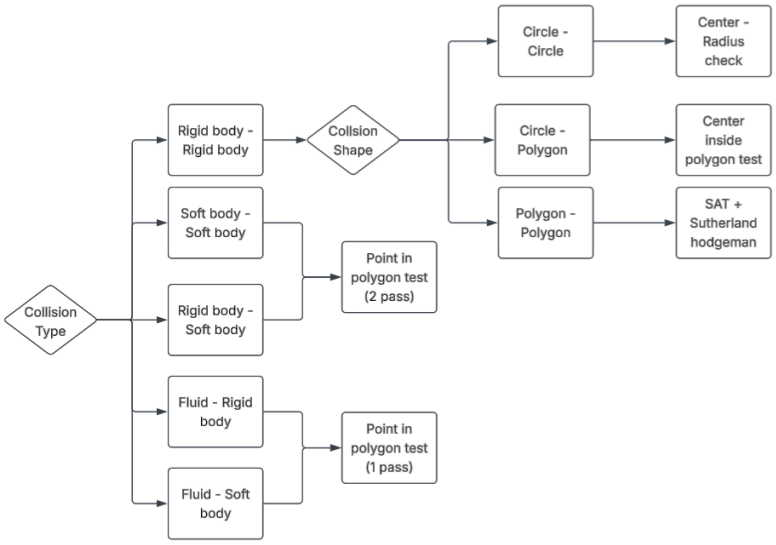

Figure 1.4: Collision narrow phase detection case

Figure 1.5 displays the solver loop for PGS resolution (rigid body), XPBD resolution (soft body) and PBF resolution (fluid). The specifics of the physics, mathematics in the algorithm will be discussed in further detail in their corresponding chapters.

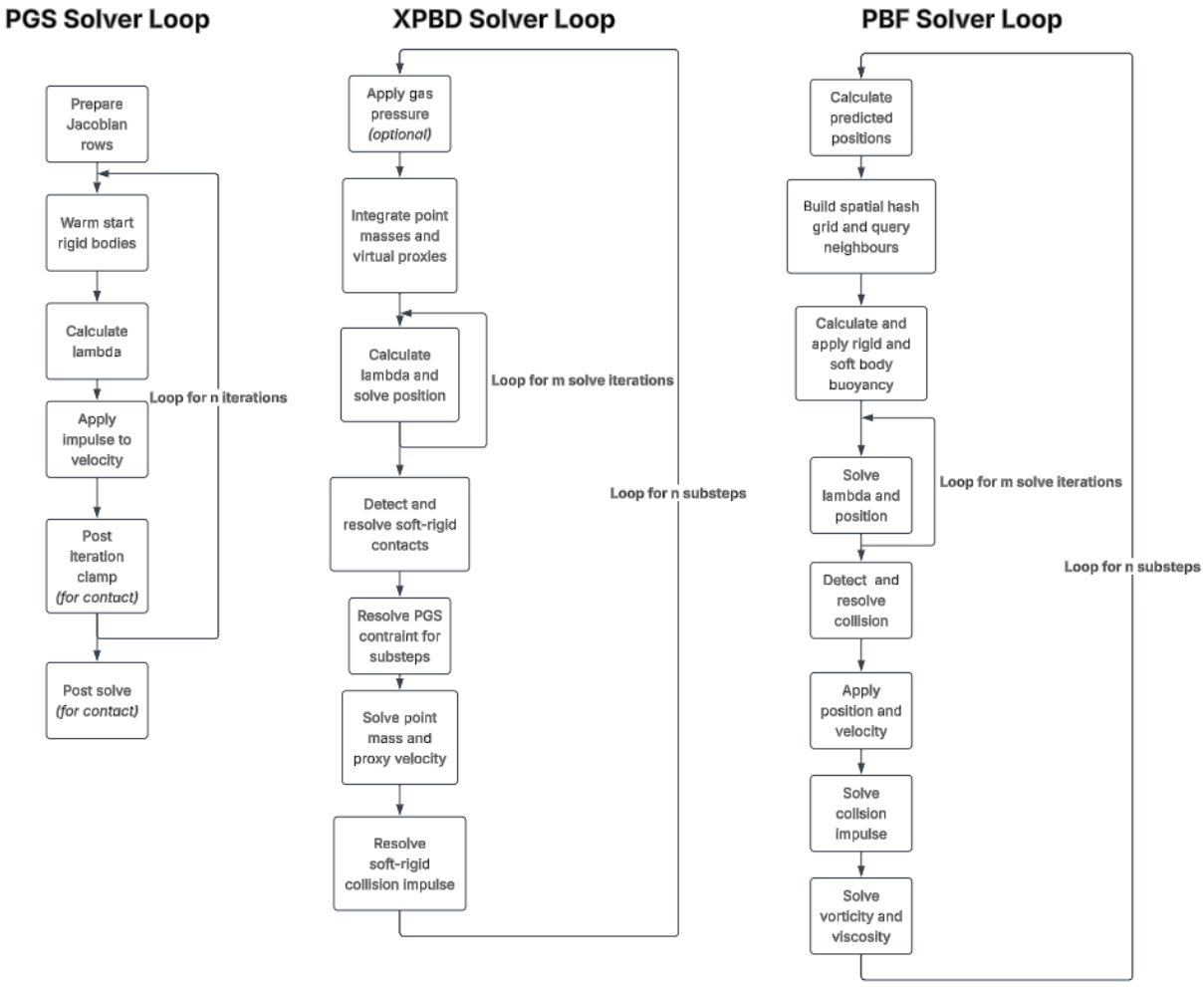

Figure 1.5: Physics Solver Loop

1.4 Entity - Component System (ECS)

My engine use an entity component system as the core system for managing the objects in my engine. an entity (or Object) in my engine, similar to Unity, only have a transform, a 2D position, rotation and scale but does not perform any other functionality. Components are added to the object to modify or change the object in certain ways. There are a few rules I follow for creating component

- Uniqueness: each object can only have one component of a certain type, so each component must not require multiple of itself to function.
- Single functionality: each component must only perform a specific function that it is designed for. For example: rigid body component should only simulate rigid body, but it should not also define collision shape, render the object,.. The component should also not perform the functionality of another component.
- No dependencies: a component can not depend on another component to function, unless that component is guarantee to be on the object or there is a backup option in case that component is not present. In these cases, a warning should be given.

Figure 1.6 shows the memory management approach I used for my engine. In my engine, the object manager should hold a list of all objects in the scene, each object should hold a list of all its components, each component should hold a reference to its parent. Because when deleting an object, we want the component to automatically be free from memory to avoid memory leaks and dangling pointers, I used the C++ unique pointers to handles these cases. Deleting a component however should not delete its parent, so the reference each component hold will be a raw pointer.

Figure 1.6: ECS memory management

Each object in the scene is differentiated by a unique identifier, an Object ID. The ID is generated randomly for each object when it is created and is ensured not to be a duplicate of any other ids. The id is useful in linking object, instead of tracking pointers, or the object itself, we simply store the id and get the object from the id when necessary.

Additionally, the object list is more like an object tree, each object can have a parent, an object can have infinitely many children, moving the parent will move its children. Some properties may also be shared between the parent and children.

Multiple objects can be arranged together into a scene which can be saved into the file system. When running the game, a selected main scene is played. Scenes can also be instantiated into another scene, the entire scene is treated as one object, however one scene, all of the objects that is part of a scene will update every instance of it in every other scenes, this object and scene structure is inspired by the approach used by Godot Engine.

1.5 Editor

The editor is an important part of the engine, it allows anyone to interact with the engine without touching the code. The two main components of the Editor is the windows and the gizmos. An editor window usually provide a lot information and allow the user to interact with a lot of things while the gizmos only allow for a specific set of actions.

In my engine the most important windows are

- Hierarchy: Shows the list of objects in the scene (in a tree format)
- Inspector: Shows details about each object in the scene when interacted with (components, properties,..)
- Console: Shows print, warning and error messages
- Status: Shows fps count, play, stop, pause buttons,..
- File system: Shows the resources file of the current project in a tree structure

Some other useful window can include, add object window, export project window, engine profiler window,.. For the gizmos, the most common ones are the object editing gizmos, polygon edit gizmos and constraint edit gizmos. *Figure 1.7* shows the editor UI of Fusion Engine in an example project. Even though the majority of the editor is finished, it should be noted that these are subject to change in the future.

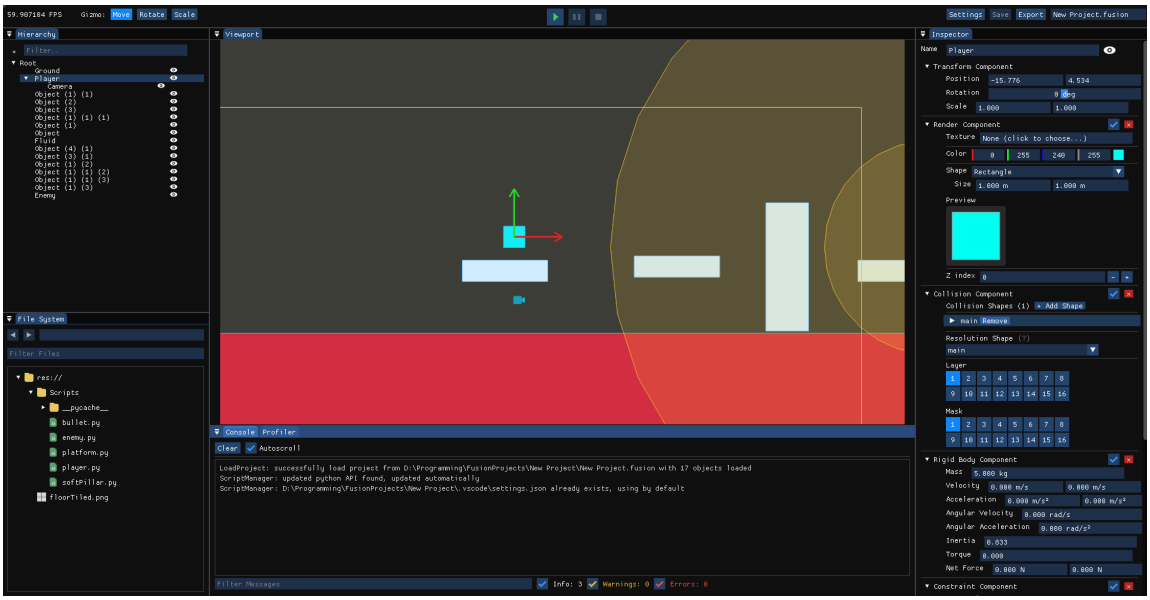

Figure 1.7: Editor UI

The editor UI is only meant for editing the game / simulation, the editor should not be included in the exported game. The way I handle this is to simply excluding the editor part when compiling the game for export.

1.6 File and Project Management

The file and project managers handles the files within a project in my engine as well as the anything file related in the functioning of the engine.

Every resources in a project like images and scripts are saved in relative path, this is the same way as how Godot engine handles resources. The relative path is basically a path that starts at the project folder instead of at the disk (absolute path). For example *res://img.png* is a relative path while *C://users/Fusion/img.png* is an absolute path. While everything is stored in relative path, the project path is stored in absolute path since its value is used to convert from relative to absolute path and back.

When any changes is done, for it to be saved permanently, the changes needs to be saved in a file. In my engine, the data of every objects and components is serialize into binary and stored in a scene file. This file has the extension *.fscene*, when opening a scene, the contents of the file is deserialize and loaded. A project also have a project save file which is a *.fusion* file, this file contains the info of the project such as settings and configurations that affect the entire project as a whole. When opening the project, this file is first loaded, inside this file contains the path to a main scene, the engine loads up the main scene from the path.

One of the most important things that the project and file manager handles is the project exporting. This is not as simple as saving and loading the project since the code of the game as well as resources about the game should not be readable to the players. Because my engine uses Python as its programming language, simply compiling the python code would not be enough, so I use a simple encryption scheme to encrypt the python code as well as the project files (.fusion and .fscene files). Other resources like images and textures are not encrypted however, since they usually don't contain a lot of critical information so it is not worth it to encrypt these files.

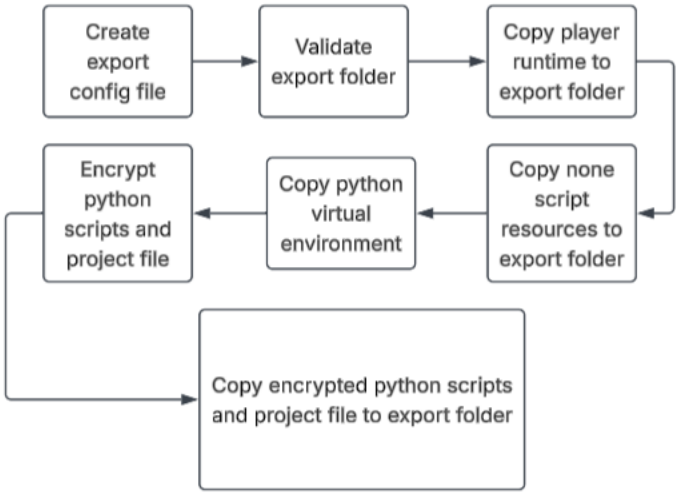

Figure 1.8: Project exporting process

Figure 1.8 shows the process of exporting a project. When exporting, an export config file will be created, this file is also copied to the export folder and basically contains info about the author, version and identify that folder as an export folder for a Fusion engine project. The encryption algorithm I use for encrypting the python scripts and project file is the XOR cipher, while it is less secure than other more modern cipher like AES, I choose it because of its simplicity to implement while providing enough security to deter most people.

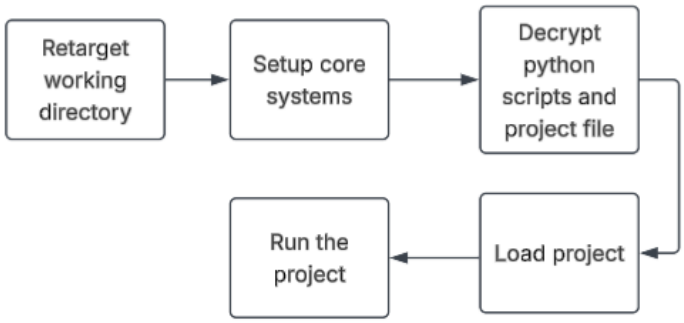

Figure 1.9: Running exported project

Figure 1.8 shows the process of opening an exported project and running it. Since the exported executable will most likely not be in the same folder as the project folder, the project path property is incorrect which will lead to all the relative path being incorrect as well. This is why the first step is to change the project path to the directory holding the executable.

The XOR cipher used to encrypt the core files works in a pretty simple way, first when exporting the project, a secret key is generated and attached to the exported project. This secret key is buried in the compiled binary so it is difficult for some one to reverse engineer the secret key. Because the secret key is randomly generated, cracking one key for a single exported project will not automatically break every single export projects. The final binary is encoded from the secret key as

$$\textbf{final binary} = \textbf{secret key} \oplus \textbf{raw binary} \tag{1.1}$$

raw binary is the binary written simply by converting the raw text into binary for python scripts and the *.fusion*, *.fscene* file binary. $\oplus$ is the symbol for XOR (exclusive or) operation. Decrypting the final binary back into the raw binary when decrypting is also very simple

$$\textbf{raw binary} = \textbf{secret key} \oplus \textbf{final binary} \tag{1.2}$$

1.7 Scripting

My engine core is coded in C++ however, almost every core functionality is exposed to Python for scripting. I chose python for a few reasons, first since the purpose of the engine is to develop training environment for RL agents aside from acting as a game engine, python provide the perfect environment with many libraries possible for this purpose. Another reason is that the syntax of python is much simpler compared to the complexity of C++ code.

Since C++ is a compiled language and python is an interpreted language, there is many differences in their syntax so a translation layer is needed to allow C++ to call python functions and python code to call C++ functions. The library I used for the translation layer is pybind11 which is a robust library for integrating python with C++, the python version used in my engine is python 3.11, however any later version is can also be used.

When setting the project, other than the resources folder, the python virtual environment also needs to be set up. Normally when creating a virtual environment, it is as simple as entering the command *python -m venv .venv*, however, this process is extended a bit in my engine in order to set up the engine API. This process is demonstrated in *Figure 1.10*

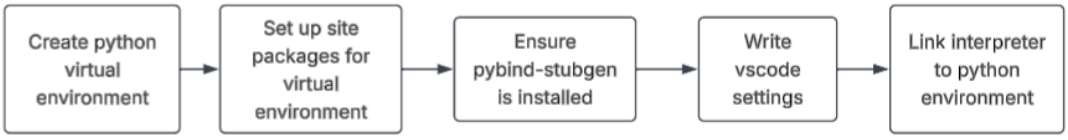

Figure 1.10: Python environment set up

Site packages are where python libraries are store, which is also where the python compiled module that links the C++ engine core and python. For visual studio code (which my engine natively support) and for any other code editor to provide code recommendation, syntax highlighting for my engine specific syntax, a python stub file (*.pyi*) needs to be generated. Pybind11 already provide a native solution for generating python stub file, by first compiling to a *.pyd* file and using the tag **PYBIND11_MODULE**, the compiled file is not a stub file yet and merely a compiled python bindings. The second step is installing pybind-stubgen into the python virtual environment and running it against the compiled python module to generate the actual *.pyi* python stub file, which will be stored in a typings folder.

After that, visual studio code settings file is written and the python interpreter is linked to the python environment to link everything together. The settings file will include the stub file path and the linked interpreter path. With the python environment and engine API set up, it is as simple as downloading external libraries like open AI gym or stable baseline to start integrating reinforcement learning model. The specifics of the scripting pipeline and integration with open AI gym will be detailed in the later chapters of the book.

Chapter 2

Position and Transformation

2.1 Vector transformation

In my engine and in all engines in general, vectors are an indispensable part of it, they are used to store position, velocity, acceleration, forces,.. and form the basis of rendering and physics. To change, or transform an object position, velocity we need to be able to manipulate the vector, the 3 core transformation are translation, rotation and scale.

While it is possible to calculate the x, y components for each transformation, we can use the properties of matrices to transform the vector. This will allow us to transform the vector much more easily as well as being able to stack multiple transformation on top of each other

To translate a vector $\mathbf{u}(x, y)$ by a vector $\mathbf{t}(x_t, y_t)$, we can construct a translation matrix $\mathbf{T}$ to get the resulting vector $\mathbf{u}'(x', y')$ as

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & x_t \\ 0 & 1 & y_t \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \tag{2.1}$$

To rotate the vector by an angle θ, we can construct a rotation matrix and get the resulting vector as

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \tag{2.2}$$

To scale a vector by a scaling vector $\mathbf{s}(x_s, y_s)$, we repeat the same process

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \tag{2.3}$$

Like I said before, we can combine these three transformation together into one large transformation, allowing us to manipulate a vector to any configuration

$$\mathbf{u}' = (\mathbf{T} \cdot \mathbf{R} \cdot \mathbf{S})\mathbf{u} \tag{2.4}$$

Where $\mathbf{T}$ is the translation matrix, $\mathbf{R}$ is the rotation matrix and $\mathbf{S}$ is the scale matrix
Expanding this we can get the final transformation equation as

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x \cos\theta & -s_y \sin\theta & t_x \\ s_x \sin\theta & s_y \cos\theta & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \tag{2.5}$$

2.2 Model space, World space, Screen space

In graphics rendering, there are three spaces we need to consider, first is the model or local space, this is the space where the raw position of each vertices of an object. World space is the position of each vertices after the camera view matrix has been applied and screen space is the final position of each vertices after the projection matrix has been applied. The order of application is detailed in the equation below where $\mathbf{T}$ is the transform matrix, $\mathbf{V}$ is the view matrix and $\mathbf{P}$ is the projection matrix.

$$\mathbf{vert}_{final} = (\mathbf{P} \cdot \mathbf{V} \cdot \mathbf{T})\mathbf{vert}_{raw} \tag{2.6}$$

The first matrix is the transform matrix which controls local translation, rotation and scale. Note that due to the quirks of OpenGL, before constructing the transform matrix, we need to translate to the world origin, apply the transform and then translate back. Next is the view matrix, in games and computer graphics in general, when the camera move, we don't move the camera but move everything around the camera in the opposite direction in order to simulate the illusion of movement. Finally the projection matrix project the vertices onto the screen and give in perspective, this is especially useful in 3D with perspective projection, but for 2D we use the orthogonal projection. The formula for the orthogonal projection matrix is

$$\begin{bmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & -\frac{2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2.7}$$

Where r is right which is the aspect ratio, l is left which is negative of the aspect ratio. t, b is top and bottom which is set to 1 and -1, n, f is near and far which is set to -1 and 1 respectively. One thing of note is that the projection matrix is a 4x4 matrix while the transformation and view matrix is 3x3 matrix, multiplying these matrices together won't work. The fix is append to the transformation and view matrix to make it a 4x4 matrix.

One of the most important task is to convert from local to world space and convert from world to local space. The formula to project from local to world space and from world to local space is

$$\begin{aligned} \mathbf{p}_{world} &= (\mathbf{O} \cdot \mathbf{T})\mathbf{p}_{local} \\ \mathbf{p}_{local} &= (\mathbf{O} \cdot \mathbf{T})^{-1}\mathbf{p}_{world} \end{aligned} \tag{2.8}$$

Where $\mathbf{O}$ is the origin transform matrix of the object which is the inverse of the view matrix when the object was created $\mathbf{O} = \mathbf{V}_{t_0}^{-1}$

Chapter 3

Basic forces and kinematics

3.1 Forces and Euler integrator

The physics engine for Fusion Engine first started not as the current impulse based systems with different solvers but as a simple force-based physics engine. The core of the force-based physics engine rely on Newton 2nd law of motion

$$\mathbf{F} = m\mathbf{a} \tag{3.1}$$

Where $\mathbf{F}$ is the force, $\mathbf{a}$ is the acceleration and m is the mass. From the acceleration, we can calculate the velocity and position of an object using a physics integrator. There are two popular physics integrator, these are

- Euler integrator: Simple to implement, delta-time dependent, can be unstable. Velocity is calculated from acceleration and position is calculated from velocity.
- Verlet integrator: More difficult to implement, more stable, less reliant and delta-time. Predicted position is calculated from acceleration, then velocity is derived from the change in position.

There are other physics integrator like RK4, Leapfrog, Implicit Euler,.. however they are either a variation of or extension of the two basic integration methods above. The integrator I chose for my engine was the Euler integrator, the formula for Euler integration is

$$\begin{aligned} \mathbf{v}' &= \mathbf{v} + \mathbf{a}\Delta t \\ \mathbf{x}' &= \mathbf{x} + \mathbf{v}\Delta t \end{aligned} \tag{3.2}$$

Instead of calculating velocity and position every time a force is apply, I instead have a force accumulator, each frame the net force is reset and forces are accumulated, at the end of the frame, the acceleration is calculated from the force.

3.2 Global forces

Global forces are forces that are always applied to an object regardless of where they are. The first of these is gravity which is calculated as

$$\mathbf{F}_g = \begin{bmatrix} 0 \\ mg \\ 0 \end{bmatrix} \tag{3.3}$$

where g is the gravitational acceleration, which for my engine is set as $g = -9.8$. The other global force is the drag force which is a force opposing the object's velocity. The drag force is mainly to prevent energy injection that is common in Euler integrator which can cause instability. The equation for the drag force is

$$\mathbf{F}_d = -\hat{\mathbf{v}} \left(k_1 \|\mathbf{v}\| + k_2 \|\mathbf{v}\|^2 \right) \tag{3.4}$$

where $\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|}$ is the normalized velocity, k_1 and k_2 are the drag coefficients.

3.3 Spring force

Other than the global forces, some forces only act on objects in a certain way and under a certain configuration. One of the most basic force is the spring force which is of course caused by springs. The spring force is calculated from Hooke's law as

$$F_s = k_s \left(l - l_0 \right) \tag{3.5}$$

Where l is the current length of the spring currently, l_0 is the rest length and k_s is the spring constant. l can be calculated as the distance between two objects $||\mathbf{d}||$, the direction of the force is pushing the object away if $l - l_0 < 0$ and closer together if $l - l_0 > 0$. With that the equation from the spring force becomes

$$\mathbf{F}_s = -k_s \left(\|\mathbf{d}\| - l_0 \right) \hat{\mathbf{d}} \tag{3.6}$$

Where $\hat{\mathbf{d}}$ is the normalized direction vector of the two objects. However, currently the spring is not damped and due to the instability of Euler integration, the system can get injected with energy and quickly spun out of control. A way to combat this is to add a damping factor to the spring, making the system loose energy overtime. The damping factor is calculated based on the difference in velocity of the two objects

$$\mathbf{F}_s = -k_s \left(\|\mathbf{d}\| - l_0 \right) \hat{\mathbf{d}} - c_s \left(\mathbf{v}_A - \mathbf{v}_B \right) \tag{3.7}$$

Where c_s is the damping coefficient, for a perfectly damped spring, the damping coefficient is calculated from the spring coefficient as $c_s = 2\sqrt{k_s m}$

Chapter 4

Rigid body physics

4.1 Euler integration for rotation

In the previous chapter, we have been working with motion of a particle which has only position and scale but no rotation. A rigid body is basically a particle but with rotation and rotational motion and similar to how linear velocity is integrated, we calculate angular velocity and rotation the same way.

In 3D, rotation is determined using complex quaternions however in 2D it is a simple scalar value, this is the same for angular velocity and acceleration. These properties can be calculated as

$$\begin{aligned} \theta' &= \theta + \omega \Delta t \\ \omega' &= \omega + a_\omega \Delta t \end{aligned} \tag{4.1}$$

Where θ is the object's rotation, ω is the angular velocity and a_ω is the angular acceleration. There is also an equivalent of mass and force which is nertia $\mathbf{I}$ and torque τ. The torque is calculated similar to force using Newton 2nd law for rotation as

$$\tau = I a_\omega \tag{4.2}$$

4.2 Torque and Inertia

Other than $\tau = I a_\omega$, torque can also be calculated from the force that is applied on the object to a point in an object surface.

$$\tau = x F_x - y F_y \tag{4.3}$$

where $\mathbf{F}(F_x, F_y)$ is the force applied to the object and $\mathbf{P}(x, y)$ is the point in which the force was applied. This position is relative to the center of mass of the object, when the force is applied directly to the center of mass, the torque will be 0.

The formula to calculate inertia for objects in two dimension is

$$I = \iint_A \rho(x, y) r^2 dx dy \tag{4.4}$$

where $\rho(x, y)$ is the density of the object at point (x, y), r is the distance from point (x, y) to the position of the center of mass $\mathbf{p_c}$ and A is the two dimensional surface that represents the object

This formula however, can't be used in programming since it represent a continuous surface however we only have discrete points (vertices of the object), so we need to approximate this integral through summation.

$$I = \sum_i \rho_i r_i^2 \tag{4.5}$$

ρ_i is the density of the object at vertex i and r_i is the distance from that vertex to the center of mass

We can further simplify this expression by assuming uniform mass distribution

$$I = \sum_i \frac{m}{A} r_i^2 \tag{4.6}$$

Where m is the mass of the object and A is the area of the object

However, if we only use the vertices of the polygon to calculate this, the object will feel hollow (since the density will seem to be focused around the edges). Instead, what I use to calculate the inertia is to triangulate the polygon and then calculate the inertia of each of the triangles and combine them to get the final inertia.

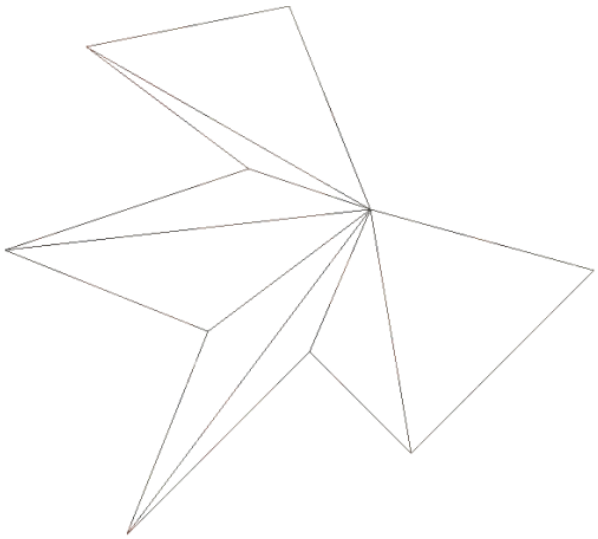

Figure 4.1: Triangulated polygon

Figure 4.1 show the image of a complex polygon when triangulated using the ear-clipping algorithm. The equation to calculate the inertia of a triangle with uniformly distributed mass is

$$I_{triangle} = \frac{m_{triangle}}{36}(||\mathbf{A}-\mathbf{B}||^2 + ||\mathbf{B}-\mathbf{C}||^2 + ||\mathbf{C}-\mathbf{A}||^2) \tag{4.7}$$

Where $\mathbf{A}, \mathbf{B}, \mathbf{C}$ are the vertices of the triangle, $m_{triangle}$ is the mass of the triangle which is calculated as

$$m_{triangle} = m_{polygon}\frac{A_{triangle}}{A_{polygon}} \tag{4.8}$$

The final center of mass is now calculated as

$$I = \sum(I_{triangle} + m_{triangle}||\mathbf{C_{triangle}} - \mathbf{CM}||^2) \tag{4.9}$$

Where $\mathbf{C}_{triangle}$ is the center of the triangle and $\mathbf{CM}$ is the center of mass of the polygon

Chapter 5

Collision detection, contact points generation and collision resolution

5.1 Overview

Collision detection is divided into two distinct phases

- Broad phase: A simple check that are done on all objects to filter out objects that are potentially colliding
- Narrow phase: A more detailed check that check if two objects are actually colliding and generate the contact points between the objects

Collision resolution is also divided into two phases

- Velocity solver: Solve for the velocity after the collision (both linear and angular)
- Interpenetration solver: Resolve interpenetration of objects

5.2 Broad phase

There are many algorithm for broad phase collision checking, however the one I used for this engine is using bounding circles and insertion approach for populating the collision hierarchy

The main idea for broad phase checking is instead of checking the complex geometry of the object, we simplify the geometry, in this case turning it into a circle. The circle must be big enough to encompass the polygon

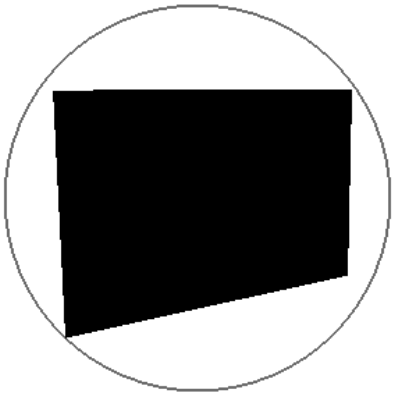

Figure 5.1: Polygon bounded by a circle

The formula for checking if two circles are colliding is simple

$$\text{isColliding} = ||\mathbf{C_2} - \mathbf{C_1}||^2 < (R_1 + R_2)^2 \tag{5.1}$$

Where $\mathbf{C_1}, \mathbf{C_2}$ are the center of the two circles and R_1, R_2 are the radius of the two circles.

While it is possible to check every single pair of collision, it is better to group objects that are close together into groups. To do this I used a tree data structure to represent the hierarchy of collisions, if the root of a branch does not collide with another root then that entire branch can be discarded for the collision check, drastically reducing the amount of operations required.

When an object is added to the world, it is also inserted to the collision hierarchy. The tree is traversed from the root to each leafs. I used a recursive algorithm to traverse through the tree for insertion:

- Start from a root node. If it does not have any children and the root does not have any object stored, then set the root node's stored object to this node.
- If the root has no children but stores an object, then change it from a leaf (a node that stores an object) to a branch (a node that stores children). The left child will hold the object that the root is currently storing, and the right child will hold the object that is inserted. This can be seen in *Figure 5.2* where two objects are bounded by a larger parent branch.
- If the root is a branch, then check if inserting a node in the left node will grow the tree more (in terms of circle size) or if the right node will grow the tree more. Select the smaller size growth and recursively call insert on that child node. This is clearly displayed in *Figure 5.3* where the third object is bounded together with the object closest to it.

- Then finally recalculate the size of the bounding circle after the node has been inserted (make sure the bounding node bounds every child node).

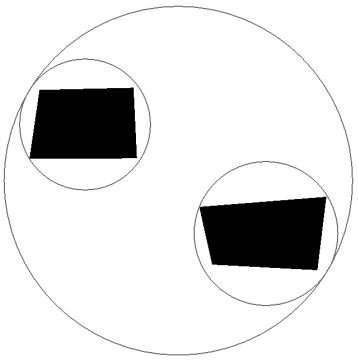

Figure 5.2: Two objects bounded by a larger parent

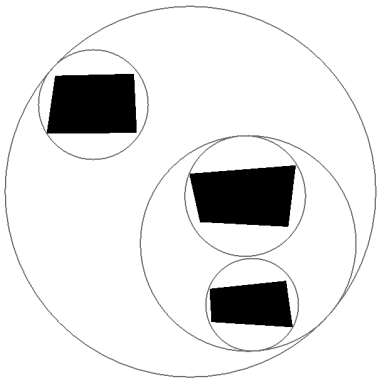

Figure 5.3: Object bounded with its closest neighbor

Removing objects from a hierarchy goes through a similar process.

- First, the tree is traversed until the node that stores the object is found.
- Because a branch can only have two children (left and right), when an object is removed, there is no reason to store the branch anymore. The branch can be removed and turned into a leaf node instead; the data stored in the node will be the other child of the branch node (the one node not removed). *Figure 5.4* displays this: when the third object is removed, the bounding circle around the second and third object is also removed.

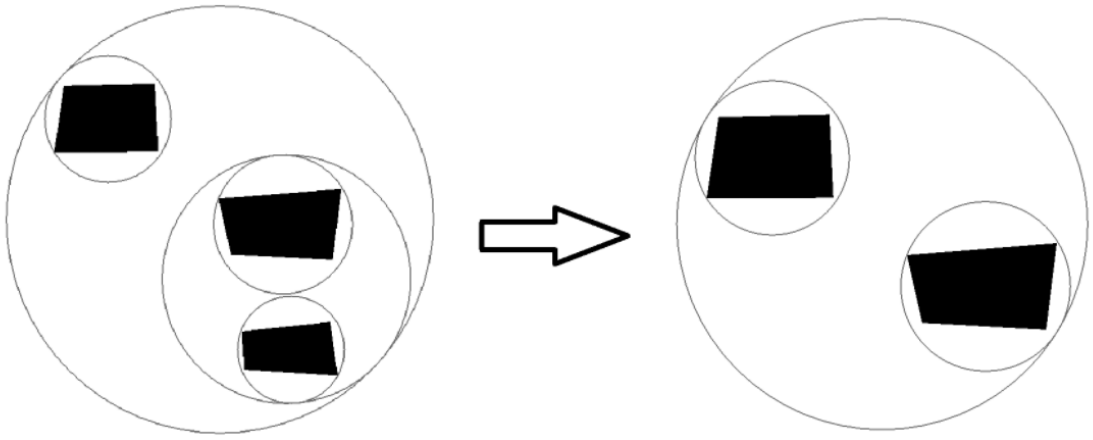

Figure 5.4: Result after object removal

The algorithm for checking for overlap traverses through the hierarchy, if the parent node is colliding then it continues checking else the checking halt. If it comes across two leaf nodes that are colliding then the two objects will be registered for narrow phase collision check. *Figure 5.5* shows the result of broad phase collision checking, objects in red are checked but no potential collision are detected while objects in green are registered for narrow phase checking.

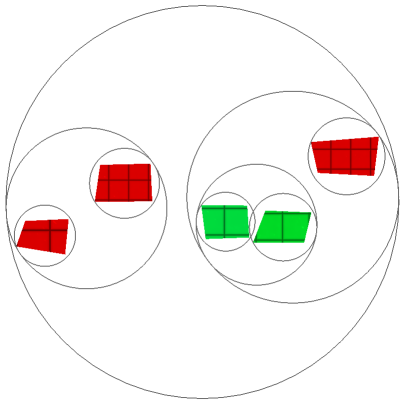

Figure 5.5: Registered objects after broad phase

Other than using circles, it is also possible to use Axis Aligned Bounding Boxes (AABB) instead. While the process of checking overlap for AABB require more processing power than between circles, the hierarchy is much more space efficient when objects are very wide or very tall (circles leave a lot of unecessary space), so AABB are generally considered faster than bounding circle. The only difference between bounding circle and AABB are the overlap calculation, the rest of the process like inserting, removing nodes from the hierarchy stays the same.

Two AABB boxes are overlapping if it satisfied

$$\text{overlap} = \begin{cases} x_A < x_B + w_B \\ x_A + x_A + w_A > x_B \\ y_A < y_B + h_B \\ y_A + h_A > y_B \end{cases} \tag{5.2}$$

Where x_A, y_A is the position of the center of the first object and x_B, y_B is the position of the center of the second object. w_A, w_B, h_A, h_B are the width and height of the bounding boxes wrapped around the two objects.

5.3 Narrow phase (SAT)

For narrow phase checking, the goal is to check if two objects are actually colliding or not as well as finding the collision normal and penetration depth. The most popular algorithm, which is also the one I used for my engine, is the Separating Axis Theorem (SAT).

The theorem states that for any pair of convex polygons, the two polygons do not overlap if there exists an axis (a line in 2D) onto which the projected shadow of the two polygons do not overlap.

The axis will always be parallel to one of the edges of either polygon. The projection of a polygon on an axis will have a max point and a min point. To project a vertex onto the axis, we take the dot product of the point with the axis normal.

$$\begin{aligned} \text{maxPoint} &= \max_i(\mathbf{vert_i} \cdot \mathbf{n}) \\ \text{minPoint} &= \min_i(\mathbf{vert_i} \cdot \mathbf{n}) \end{aligned} \tag{5.3}$$

Where $\mathbf{vert_i}$ is a vertex of the polygon. This is repeated for each vertex in each polygon. *Figure 5.6* displays two polygons projected onto a separating axis

The projection overlaps in the case that

$$\text{overlap} = b_{min} < a_{max} \vee a_{min} < b_{max} \tag{5.4}$$

We also want to find the collision normal and penetration depth, to find this, we find the axis with the minimum overlap amount.

$$\begin{aligned} \text{overlapAmount}_i &= \min(a_{\max}, b_{\max}) - \max(a_{\min}, b_{\min}) \\ \text{penetration} &= \min_i(\text{overlapAmount}_i) \\ \mathbf{n} &= \mathbf{n}_i \quad \text{if overlapAmount}_i = \text{penetration} \end{aligned} \tag{5.5}$$

So the general workflow for the algorithm is

- Loop through each edges of both polygons and calculate the normal of each edges (this is the axis normal)
- Project each vertices of both polygons onto the axis using the axis normal

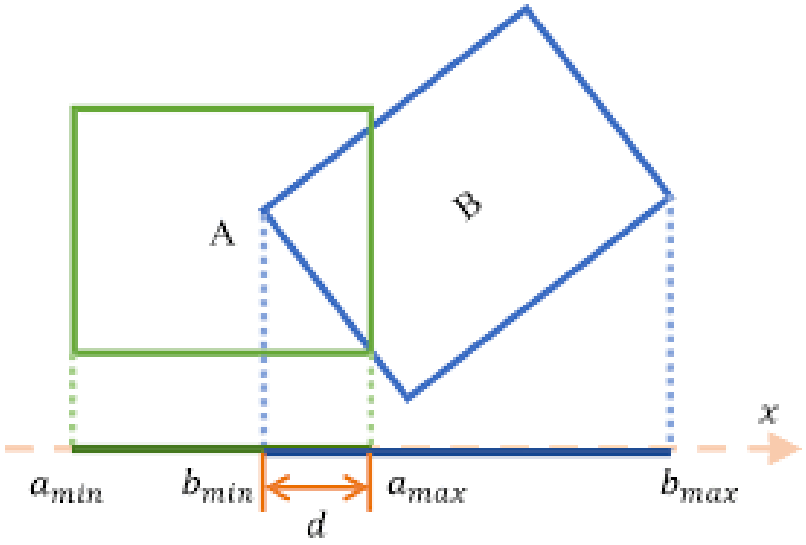

Figure 5.6: Projection of vertices on an axis

- Check if the projection overlaps, if it does not, exit early.
- Calculate the overlap amount
- Find the penetration depth and the collision normal

5.4 Narrow phase (Contact generation)

The SAT only detects whether two polygons are colliding but it does not detects where the collision occurs (the collision points). To do this, we need a separate contact points generation algorithm. For two colliding polygons, the maximum number of contact points is 2 and the minimum is 1. *Figure 5.7* and *Figure 5.8* displays example of contact points generation algorithm in action.

The algorithm I use for contact points generation is a variation of the Sutherland-Hodgman Clipping algorithm. The algorithm works by finding a reference edge and an incident edge, the incident edge is the edge that will be cut off by the reference edge. After the cut, the remaining points will be the contact points and the points depth will be the penetration depth.

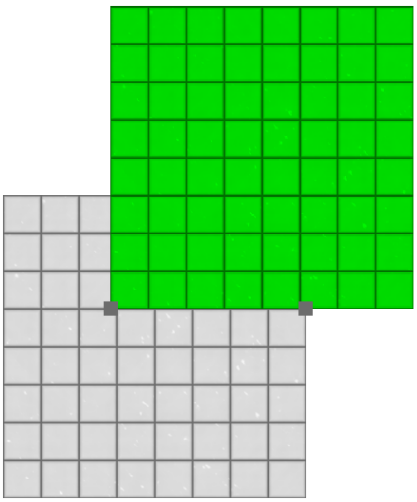

Figure 5.7: Two contact points collision

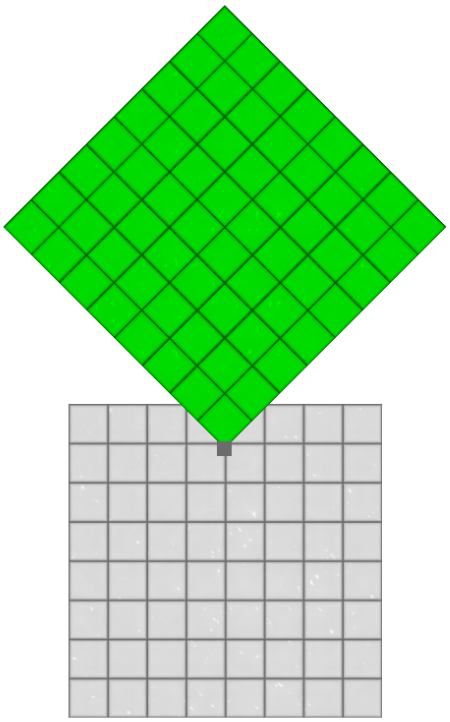

Figure 5.8: One contact point collision

The first step of the algorithm is finding out which edge is the reference edge and which edge is the incident edge. To do this, we find the edges that is most parallel to and most anti parallel to the collision normal. The reference edge will be the edge that is most parallel to the collision normal and the incident edge will be the one that is most anti parallel to the collision normal.

$$\begin{aligned} \text{ReferenceEdgePolygon} &= \text{edge}_i \quad \text{if } \mathbf{n_i} \cdot \mathbf{n} = \max_i(\mathbf{n_i} \cdot \mathbf{n}) \\ \text{IncidentEdgePolygon} &= \text{edge}_i \quad \text{if } \mathbf{n_i} \cdot \mathbf{n} = \min_i(\mathbf{n_i} \cdot \mathbf{n}) \end{aligned} \tag{5.6}$$

This formula only finds the incident and reference edge of a polygon ($\mathbf{n_i}$ is the normal of an edge of the polygon). To find which object will provide the reference and which will provide the incident edge, we will need some additional calculation

$$\begin{aligned} \text{ReferenceEdge} = R_a \quad \text{IncidentEdge} = I_b \quad \text{if } ||\mathbf{n_a} \cdot \mathbf{n}|| \geq ||\mathbf{n_b} \cdot \mathbf{n}|| \\ \text{ReferenceEdge} = R_b \quad \text{IncidentEdge} = I_a \quad \text{if } ||\mathbf{n_a} \cdot \mathbf{n}|| < ||\mathbf{n_b} \cdot \mathbf{n}|| \end{aligned} \tag{5.7}$$

where R_a, I_a and R_b, I_b are the reference edges and the incident edges calculated from the two polygons. $\mathbf{n_a}$ and $\mathbf{n_b}$ are the normals of the reference edge of the two polygons.

After finding the reference and incident edges, we clip the incident edge by the reference edge using the Sutherland-Hodgman line clipping algorithm. The algorithm computes the signed distance

from each vertex of the reference edge to the clipping line using a dot product

$$d = (\mathbf{n} \cdot \mathbf{vert}) - \text{offset} \tag{5.8}$$

Where $\mathbf{n}$ is the outward pointing normal of the clipping plane and **vert** is the position of the vertices being tested. $offset$ is the position of the clipping plane along the normal. If $d \leq 0$ the point is on the inside and is kept, else the point is outside and is discarded.

Clipping the two vertices of the incident edge results in d_0 and d_1. If one of the points are discarded, a new point must be added by finding the intersection point between the clipping line and the incident edge. The equation to find the intersection point is

$$\begin{aligned} t &= \frac{d_0}{d_0 - d_1} \\ \mathbf{p_i} &= \mathbf{v_0} + t(\mathbf{v_1} - \mathbf{v_0}) \end{aligned} \tag{5.9}$$

Where $\mathbf{p_i}$ is the position of the intersection point, $\mathbf{v_0}$ and $\mathbf{v_1}$ are the vertices of the incident edge.*Figure 5.9* displays how the incident edge is clipped by the reference edge (the reference edge is blue, the incident edge is red). The dotted lines are the clipping plane on the left and right side

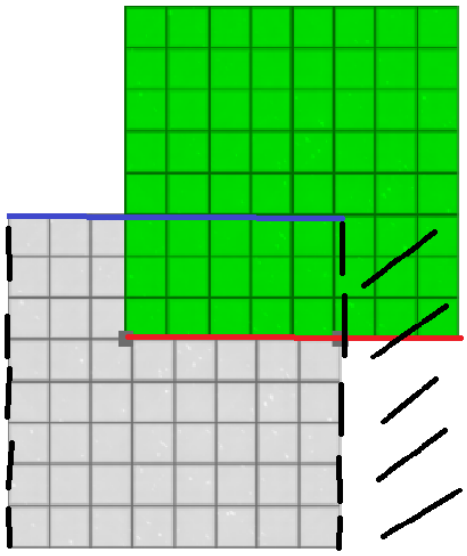

Figure 5.9: Sutherland-Hodgman clipping

The depth of the contact points is calculated as

$$\begin{aligned} \text{refOffset} &= \max(\mathbf{vert} \cdot \mathbf{n}) \\ d_i &= \text{refOffset} - \mathbf{n} \cdot \mathbf{p_i} \end{aligned} \tag{5.10}$$

Where $\mathbf{p_i}$ is the position of the contact point and $\mathbf{n}$ is the collision normal

5.5 Collision resolution (Velocity solver)

Before we have created a basic contact resolution system, however there the system does not consider the full rigid body objects (with rotation and angular velocities). The new velocity solver must combine both angular and linear velocities and resolve them for both objects. The core of the algorithm I used is still an impulse based system, which is based on the equation

$$v_s' = -cv_s \tag{5.11}$$

Where v_s is the relative velocity of the objects immediately before the collision, v_s' is the relative velocity after the collision and c is the contact restitution

The first step is working out what the relative velocities of the object before the collision are. To do this we need to find the total velocity of the contact point in each body which is given by the equation.

$$\mathbf{v}_{p,i} = \mathbf{v}_i + \vec{\omega}_i \times \mathbf{r}_i \tag{5.12}$$

Where $\mathbf{v}_{p,i}$ is the total velocity at point p for the rigid body i and $\mathbf{v_i}$, $\vec{\omega}_i = \begin{bmatrix}0, 0, \omega\end{bmatrix}$ are the velocity and angular velocity of the rigid body. $\mathbf{r_i} = \mathbf{p_{contact}} - \mathbf{p_{cm}}$ is the offset of the contact point compared to the position of the center of mass

In 2D, the cross product simplifies to

$$\vec{\omega}_i \times \mathbf{r}_i = \begin{bmatrix} -\omega r_y \\ \omega r_x \\ 0 \end{bmatrix} \tag{5.13}$$

From there, the relative velocity of two rigid bodies A and B can be calculated as

$$\mathbf{v_{rel}} = \mathbf{v_{p,B}} - \mathbf{v_{p,A}} \tag{5.14}$$

To make it easier to calculate the impulse as well as the friction that can be applied onto the objects, we can transformed the relative velocity onto a new contact space using a matrix

$$\mathbf{v_c} = \begin{bmatrix} v_n \\ v_t \end{bmatrix} = \begin{bmatrix} \mathbf{v_{rel}} \cdot \mathbf{n} \\ \mathbf{v_{rel}} \cdot \mathbf{t} \end{bmatrix} \tag{5.15}$$

Where $\mathbf{n}, \mathbf{t}$ are the collision normal and tangent, v_n and v_t are the relative velocities along the normal (used to resolve collision) and tangent (used to handle friction).

Next we need to find the desired change in velocity after the collision occurs.

$$\Delta v_n = -(1 + e)v_n \tag{5.16}$$

Where v_n is the relative velocities along the normal direction and e is the coefficient of restitution. Next we need to work out the impulse to apply to each objects to change its current velocities by the desired velocity change.

$$P_n = \frac{\Delta v_n}{K_n} \tag{5.17}$$

This equation converts the desired change in velocity above into an impulse to apply to each objects. K_n is the effective inverse mass along the normal direction, which is calculated as

$$K_n = (m_A^{-1} + m_B^{-1}) + \frac{(\mathbf{r_A} \times \mathbf{n})^2}{I_A} + \frac{(\mathbf{r_B} \times \mathbf{n})^2}{I_B} \tag{5.18}$$

Where m_A,m_B is the mass of object A and B, I_A,I_B is the inertia of object A and B. $\mathbf{r_A}, \mathbf{r_B}$ is the offset of the contact position from the position of object A and B center of mass.

For friction, we can work out the friction impulse the same way as the collision impulse but instead of using the normal, we use the tangent.

$$\begin{gathered} K_t = (m_A^{-1} + m_B^{-1}) + \frac{(\mathbf{r_A} \times \mathbf{t})^2}{I_A} + \frac{(\mathbf{r_B} \times \mathbf{t})^2}{I_B} \\ P_t = \frac{v_t}{K_t} \end{gathered} \tag{5.19}$$

For friction, we need to find the maximum amount of static friction that the object can resist before it start moving. If the friction applied is greater than the maximum static friction then dynamic friction will be applied

$$\begin{gathered} P_{t,\max} = \mu_s |P_n| \\ \text{If } |P_t| > P_{t,\max}, \quad \text{then } P_t = \operatorname{sgn}(P_t)\mu_d |P_n| \end{gathered} \tag{5.20}$$

Where μ_s is the coefficient of static friction, μ_d is the coefficient of dynamic friction and P_t, P_n are the tangent and normal impulses.

We can derive the final impulse vector from the results of the equations above and transformed it from contact space to world space.

$$\mathbf{P} = \begin{bmatrix} P_x \\ P_y \end{bmatrix} = \begin{bmatrix} n_x, t_x \\ n_y, t_y \end{bmatrix} \begin{bmatrix} P_x \\ P_y \end{bmatrix} \tag{5.21}$$

Now that we have calculated the impulse, we can use the impulse to calculate the change in linear and angular velocity

$$\begin{gathered} \Delta \mathbf{v_A} = -m_A^{-1} \mathbf{P} \\ \Delta \omega_A = -I^{-1} (\mathbf{r_A} \times \mathbf{P}) \\ \Delta \mathbf{v_B} = m_B^{-1} \mathbf{P} \\ \Delta \omega_B = I^{-1} (\mathbf{r_B} \times \mathbf{P}) \end{gathered} \tag{5.22}$$

Where m_A, I_A, m_B, I_B is the mass and inertia of object A and B, $\mathbf{P}$ is the impulse, $\mathbf{r_A}$, $\mathbf{r_B}$ is the relative offset of the contact point from the center of mass of object A and B.

The linear and angular velocity change can then be applied to each objects

$$\begin{gathered} \mathbf{v_A} = \mathbf{v_A} + \Delta \mathbf{v_A} \\ \omega_A = \omega_A + \Delta \omega_A \\ \mathbf{v_B} = \mathbf{v_B} + \Delta \mathbf{v_B} \\ \omega_B = \omega_B + \Delta \omega_B \end{gathered} \tag{5.23}$$

5.6 Collision resolution (Interpenetration solver)

Resolving interpenetration is much simpler than resolving velocity. Instead of simply moving objects apart just enough so that they don't collide like before, we both move and rotate the objects in order to achieve the most realistic results.

To do this, we make use of the effective inverse mass K_A, K_B calculated from the velocity solver. We combined the two to make a total effective inverse mass

$$K_{total} = K_A + K_B \tag{5.24}$$

We use the total effective inverse mass as a ruler to measure how much we should shift each objects. The formula to shift the object's position and rotation is

$$\begin{aligned} \Delta x_A &= d\frac{m_A^{-1}}{K_{total}} \\ \Delta\theta_A &= d\frac{(r_{A,x}n_y - r_{A,y}n_x)^2}{K_{total}I_A} \\ \Delta x_B &= -d\frac{m_B^{-1}}{K_{total}} \\ \Delta\theta_B &= -d\frac{(r_{A,x}n_y - r_{B,y}n_x)^2}{K_{total}I_B} \end{aligned} \tag{5.25}$$

Where $\mathbf{r_A} = \left[r_{A,x}, r_{A,y}\right], \mathbf{r_B} = \left[r_{B,x}, r_{B,y}\right]$ is the relative offset of the contact point from the center of mass of object A and B. $\mathbf{n} = \left[n_x, n_y\right]$ is the collision normal and d is the penetration depth of the contact point.

These position and rotation changes can be applied to each object through the equation

$$\begin{aligned} \mathbf{x_A} &= \mathbf{x_A} + \Delta x_A \mathbf{n} \\ \theta_A &= \theta_A + \frac{\Delta\theta_A}{r_{A,x}n_y - r_{A,y}n_x} \\ \mathbf{x_B} &= \mathbf{x_B} + \Delta x_B \mathbf{n} \\ \theta_B &= \theta_B + \frac{\Delta\theta_B}{r_{B,x}n_y - r_{B,y}n_x} \end{aligned} \tag{5.26}$$

Chapter 6

Jacobian matrix and constraint handling

6.1 Limitations of the current engine

Up to this point, the engine is already capable of a lot of things, simulating collision between rigid bodies, and interactions using forces and springs. However, the collision system is still quite unstable, especially when stacking objects on top of each other or handling extreme mass ratio. The engine also can not handle constrained systems like distance constraint, revolute constraint, ...

The most common fix for this is to use the Jacobian, this is a construct to handle and solve constrained systems and can entirely replace the current force generators and contact system with a more stable and expandable one.

6.2 Jacobian matrix definition and degrees of freedom

The Jacobian matrix is defined mathematically as

$$\mathbf{J} = \begin{bmatrix} \frac{\partial C_1}{\partial x_1} & \frac{\partial C_1}{\partial y_1} & \frac{\partial C_1}{\partial \theta_1} & \cdots & \frac{\partial C_1}{\partial x_n} & \frac{\partial C_1}{\partial y_n} & \frac{\partial C_1}{\partial \theta_n} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \frac{\partial C_m}{\partial x_1} & \frac{\partial C_m}{\partial y_1} & \frac{\partial C_m}{\partial \theta_1} & \cdots & \frac{\partial C_m}{\partial x_n} & \frac{\partial C_m}{\partial y_n} & \frac{\partial C_m}{\partial \theta_n} \end{bmatrix} \tag{6.1}$$

Each row in the Jacobian matrix represent a single constraint in the system, each column in the matrix represent a single degree of freedom of a single object in the system.

An object in 2D have 3 degrees of freedom, x is the object's freedom to move on the x axis, y is the object's freedom to move on the y axis and θ is the object's freedom to rotate.

$$\text{DOF} = \begin{bmatrix} x & y & \theta \end{bmatrix} \tag{6.2}$$

A constraint restrict the object's movement on one to all three degrees of freedom. The Jacobian matrix basically represents how changing the object's degree of freedom affect the constraint that is applied on the object. In physics engine programming, constructing an entire matrix is unnecessary,

since a constraint is defined to only affect two objects in the system, constructing the entire matrix will leave many redundant areas with 0 (objects not related to the constraint will not affect it and the partial derivative of its DOF with the constraint will result in 0). Instead, it is better to construct a Jacobian row, consisting of only two objects and their respective degrees of freedom.

$$\mathbf{J_{row}} = \begin{bmatrix} \frac{\partial C}{\partial x_A} & \frac{\partial C}{\partial y_A} & \frac{\partial C}{\partial \theta_A} & \frac{\partial C}{\partial x_B} & \frac{\partial C}{\partial y_B} & \frac{\partial C}{\partial \theta_b} \end{bmatrix} \tag{6.3}$$

Where C is the constraint equation, $\begin{bmatrix} x_A & y_A & \theta_A \end{bmatrix}$ are the degrees of freedom of object A and $\begin{bmatrix} x_B & y_B & \theta_B \end{bmatrix}$ are the degrees of freedom of object B. For simplicity sake, since we will only be using the Jacobian row, I will denote $\mathbf{J}$ as the jacobian row from now on.

6.3 The constraint equation

A constraint restricts two objects degree of freedom, the equation for the constraint is:

$$C(x_A, y_A, \theta_A, x_B, y_B, \theta_B) \tag{6.4}$$

The system is correct when the positions and rotation of object A and object B satisfy the constraint equation. However, this is not really helpful for our needs, what we want to find is to find the change in constraint over time (this will help us adjust the position, rotation and make sure that the constraint is always enforced). To find the change in the constraint overtime, we take the derivative of it over time, using the chain rule for partial derivative we get:

$$\frac{dC}{dt} = (\frac{\partial C}{\partial x_A}\frac{dx_A}{dt}) + (\frac{\partial C}{\partial y_A}\frac{dy_A}{dt}) + (\frac{\partial C}{\partial \theta_A}\frac{d\theta_A}{dt}) + (\frac{\partial C}{\partial x_B}\frac{dx_B}{dt}) + (\frac{\partial C}{\partial y_B}\frac{dy_B}{dt}) + (\frac{\partial C}{\partial \theta_B}\frac{d\theta_B}{dt}) \tag{6.5}$$

The derivative of x, y, θ overtime is the object's linear and angular velocity so we have

$$\frac{dC}{dt} = (\frac{\partial C}{\partial x_A}v_{Ax}) + (\frac{\partial C}{\partial y_A}v_{Ay}) + (\frac{\partial C}{\partial \theta_A}\omega_A) + (\frac{\partial C}{\partial x_B}v_{Bx}) + (\frac{\partial C}{\partial y_B}v_{By}) + (\frac{\partial C}{\partial \theta_B}\omega_B) \tag{6.6}$$

Isolating the velocities from the partial derivative we get

$$\frac{dC}{dt} = \begin{bmatrix} \frac{\partial C}{\partial x_A} & \frac{\partial C}{\partial y_A} & \frac{\partial C}{\partial \theta_A} & \frac{\partial C}{\partial x_B} & \frac{\partial C}{\partial y_B} & \frac{\partial C}{\partial \theta_b} \end{bmatrix} \begin{bmatrix} v_{Ax} \\ v_{Ay} \\ \omega_A \\ v_{Bx} \\ v_{By} \\ \omega_B \end{bmatrix} \tag{6.7}$$

It can be seen that the first matrix is the Jacobian row $\mathbf{J}$ and the second matrix is a matrix of velocities $\mathbf{V}$. So our final equation will be

$$\frac{dC}{dt} = \mathbf{JV} \tag{6.8}$$

6.4 The distance constraint

The most basic constraint is the distance constraint, it simply prevent two objects from getting closer together or moving further apart. The equation for the constraint is

$$C = (x_B - x_A)^2 + (y_B - y_A)^2 - L^2 = 0 \tag{6.9}$$

Where $\mathbf{A}(x_A, y_A)$, $\mathbf{B}(x_B, y_B)$ is the position of the anchor point on object A and B in world space. These points are calculated as

$$\begin{aligned}
\mathbf{r_A} = \begin{bmatrix} r_{Ax} \\ r_{Ay} \end{bmatrix} &= \begin{bmatrix} \cos\theta_A & -\sin\theta_A \\ \sin\theta_A & \cos\theta_A \end{bmatrix} \begin{bmatrix} r_{local,A,x} \\ r_{local,A,y} \end{bmatrix} \\
\mathbf{r_B} = \begin{bmatrix} r_{Bx} \\ r_{By} \end{bmatrix} &= \begin{bmatrix} \cos\theta_B & -\sin\theta_B \\ \sin\theta_B & \cos\theta_B \end{bmatrix} \begin{bmatrix} r_{local,B,x} \\ r_{local,B,y} \end{bmatrix} \\
\mathbf{A} = \begin{bmatrix} p_{A,x} \\ p_{A,y} \end{bmatrix} &= \begin{bmatrix} x_A + r_{Ax} \\ y_A + r_{Ay} \end{bmatrix} \\
\mathbf{B} = \begin{bmatrix} p_{B,x} \\ p_{B,y} \end{bmatrix} &= \begin{bmatrix} x_B + r_{Bx} \\ y_B + r_{By} \end{bmatrix}
\end{aligned} \tag{6.10}$$

Where $\mathbf{P_A}(p_{A,x}, p_{A,y})$ $\mathbf{P_B}(p_{B,x}, p_{B,y})$ are the global position of the center of mass of object A and B. $\mathbf{R_{localA}}(r_{local,A,x}, r_{local,A,y})$ $\mathbf{R_{localB}}(r_{local,B,x}, r_{local,B,y})$ are the position of the anchor points in local space. θ_A θ_B are the rotation of object A and B. L is the desired distance between the two anchor points

To enforce the constraint, we have $\frac{dC}{dt} = 0 \implies \mathbf{JV} = 0$ Since the object's velocities is already know, we need to find the Jacobian $\mathbf{J}$. Let $\mathbf{d} = \mathbf{B} - \mathbf{A}$, the constraint equation becomes

$$C = d_x^2 + d_y^2 - L^2 = 0 \tag{6.11}$$

We have:

$$\begin{aligned}
\frac{\partial C}{\partial x_A} &= \frac{\partial}{\partial x_A}(d_x^2) + \frac{\partial}{\partial x_A}(d_y^2) \\
&= 2d_x \frac{\partial}{\partial x_A}(d_x) + 2d_y \frac{\partial}{\partial x_A}(d_y)
\end{aligned}$$

$$\begin{aligned}
&\frac{\partial}{\partial x_A}(d_x) = \frac{\partial}{\partial x_A}(x_B + r_{Bx} - x_A - r_{Ax}) = -1 \\
&\frac{\partial}{\partial x_A}(d_y) = \frac{\partial}{\partial x_A}(y_B + r_{By} - y_A - r_{Ay}) = 0 \\
&\implies \frac{\partial C}{\partial x_A} = -2d_x
\end{aligned} \tag{6.12}$$

Similarly, deriving $\frac{\partial C}{\partial y_A}$ gets $\frac{\partial C}{\partial y_A} = -2d_y$. For $\frac{\partial C}{\partial \theta_A}$ some extra steps are needed. Above we have the formula to calculate the relative offset

$$\mathbf{r} = \begin{bmatrix} r_x \\ r_y \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} r_{local,x} \\ r_{local,y} \end{bmatrix} \implies \begin{cases} r_x = r_{local,x}\cos\theta - r_{local,x}\sin\theta \\ r_y = r_{local,y}\sin\theta + r_{local,y}\cos\theta \end{cases} \tag{6.13}$$

Differentiating this equation with θ we get

$$\begin{aligned} \frac{\partial r_x}{\partial \theta} &= -r_{local,x}\sin\theta - r_{local,y}\cos\theta = -r_y \\ \frac{\partial r_y}{\partial \theta} &= r_{local,x}\cos\theta - r_{local,y}\sin\theta = r_x \end{aligned} \tag{6.14}$$

Using this we can derive the result of $\frac{\partial C}{\partial \theta_A}$ as

$$\begin{aligned} \frac{\partial C}{\partial \theta_A} &= \frac{\partial}{\partial \theta_A}(d_x^2) + \frac{\partial}{\partial \theta_A}(d_y^2) \\ &= 2d_x\frac{\partial}{\partial \theta_A}(d_x) + 2d_y\frac{\partial}{\partial \theta_A}(d_y) \end{aligned}$$

$$\begin{aligned} \frac{\partial}{\partial \theta_A}(d_x) &= \frac{\partial}{\partial \theta_A}(x_B + r_{Bx} - x_A - r_{Ax}) \\ &= -\frac{\partial}{\partial \theta_A}(r_{Ax}) = r_{Ay} \\ \frac{\partial}{\partial \theta_A}(d_y) &= \frac{\partial}{\partial \theta_A}(y_B + r_{By} - y_A - r_{Ay}) \\ &= -\frac{\partial}{\partial \theta_A}(r_{Ay}) = -r_{Ax} \\ \implies \frac{\partial C}{\partial \theta_A} &= -2(r_{Ax}d_y - r_{Ay}d_x) \end{aligned} \tag{6.15}$$

Doing the same for B we get

$$\frac{\partial C}{\partial x_B} = 2d_x \quad \frac{\partial C}{\partial y_B} = 2d_y \quad \frac{\partial C}{\partial \theta_B} = 2(r_{Bx}d_y - r_{By}d_x) \tag{6.16}$$

So the final Jacobian row is:

$$\mathbf{J} = \begin{bmatrix} -2d_x & -2d_y & -2(r_{Ax}d_y - r_{Ay}d_x) & 2d_x & 2d_y & 2(r_{Bx}d_y - r_{By}d_x) \end{bmatrix} \tag{6.17}$$

Let $\mathbf{d} = \begin{bmatrix} d_x, d_y \end{bmatrix}$ and $\mathbf{n} = \frac{\mathbf{d}}{||\mathbf{d}||}$ we can simplify the Jacobian matrix to

$$\mathbf{J} = \begin{bmatrix} -\mathbf{n} & -(\mathbf{r_A} \times \mathbf{n}) & \mathbf{n} & (\mathbf{r_B} \times \mathbf{n}) \end{bmatrix} \tag{6.18}$$

6.5 Constraint resolution using Projected Gauss Seidel (PGS)

The constraint applies a force to the objects involved in order to keep them in equilibrium. Newton second law states that $\mathbf{F} = m\mathbf{a}$. We can rearrange this and solve for the acceleration which can be used to derive the velocity

$$\begin{aligned} \mathbf{a} &= \mathbf{M^{-1}F_{constraint}} \\ \implies \mathbf{v_{new}} &= \mathbf{v_{old}} + \mathbf{M^{-1}F_{constraint}} \end{aligned} \tag{6.19}$$

Where $\mathbf{F_{constraint}}$ is the force of the constraint, and $\mathbf{M^{-1}}$ is the inverse mass matrix of the system, which is defined as

$$\mathbf{M^{-1}} = \begin{bmatrix} \frac{1}{m_A} & 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{m_A} & 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{I_A} & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{m_B} & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{m_B} & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1}{I_B} \end{bmatrix} \tag{6.20}$$

Where $m_A \quad m_B$ is the mass of object A and B and $I_A \quad I_B$ is the inertia of object A and B.

Deriving from D'Alembert law of virtual work, we can get an equation combining the Jacobian with the constraint force

$$\mathbf{F_{constraint}} = \mathbf{J}^T\lambda \tag{6.21}$$

Where λ is the Lagrange multiplier which determines how hard the constraint needs to be to keep the objects in equilibrium. From this we can derive an equation for a change in velocity, which can be used to solve for the new velocity of each objects later

$$\begin{aligned} \mathbf{v_{new}} &= \mathbf{v_{old}} + \mathbf{M^{-1}J}^T\lambda \\ \Delta\mathbf{v} &= \mathbf{v_{new}} - \mathbf{v_{old}} = \mathbf{M^{-1}J}^T\lambda \end{aligned} \tag{6.22}$$

We want the constraint to be consistent across time so $\frac{dC}{dt} = \mathbf{JV} = 0$. Multiplying both side by $\mathbf{J}$, we can derive an equation for λ

$$\begin{aligned} \mathbf{Jv_{new}} &= \mathbf{J}(\mathbf{v_{old}} + \mathbf{M^{-1}J}^T\lambda) = 0 \\ -\mathbf{Jv_{old}} &= \mathbf{JM^{-1}J}^T\lambda \\ \lambda &= \frac{-\mathbf{Jv_{old}}}{\mathbf{JM^{-1}J}^T} \end{aligned} \tag{6.23}$$

However, to further stabilize the engine, most engines use Baumgarte Stabilization to add an additional term when calculating lambda to correct certain constraint violations (Ex: object sinking into each other). It is defined as

$$B = -\frac{\beta}{\Delta t} C(\mathbf{x}) \tag{6.24}$$

Where $C(\mathbf{x})$ is the constraint error (Ex: penetration distance), β is a hyperparameter that can be tuned, for my engine, I set it as $\beta = 0.2$. We apply the stabilization when calculating lambda

$$\lambda = \frac{B - \mathbf{J}\mathbf{v}_{\mathbf{old}}}{\mathbf{J}\mathbf{M}^{-\mathbf{1}}\mathbf{J}^T} \tag{6.25}$$

Finally, we can derive the equation for the change in velocity as

$$\Delta\mathbf{v} = \mathbf{M}^{-\mathbf{1}}\mathbf{J}^T \frac{B - \mathbf{J}\mathbf{v}_{\mathbf{old}}}{\mathbf{J}\mathbf{M}^{-\mathbf{1}}\mathbf{J}^T} \tag{6.26}$$

The Projected Gauss Seidel (PGS) algorithm is an iterative solution that loops through every constraint, calculates lambda and apply delta velocity repeatedly to converge the lambda value. This is an algorithm that is used extensively in many engine and was popularize by the Box2D physics engine.

The PGS algorithm is designed to solve Linear Complementarity Problem, it basically find a λ value that satisfies all constraints by iteratively looping through them.

The algorithm is as follows:

- Start by preparing the data, loop through each constraints and calculate the Jacobian $\mathbf{J}$ and the effective mass $\mathbf{J}\mathbf{M}^{-\mathbf{1}}\mathbf{J}^T$
- The algorithm will loop through all the constraints n times (the value of n I use tend to be around 20 to 30) to slowly converge the lambda of each constraint. For each constraint C_i:
  - Calculate λ_{raw} using the formula above
  - Calculate $\lambda_{i,new}$ as $\lambda_{new} = \lambda_{i,old} + \lambda_{raw}$ where $\lambda_{i,old}$ is the stored lambda of this constraint from the previous iteration
  - We clamp $\lambda_{i,new}$ to specific ranges (this is especially useful later for inequality constraints). $\lambda_{i,new} = \text{clamp}(\lambda_{i,new}, \lambda_{i,min}, \lambda_{i,max})$
  - We have $\Delta\lambda = \lambda_{i,new} - \lambda_{i,old}$. This is a small impulse increment that is apply to slowly converge to the desired λ value.
  - From there, calculate the change in velocity as $\Delta\mathbf{v} = \mathbf{M}^{-1}\mathbf{J}_i^T\Delta\lambda$. After that apply the change in velocity (both linear and angular) to each objects in the constraint

As an example, I will derive the calculation for the distance constraint. We can calculate the effective mass as

$$\begin{aligned}\mathbf{JM^{-1}J}^T &= \begin{bmatrix}-\mathbf{n} & -(\mathbf{r_A}\times\mathbf{n}) & \mathbf{n} & (\mathbf{r_B}\times\mathbf{n})\end{bmatrix}\begin{bmatrix}\frac{1}{m_A} & 0 & 0 & 0 & 0 & 0\\ 0 & \frac{1}{m_A} & 0 & 0 & 0 & 0\\ 0 & 0 & \frac{1}{I_A} & 0 & 0 & 0\\ 0 & 0 & 0 & \frac{1}{m_B} & 0 & 0\\ 0 & 0 & 0 & 0 & \frac{1}{m_B} & 0\\ 0 & 0 & 0 & 0 & 0 & \frac{1}{I_B}\end{bmatrix}\begin{bmatrix}-\mathbf{n}\\ -(\mathbf{r_A}\times\mathbf{n})\\ \mathbf{n}\\ \mathbf{r_B}\times\mathbf{n}\end{bmatrix}\\ &= \frac{1}{m_A}+\frac{(\mathbf{r_A}\times\mathbf{n})^2}{I_A}+\frac{1}{m_B}+\frac{(\mathbf{r_B}\times\mathbf{n})^2}{I_B}\end{aligned} \tag{6.27}$$

We can also calculate the Baumgarte Stabilization term as

$$B = \frac{\beta}{\Delta t}(d_{current} - d_{prev}) \tag{6.28}$$

Where $d_{current}$ is the current distance between the two attach points (in world space) and d_{prev} is the distance between the two points in the previous frame. From there the PGS algorithm can calculate λ and applies the correct velocity to each object.

6.6 Inequality constraints and soft constraints

For inequality constraint, we simply change the constraint equation from an equality to an inequality.

$$\mathbf{Jv} < 0 \tag{6.29}$$

To do this, it is as simple as changing the values of λ_{min} and λ_{max} and the PGS solver will handle the rest. For an equality constraint, the values are $\lambda_{min} = -\infty \quad \lambda_{max} = +\infty$. For a simple distance constraint, we can make it retractable or extendable by modifying the lambda values

$$\begin{aligned}\text{Retractable: } &\lambda_{min} = 0 \quad \lambda_{max} = +\infty\\ \text{Extendable: } &\lambda_{min} = -\infty \quad \lambda_{max} = 0\\ \text{No constraint: } &\lambda_{min} = 0 \quad \lambda_{max} = 0\end{aligned} \tag{6.30}$$

From this we can see that decreasing λ_{min} prevent objects from getting close together , increase λ_{max} prevent objects from getting further apart. We can also dynamically change these values to control how fast the system corrects itself, this is especially crucial for implementing friction (which will be discussed in the next chapter) where λ_{max} will scales with the normal force.

For soft constraint, the way we do it is to add a new coefficient when calculating λ. Soft constraint can be very useful when simulating things like springs, constraint can allow for much more stable spring simulation than the previous approach of using forces. The new equation for λ is

$$\lambda = \frac{B - \mathbf{J}\mathbf{v}_{\text{old}}}{\mathbf{J}\mathbf{M}^{-1}\mathbf{J}^T + \text{CFM}} \tag{6.31}$$

Where CFM is the softness constraint force mixing. This is calculated differently for each type of constraint, however for a soft distance constraint (spring), it is calculated as

$$\text{CFM} = \frac{1}{\Delta t(k + d\Delta t)} \tag{6.32}$$

Where k is the stiffness of the spring and d is the damping coefficient of the spring. Additionally, we can embed the softness directly to the effective mass and bias

$$\begin{aligned} K &= K + \text{CFM} \\ B &= \frac{kd\Delta t}{\Delta t}(d_{current} - d_{prev}) \end{aligned} \tag{6.33}$$

Chapter 7

Contact constraint and general 3 DOF constraint

7.1 Contact constraint

The contact constraint is divided into two different constraint, the normal constraint (apply along the collision normal) and the friction constraint (apply along the collision tangent). For these constraints, I will simply be giving the final derived equation since the process of deriving the Jacobian is consistent across all constraints.

For the normal constraint, the Jacobian matrix is quite similar to the distance constraint (since its purpose is to separate the objects, it is basically a distance constraint that is extendable)

$$\mathbf{J_n} = \begin{bmatrix} -\mathbf{n} & -(\mathbf{r_A} \times \mathbf{n}) & \mathbf{n} & (\mathbf{r_B} \times \mathbf{n}) \end{bmatrix} \tag{7.1}$$

Where $\mathbf{n}$ is the collision normal and $\mathbf{r_A}$ $\mathbf{r_B}$ are the contact point offset from the center of mass of the two objects. Because it is simply a distance constraint that is extendable, the λ clamping also reflect that

$$\lambda_{min} = -\infty \quad \lambda_{max} = 0 \tag{7.2}$$

The effective mass is also calculated similar to the distance constraint

$$\mathbf{J_n M^{-1} J_n}^T = \frac{1}{m_A} + \frac{(\mathbf{r_A} \times \mathbf{n})^2}{I_A} + \frac{1}{m_B} + \frac{(\mathbf{r_B} \times \mathbf{n})^2}{I_B} \tag{7.3}$$

For the Baumgarte stabilization bias, we can calculate it based on the penetration depth

$$B_n = \frac{\beta}{\Delta t} \max(0, d - \text{slop}) \tag{7.4}$$

Where d is the penetration depth of the contact point (generated by the contact point generation algorithm), slop is added to ignore tiny changes due to floating point errors.

For friction, we simply change from the collision normal to the collision tangent.

$$\mathbf{J_f} = \begin{bmatrix} -\mathbf{t} & -(\mathbf{r_A} \times \mathbf{t}) & \mathbf{t} & (\mathbf{r_B} \times \mathbf{t}) \end{bmatrix}$$
$$\mathbf{J_f M^{-1} J_f}^T = \frac{1}{m_A} + \frac{(\mathbf{r_A} \times \mathbf{t})^2}{I_A} + \frac{1}{m_B} + \frac{(\mathbf{r_B} \times \mathbf{t})^2}{I_B} \tag{7.5}$$

Where $\mathbf{t} = \begin{bmatrix} -n_y & n_x & 0 \end{bmatrix}$ is the collision tangent.

For the friction constraint, the friction force is bounded by the coefficient of friction

$$\lambda_t \in \begin{cases} [-\mu_s \lambda_n, \mu_s \lambda_n] & \text{if } |\lambda_t| \leq \mu_s \lambda_n \\ [-\mu_d \lambda_n, \mu_d \lambda_n] & \text{if } |\lambda_t| > \mu_s \lambda_n \end{cases} \tag{7.6}$$

Where λ_n is the impulse λ of the normal constraint and μ_s, μ_d is the coefficient of static and dynamic friction. For friction, no stabilization is need so $B = 0$.

7.2 Contact caching

The contact constraint is removed at the end (or the beginning) of each frame and then recalculated for every objects. This is fine for quick collision where objects bounce off each other, however for collision that achieve stability (a box resting on ground or multiple boxes stacked on top of each other), recalculating lambda every frame can lead to random movement due to floating point error.

To solve this, we can store the previous frame lambda in a cache and then continue from the cache lambda during the next frame (if the collision is still active). To do this, we need a way to identify each collision, the way I do it is to assign a unique ID on each collision. The ID is calculated based on the geometric features of the collision (reference, incident edge,..) as well as the objects involved in the collision.

For the geometric features I store are

- Reference Edge index: The index of the reference edge in the edges array of the polygon
- Incident Edge index: The index of the incident edge in the edges array of the polygon
- Incident Edge origin: Tracks the endpoint of the incident edge this contact point descended from through clipping
- Clip plane ID: Tracks whether the vertex is unclipped, clipped by left plane or clipped by the right plane.

Before starting the PGS loop, we set the current lambda to the previous frame lambda (if it is cached). We also apply the previous frame lambda to the velocity to ensure that the solver begins at a physically meaningful state rather than at rest. Because of this, the solver only need to correct the slight difference (cause by floating point errors, instability,..) from the previous frame instead of having to recalculate everything.

7.3 3 DOF constraint: Weld, Revolute, Prismatic constraint

The revolute, prismatic and weld constraint are constraints that restrict one, two or all three of the object's degrees of freedom. The code for these a very similar to each other so I grouped them up as a 3 DOF constraint. Some physics engine group them together into a single constraint then simply disable, or enable a degree of freedom of the object. In my engine however, I prefer splitting them which allows for more direct control (for example visualizing the constraint or adding actuator to revolute constraint). For each constraint, as demonstrated in the distance and contact constraint, we will have to calculate three things: the Jacobian row, the effective mass, the Baumgarte stabilization bias

Restriction of each degrees of freedom require a separate Jacobian row in the Jacobian matrix, so restricting one degree of freedom would use one row, two will be two rows and three rows to restrict all freedom in movement.

For these constraint, we will still make use of the relative offset from the center of mass of the two objects $\mathbf{r_A}, \mathbf{r_B}$ where

$$\begin{aligned} \mathbf{r_A} &= \begin{bmatrix} r_{Ax} \\ r_{Ay} \end{bmatrix} = \begin{bmatrix} \cos\theta_A & -\sin\theta_A \\ \sin\theta_A & \cos\theta_A \end{bmatrix} \begin{bmatrix} r_{local,A,x} \\ r_{local,A,y} \end{bmatrix} \\ \mathbf{r_B} &= \begin{bmatrix} r_{Bx} \\ r_{By} \end{bmatrix} = \begin{bmatrix} \cos\theta_B & -\sin\theta_B \\ \sin\theta_B & \cos\theta_B \end{bmatrix} \begin{bmatrix} r_{local,B,x} \\ r_{local,B,y} \end{bmatrix} \end{aligned} \tag{7.7}$$

The first constraint we will look at is the revolute constraint which restricts the object's movement on the x and y axis, only allowing the objects to rotate. To do this we will use two Jacobian rows, one for x axis and the other for the y axis. This can be clearly seen when we look at the constraint equation

$$\begin{aligned} & \mathbf{C} = \mathbf{p_B} - \mathbf{p_A} = 0 \\ \iff & \mathbf{C} = (\mathbf{p_{B,local}} + \mathbf{r_B}) - (\mathbf{p_{A,local}} + \mathbf{r_A}) \\ \iff & \mathbf{C} = \begin{bmatrix} C_x \\ C_y \end{bmatrix} = \begin{bmatrix} (p_{B,local,x} + r_{Bx}) - (p_{A,local,x} + r_{Ax}) \\ (p_{B,local,y} + r_{By}) - (p_{A,local,y} + r_{Ay}) \end{bmatrix} \end{aligned} \tag{7.8}$$

Where $\mathbf{p_A}, \mathbf{p_B}$ is the position of object's A and B center of mass in world space and $\mathbf{p_{A,local}}, \mathbf{p_{B,local}}$ is the position of object's A and B center of mass in local space. Differentiating this equation with respect to time we can find the Jacobian rows for the x and y constraint.

$$\mathbf{J} = \begin{bmatrix} \mathbf{J_x} \\ \mathbf{J_y} \end{bmatrix} = \begin{bmatrix} -1 & 0 & -r_{Ay} & 1 & 0 & r_{By} \\ 0 & -1 & r_{Ax} & 0 & 1 & -r_{Bx} \end{bmatrix} \tag{7.9}$$

We can calculate the effective mass as

$$\begin{aligned} \mathbf{J_x M^{-1} J_x}^T &= \frac{1}{m_A} + \frac{1}{m_B} + \frac{r_{Ay}^2}{I_A} + \frac{r_{By}^2}{I_B} \\ \mathbf{J_y M^{-1} J_y}^T &= \frac{1}{m_A} + \frac{1}{m_B} + \frac{r_{Ax}^2}{I_A} + \frac{r_{Bx}^2}{I_B} \end{aligned} \tag{7.10}$$

Finally, we calculate the Baumgarte stabilization bias as

$$B_x = \frac{\beta}{\Delta t}(p_{Bx} - p_{Ax}) \qquad B_y = \frac{\beta}{\Delta t}(p_{By} - p_{By}) \tag{7.11}$$

The second constraint we will look at is the prismatic constraint which restricts the object's rotation and movement to a singular axis. This constraint is useful for simulating things like piston or moving platforms. Let $\mathbf{d} = \begin{bmatrix} d_x & d_y \end{bmatrix}$ is the direction vector that represents the axis the object can travel in (For my engine this is the direction between the center of mass of the two objects). The constraint equation of this constraint is

$$\begin{aligned} C_{linear} &= \mathbf{t} \cdot (\mathbf{p_B} - \mathbf{p_A}) = 0 \\ C_\theta &= \theta_B - \theta_A = 0 \end{aligned} \tag{7.12}$$

Where $\mathbf{t} = \begin{bmatrix} -d_y & d_x \end{bmatrix}$ is the perpendicular tangent axis from the direction vector. What this means is that the constraint restricts the object's linear movement such that it can only move along the $\mathbf{d}$ vector and can not move along the $\mathbf{t}$ vector. *Figure 7.1* shows the direction and tangent vector of an object under prismatic constraint.

Differentiating these two equations with respect to time we can find the Jacobian row for the linear and angular term

$$\mathbf{J} = \begin{bmatrix} \mathbf{J_{linear}} \\ \mathbf{J}_\theta \end{bmatrix} = \begin{bmatrix} -\mathbf{t} & -(\mathbf{r_A} \times \mathbf{t}) & \mathbf{t} & (\mathbf{r_B} \times \mathbf{t}) \\ 0 & -1 & 0 & 1 \end{bmatrix} \tag{7.13}$$

The effective mass for the prismatic constraint is calculated as

$$\begin{aligned} \mathbf{J_{linear}} \mathbf{M^{-1}} \mathbf{J_{linear}}^T &= \frac{1}{m_A} + \frac{(\mathbf{r_A} \times \mathbf{t})^2}{I_A} + \frac{1}{m_B} + \frac{(\mathbf{r_B} \times \mathbf{t})^2}{I_B} \\ \mathbf{J}_\theta \mathbf{M^{-1}} \mathbf{J}_\theta^T &= \frac{1}{I_A} + \frac{1}{I_B} \end{aligned} \tag{7.14}$$

For the bias, it is calculated as

$$\begin{aligned} B_{linear} &= \frac{\beta}{\Delta t}(\mathbf{d} \cdot \mathbf{t}) \\ B_\theta &= \frac{\beta}{\Delta t}(\theta_B - \theta_A) \end{aligned} \tag{7.15}$$

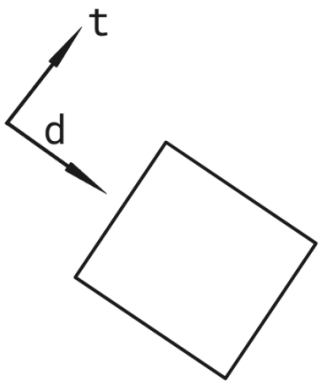

Figure 7.1: Prismatic constraint direction

The final constraint we will look at is the weld constraint. This constraint restricts all degrees of freedom of the object, it is useful to create fixed structures that need to handle physics as well. Because we are limiting all three degrees of freedom, 3 Jacobian rows will be needed. The constraint equation for the weld constraint is

$$\mathbf{C} = \begin{bmatrix} C_x \\ C_y \\ C_\theta \end{bmatrix} = \begin{bmatrix} p_{Bx} - p_{Ax} \\ p_{By} - p_{Ay} \\ \theta_B - \theta_A - \theta_{offset} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \tag{7.16}$$

Where θ_{offset} is the rotational offset between the two objects after it is welded together. Differentiating this with respect to time we can once again find the Jacobian rows used to represent this constraint.

$$\mathbf{J} = \begin{bmatrix} \mathbf{J_x} \\ \mathbf{J_y} \\ \mathbf{J}_\theta \end{bmatrix} = \begin{bmatrix} -1 & 0 & -r_{Ay} & 1 & 0 & r_{By} \\ 0 & -1 & r_{Ax} & 0 & 1 & -r_{Bx} \\ 0 & 0 & -1 & 0 & 0 & 1 \end{bmatrix} \tag{7.17}$$

We can see that the Jacobian matrix for the weld constraint is just the $\mathbf{J_x}, \mathbf{J_y}$ term of the revolute constraint and the $\mathbf{J}_\theta$ of the prismatic constraint. For the effective mass, it is also a combination of the two constraints

$$\begin{aligned} \mathbf{J_x M^{-1} J_x}^T &= \frac{1}{m_A} + \frac{1}{m_B} + \frac{r_{Ay}^2}{I_A} + \frac{r_{By}^2}{I_B} \\ \mathbf{J_y M^{-1} J_y}^T &= \frac{1}{m_A} + \frac{1}{m_B} + \frac{r_{Ax}^2}{I_A} + \frac{r_{Bx}^2}{I_B} \\ \mathbf{J}_\theta \mathbf{M^{-1}} \mathbf{J}_\theta^T &= \frac{1}{I_A} + \frac{1}{I_B} \end{aligned} \tag{7.18}$$

Finally, the bias for the Baumgarte stabilization is

$$\begin{aligned} B_x &= \frac{\beta}{\Delta t}(p_{Bx} - p_{Ax}) \\ B_y &= \frac{\beta}{\Delta t}(p_{By} - p_{By}) \\ B_\theta &= \frac{\beta}{\Delta t}(\theta_B - \theta_A) \end{aligned} \tag{7.19}$$

Chapter 8

Soft body physics

8.1 Mass Spring System

There are several options of doing soft body physics, the most popular options are mass spring system or position based dynamics (PBD / XPBD). The easiest method that we can directly integrate with the constraint solver we've made in the past two chapters is the mass spring system. This method simply links point masses (objects with only position and velocity and no rotation, angular velocity) together with soft distance constraints (spring constraints).

This will be the method I will use in this chapter, although I will be replacing it with a more robust system in the later chapters since mass spring systems have a lot of stability problems.

There are a lot of ways to attach springs to create a stable soft bodies, there are three sets of springs that I used in my soft body. This can be seen in *Figure 8.1*

- Structure springs: springs that connect the point mass to form the outer structure of the shape
- Spoke springs: springs that connect the center of the object to each point mass
- Shear springs: springs that connect every 2 mass points

8.2 Point mass and mass aggregate

A mass aggregate is simply a list of point masses that make up a soft body object. A point mass differ from a rigid body in that it does not have a rotational property, only position and linear velocity. The mass of the point mass is a fraction of the mass aggregate, for an uniformly divided soft body, it is calculated as

$$m_i = \frac{m}{n} \tag{8.1}$$

Where m_i is the mass of the point mass, m is the mass of the entire soft body, n is the amount of point masses in the mass aggregate. We also sync the position of the point mass with the vertex of the soft body. The position and velocity of the entire soft body is the position and velocity of

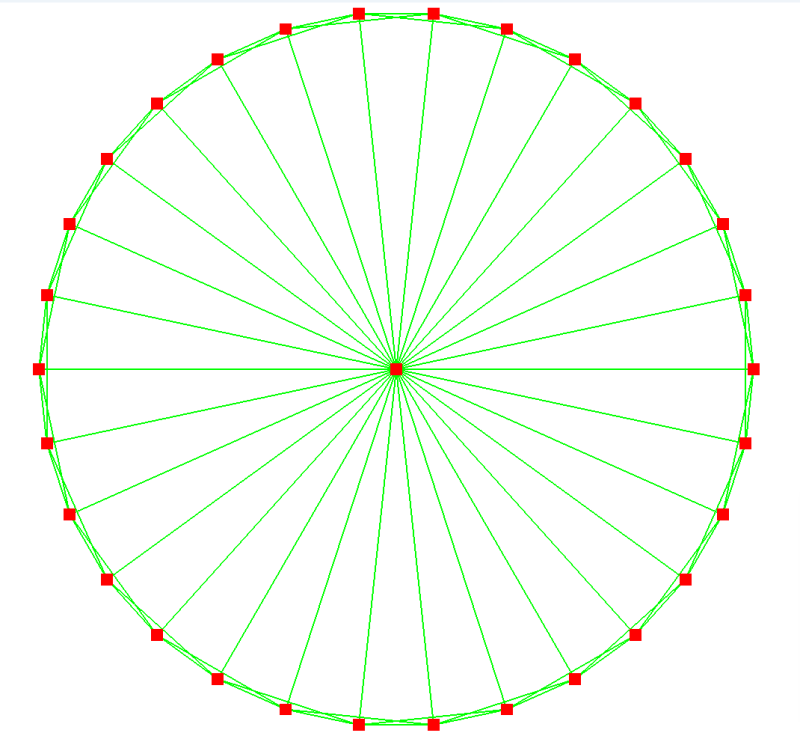

Figure 8.1: Soft body mass spring system

the center of mass of the object. For a uniformly distributed mass object, the center of mass is also the geometric center and the center that the spoke springs are connected to

8.3 Soft body collision detection and resolution

In rigid body collision, we detect collision by utilizing the separating axis theorem, projecting each vertex of both objects onto the axis. We could do the same for soft body collision however this would be both inefficient and prone to errors. One of the caveat of SAT is that it only works on convex polygons. This is a problem because the soft body might start out as a convex polygon but during deformation it could become concave which could lead to false positives or negatives.

To combat this and to make full use of the mass aggregate system, we can detection collision by checking each vertex of the first polygon with each edge of the second polygon and then check each vertex of the second polygon with each edge of the first polygon. For my engine, I create a helper function that takes in a point mass of a soft body and compare it with the edges of another soft body. The algorithm is as follows:

- Check if the point mass is inside the other polygon using a ray cast algorithm
- Use projection to find the closest edge to the point mass

- Find the collision normal, penetration as well as contact points
- Generate a contact constraint

The first step is to check if the point mass is inside the other polygon or not. This is important because the second step, the projection step only works when the point mass is outside of the polygon, if it is inside we will have to use another method, I will touch on this later.

To check if the point is inside of the other polygon or not, we cast an imaginary ray from the origin to the point, if the number of intersection is an odd number, the point is inside, if it is even, the point is outside. For a point $\mathbf{p}(x, y)$ and an edge with ends $\mathbf{A}(x_A, y_A)$ and $\mathbf{B}(x_B, y_B)$, the ray cast intersect the edge when

$$\begin{gathered}(y_A > y) \neq (y_B > y) \\ x < x_A + \frac{(x_B - x_A)(y - y_A)}{y_B - y_A}\end{gathered} \tag{8.2}$$

Figure 8.2 shows how ray casting can be used to check if a point is inside of a polygon. P_1 has an odd number of intersection so it is inside while P_2 has an even number of intersection so it is outside

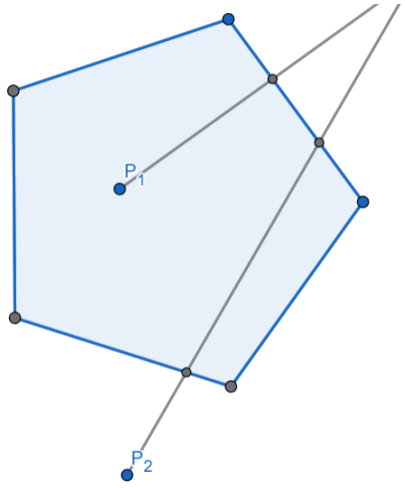

Figure 8.2: Ray cast to detect point inside polygon

The next part of the collision detection algorithm is finding the closest edge to the point mass. This will only be optimal when the point mass is outside of the polygon. To do this, we still iterate over all the edges of the polygon. Let the point mass position be $\mathbf{C}$ and the position of the end point of the edge $\mathbf{A}$ and $\mathbf{B}$. We can first calculate the projection factor t

$$\begin{gathered}\mathbf{AB} = \mathbf{B} - \mathbf{A} \\ \mathbf{AC} = \mathbf{C} - \mathbf{A} \\ t = \text{Clamp}(\frac{\mathbf{AC} \cdot \mathbf{AB}}{||\mathbf{AB}||^2}, 0, 1)\end{gathered} \tag{8.3}$$

If $t = 0$, the closest point is exactly at $\mathbf{A}$, if $t = 1$, the closest point is exactly at $\mathbf{B}$, else the point lies somewhere on the edge. The exact position of the point is calculated as

$$\mathbf{P}_{closest} = \mathbf{A} + t\mathbf{AB} \tag{8.4}$$

After that what we want to do is calculate the distance from the closest point to the point mass

$$d = ||\mathbf{P_{closest}} - \mathbf{C}|| \tag{8.5}$$

The final edge will be the edge with the smallest distance, from this final edge, we can move onto the next step and calculate the collision normal, penetration and contact point.

$$\begin{aligned} \mathbf{n} &= \begin{bmatrix} y_{AB} & -x_{AB} \end{bmatrix} \\ \text{penetration} &= d - r \\ \text{contact point} &= \mathbf{P_{closest}} \end{aligned} \tag{8.6}$$

Where $\mathbf{AB}$ is the direction vector between the end points of the closest edge, d is the distance between the closest point and the point mass, r is the radius of the point mass (usually a small value, for my engine $r = 0.01$). If $d < r$ then no collision is happening.

Finally we can generate a contact constraint, the same one we used for rigid body collision, passing in the normal, point and penetration depth and the collision will be resolved correctly. Since this is ran for each point mass of both objects, multiple contact constraint may be generated and some may be duplicates, however this is fine since PGS solver can easily handle duplicate or redundant contacts.

If you remember what I said earlier, the edge projection only works for when the point mass is not inside the other object, if the point mass is inside the objects, situation may occurred when closest edge may not be the edge we want. Consider the point $\mathbf{P}$ in *Figure 8.3*, first it is inside of the other polygon and it is aligned in such a way that the closest edge is the top edge (highlighted in red) instead of the desirable edge (highlighted in green). This lead to the normal pointing up and unable to separate the two objects.

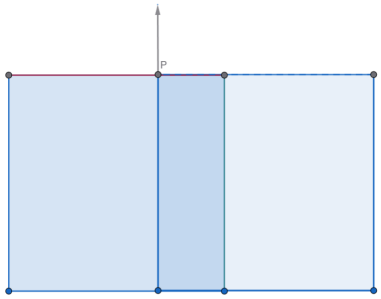

Figure 8.3: Incorrect normal detection for soft body collision

The main reason this happen is because we are treating each point mass as its own individual object instead of considering the whole soft body itself. To fix this I simply find a separating axis to fall back on using SAT when the center is inside the shape in the case where the shape is a convex polygon. In the case where the shape is concave, it is rather difficult to determine whether the objects are colliding or not, so I still rely on the best edge method which works well enough for almost all cases. The implementation of the code is exactly like the code for SAT, however I strip away some of the features and make the function only return the calculated normal. We can still rely on the edge algorithm to find the best contact point, while the penetration depth can be calculated via this equation

$$\text{penetration} = r + (\max_i(\mathbf{p_i} \cdot \mathbf{n}) - \mathbf{C} \cdot \mathbf{n}) \tag{8.7}$$

Where $\mathbf{p_i}$ is the global position of a point mass in the other object's mass aggregate, $\mathbf{n}$ is the calculated SAT collision normal and $\mathbf{C}$ is the global position of the point mass.

8.4 Soft and rigid body collision detection and resolution

The logical next step after colliding soft body and soft body together is to integrate it with rigid body and do collision between soft body and rigid body. The algorithm is quite similar to soft body collision and is divided into two passes. The first pass checks each point mass of the soft body against the edges of the rigid body. The second pass checks each vertex of the soft body with the edges of the soft body. Since rigid body does not store each vertex as a physics object and soft body does not store a list of edges only vertices, I dynamically construct them during collision detection.

For both pass we basically repeat the process we've done for the soft body collision.

- Use a ray casting algorithm to check if the point mass is inside the rigid body for the first pass and check if the rigid body vertex is inside of the soft body for the second pass
- If the point mass / vertex is outside, find the closest edge then derive the contact point, penetration depth and contact normal using the same algorithm as the soft body collision
- If the point mass / vertex is inside, calculate the normal using SAT and then from there calculate penetration depth similar to the soft body collision
- Generate a contact constraint using the contact normal, penetration depth and contact point

Chapter 9

XPBD solver for soft body physics

9.1 Limitations of mass spring system

One of the main problem with mass spring system is that it is incredibly unstable, especially when many objects are colliding with each other. The reason they are unstable is because everything is solved in velocity space and then is translated to position space via $\mathbf{x_{new}} = \mathbf{x} + \mathbf{v}\Delta t$. The main problem here is Δt, due to floating point errors, slight inconsistencies and occurs. In an enclosed constrained system such as a soft body (where constraints connects objects to form a shape), slight inconsistencies can quickly compound making it difficult for PGS to resolve everything without breaking other constraints. This can quickly cause the objects to explode or expand, contract unnaturally.

The other problem is that when a soft body gets deform too much, the point masses may pass through each other, this can cause the objects to reach a stable configuration other than the desirable one. **P** in *Figure 9.1* displays this exact scenario, the mesh is deform but the engine thinks that the state is correct because it is in a stable configuration. One of the most common methods to resolve this with mass spring system is shape matching, basically constructing a frame of the desirable shape and then try to force the soft body to match that shape. This is fine but required a lot of effort to do and may look unrealistic in certain cases.

Switching to an XPBD solver can solve the first issue by solving the constraints directly in position space instead of in velocity space. The second issue can be solved with a simple area constraint that force the soft body surface area to stay constant which allow it to go back when it reaches an undesirable stable configuration.

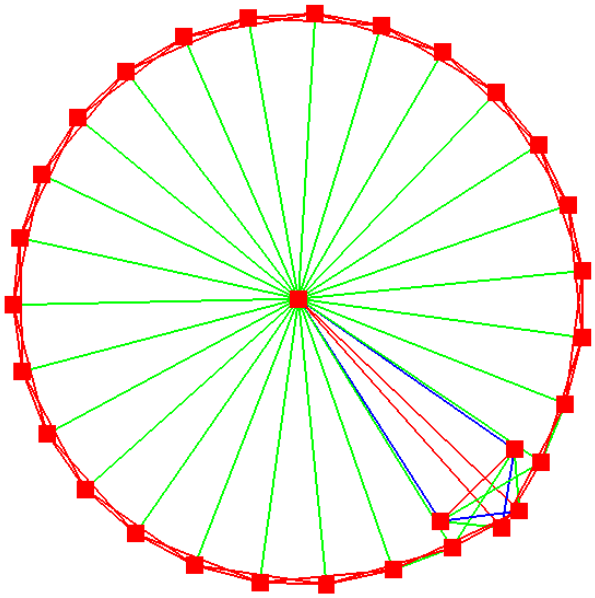

Figure 9.1: Mass spring system adjusting to undesirable stable configuration

9.2 Extended position based dynamics (XPBD)

Position based dynamics is a method introduced by *Matthias Müller*, *Bruno Heidelberger*, *Marcus Teschner*, and *Markus Gross* in the research paper *Position Based Dynamics* published in 2006. This is later extended upon in the research paper *XPBD: Position-Based Simulation of Compliant Constrained Dynamics* by *Miles Macklin*, *Matthias Muller*, and *Nuttapong Chentanez* published in 2016.

Let $\mathbf{x}(t)$ be a function that takes in a time t and returns the position of the object at that time. Let $\mathbf{x_k} = \mathbf{x}(t)$ and $\mathbf{x_{k+1}} = \mathbf{x}(t + \Delta t)$, $\mathbf{x_{k-1}} = \mathbf{x}(t - \Delta t)$. We also have the general formula for the Taylor forward expansion commonly used for linearization

$$\begin{aligned} f(t + \Delta t) &= \sum_{n=0}^{\infty} \frac{\Delta t^n}{n!} f^n(t) \\ &= f(t) + \Delta t f^{'}(t) + \frac{\Delta t^2}{2} f^{''}(t) + \frac{\Delta t^3}{6} f^{'''}(t) + \dots \end{aligned} \tag{9.1}$$

Using 3rd order expansion of the Taylor forward expansion, we can linearize $\mathbf{x_{k+1}}$ and $\mathbf{x_{k-1}}$ as

$$\begin{aligned} \mathbf{x_{k+1}} &= \mathbf{x_k} + \Delta t \frac{d\mathbf{x_k}}{dt} + \frac{\Delta t^2}{2} \frac{d^2\mathbf{x_k}}{dt^2} + \frac{\Delta t^3}{6} \frac{d^3\mathbf{x_k}}{dt^3} \\ \mathbf{x_{k-1}} &= \mathbf{x_k} - \Delta t \frac{d\mathbf{x_k}}{dt} + \frac{\Delta t^2}{2} \frac{d^2\mathbf{x_k}}{dt^2} - \frac{\Delta t^3}{6} \frac{d^3\mathbf{x_k}}{dt^3} \end{aligned} \tag{9.2}$$

Adding the two sides together we get

$$\mathbf{x_{k+1}} + \mathbf{x_{k-1}} = 2\mathbf{x_k} + \Delta t^2 \frac{d^2\mathbf{x_k}}{dt^2} \tag{9.3}$$

From there we can derive an expression for the acceleration based on the objects current $\mathbf{x_k}$, previous $\mathbf{x_{k-1}}$ and future $\mathbf{x_{k+1}}$ position

$$\mathbf{a} = \frac{d^2\mathbf{x_k}}{dt^2} = \frac{\mathbf{x_{k+1}} - 2\mathbf{x_k} + \mathbf{x_{k-1}}}{\Delta t^2} \tag{9.4}$$

In classical mechanics, we can define the net force applied to an object as a negative change in potential energy. We use $\mathbf{x_{k+1}}$ because we want to find the force that will need to be applied to the system to bring the object from the current position $\mathbf{x_k}$ to the future position $\mathbf{x_{k+1}}$

$$F_{system} = -\nabla U(\mathbf{x_{k+1}}) \tag{9.5}$$

From there we can finally derive the base equation for XPBD

$$M\frac{\mathbf{x_{k+1}} - 2\mathbf{x_k} + \mathbf{x_{k-1}}}{\Delta t^2} = -\nabla U(\mathbf{x_{k+1}}) \tag{9.6}$$

Where M is the mass of the system
In XPBD, we define the equation of potential energy for the system as

$$U(\mathbf{x}) = \frac{1}{2}\mathbf{C}(\mathbf{x})^T\alpha^{-1}\mathbf{C}(\mathbf{x}) \tag{9.7}$$

Where $\mathbf{C} = \begin{bmatrix} C_1(\mathbf{x}) & C_2(\mathbf{x}) & \dots & C_n(\mathbf{x}) \end{bmatrix}$ is a vector of constraint equations. α is a diagonal matrix of compliance corresponding to inverse stiffness. Low compliance means that the system is very stiff, not allowing for any errors, high compliance means that the system can deform more freely. After that, we can derive the force of elastic potential as the negative gradient of U with respect to $\mathbf{x}$

$$F_{constraint} = -\nabla_{\mathbf{x}} U^T = -\nabla \mathbf{C}^T \alpha^{-1}\mathbf{C} \tag{9.8}$$

Here $F_{constraint}$ is the constraint force. In mechanics, a constraint force is defined as the gradient followed by some multiplier λ. This is the same way we derived the constraint force for rigid body constraints where $F_{constraint} = \nabla C^T\lambda$. So from there we know that

$$\begin{aligned} & F_{constraint} = \nabla \mathbf{C}^T\lambda \\ \iff & \nabla \mathbf{C}^T\lambda = \nabla \mathbf{C}^T\alpha^{-1}\mathbf{C} \\ \iff & \lambda = \alpha^{-1}\mathbf{C} \end{aligned} \tag{9.9}$$

Where $\lambda = \begin{bmatrix} \lambda_1 & \lambda_2 & \dots & \lambda_n \end{bmatrix}$ is a vector of constraint multipliers. Currently we are using α, however since we are solving this every timestep Δt, we need to embed this into the equation. The way XPBD method does this is to define a new compliance $\tilde{\alpha} = \frac{\alpha}{\Delta t^2}$ and then substituting the original compliance

$$\lambda = \tilde{\alpha}^{-1}\mathbf{C} \tag{9.10}$$

The net force applied onto the object in the system can be decomposed into two forces, the external force (gravity, drag) and the internal (constraint) force. From there we can rewrite *Equation (9.6)* as

$$M(\mathbf{x_{k+1}} - 2\mathbf{x_k} + \mathbf{x_{k-1}}) = F_{external} + \nabla \mathbf{C}(\mathbf{x})^T \lambda \tag{9.11}$$

Here Δt is removed because it is integrated in $\tilde{\alpha}$ in λ. If we ignore the constraint force, we can find the predicted position of the object in an unconstrained system, this is usually called the inertial position.

$$\tilde{\mathbf{x}} = 2\mathbf{x_k} - \mathbf{x_{k-1}} + \Delta t^2 M^{-1} F_{external} \tag{9.12}$$

If we substitute this back into *Equation (9.11)*, we get

$$M(\mathbf{x_{k+1}} - \tilde{\mathbf{x}}) = \nabla \mathbf{C}(\mathbf{x})^T \lambda \tag{9.13}$$

Combining this with *Equation (9.11)*, we have a system of equation that can be solved

$$\begin{aligned} M(\mathbf{x_{k+1}} - \tilde{\mathbf{x}}) - \nabla \mathbf{C}(\mathbf{x})^T \lambda = 0 \\ \mathbf{C}(\mathbf{x_{k+1}}) + \tilde{\alpha}\lambda = 0 \end{aligned} \tag{9.14}$$

From the first equation we can solve for the change in position as

$$\Delta \mathbf{x} = \mathbf{x_{k+1}} - \tilde{\mathbf{x}} = M^{-1} \nabla \mathbf{C}(\mathbf{x}) \lambda \tag{9.15}$$

Using the Taylor forward expansion we can linearize the constraint equation

$$\mathbf{C}(\mathbf{x_{k+1}}) \approx \mathbf{C}(\mathbf{x_k}) + \nabla \mathbf{C} \Delta \mathbf{x} = 0 \tag{9.16}$$

From here we perform a Gauss Seidel update loop to solve the linear complementarity problem. This process is quite similar to what we did in PGS solver. For each constraint C_j, the change in position $\Delta \mathbf{x}$ caused by the change in λ, $\Delta\lambda$ is

$$\Delta \mathbf{x} = M^{-1} \nabla C_j^T \Delta \lambda_j \Delta t^2 \tag{9.17}$$

Substituting this into *Equation (9.16)* and solving for each constraint C_j

$$C_j(\mathbf{x}_i) + \nabla C_j (M^{-1} \nabla C_j^T \Delta \lambda_j) + \tilde{\alpha}_j (\lambda_{ij} + \Delta \lambda_j) = 0 \tag{9.18}$$

Where $\mathbf{x_i}$ is the current position at the i^{th} iteration, λ_{ij} is the lambda of the constraint C_j at the i^{th} iteration. After that we can expand, isolate and solve for $\Delta\lambda_j$

$$\begin{aligned} \Delta \lambda_j (\nabla C_j M^{-1} \nabla C_j^T + \tilde{\alpha}_j) = -C_j(\mathbf{x}_i) - \tilde{\alpha}_j \lambda_{ij} \\ \iff \Delta \lambda_j = \frac{-C_j(\mathbf{x}_i) - \tilde{\alpha}_j \lambda_{ij}}{\nabla C_j M^{-1} \nabla C_j^T + \tilde{\alpha}_j} \end{aligned} \tag{9.19}$$

The Gauss Seidel loop will loop through and slow converge λ over multiple iterations i for each constraint C_j

9.3 Damping for XPBD constraints

For certain constraints we might want to apply a damping force to the constraint so that the object lose energy and behave more realistically. The algorithm to apply damping to XPBD is based on the Rayleigh dissipation function.

$$\begin{aligned} D(\mathbf{x}, \mathbf{v}) &= \frac{1}{2}\dot{\mathbf{C}}(\mathbf{x})^T \beta \dot{\mathbf{C}}(\mathbf{x}) \\ &= \frac{1}{2}\mathbf{v}^T \nabla \mathbf{C}^T \beta \nabla \mathbf{C}\mathbf{v} \end{aligned} \tag{9.20}$$

Where β is a block diagonal matrix of damping coefficients and $\mathbf{v}$ is the velocity of the object. According to Lagrangian mechanics, the damping force is calculated from the negative gradient of D with respect to $\mathbf{v}$

$$F_{damping} = -\nabla_{\mathbf{v}} D^T = -\nabla \mathbf{C}^T \beta \nabla \mathbf{C}\mathbf{v} \tag{9.21}$$

Similar to the constraint force, we can break down the damping force into the gradient combined with a multiplier λ

$$\begin{aligned} F_{damping} &= \nabla \mathbf{C}^T \lambda_{\mathbf{damping}} = -\nabla \mathbf{C}^T \beta \nabla \mathbf{C}\mathbf{v} \\ \lambda_{\mathbf{damping}} &= -\beta \nabla \mathbf{C}\mathbf{v} \end{aligned} \tag{9.22}$$

Similar to compliance, we should also scaled damping factor with Δt

$$\begin{aligned} \tilde{\beta} &= \frac{\beta}{\Delta t^2} \\ \lambda_{\mathbf{damping}} &= -\tilde{\beta} \nabla \mathbf{C}\mathbf{v} \end{aligned} \tag{9.23}$$

While it is possible to solve for $\lambda_{\mathbf{damping}}$ on its own using the same process as we did for $\lambda_{\mathbf{constraint}}$, however we usually only care about the total combined force

$$\begin{aligned} \lambda &= \lambda_{\mathbf{constraint}} + \lambda_{\mathbf{damping}} \\ &= -\tilde{\alpha}\mathbf{C}(\mathbf{x}) - \tilde{\beta} \nabla \mathbf{C}\mathbf{v} \end{aligned} \tag{9.24}$$

We can rearrange the equation above into the constraint form

$$\mathbf{h}(\mathbf{x}, \lambda) = \mathbf{C}(\mathbf{x}) + \tilde{\alpha}\lambda + \tilde{\alpha}\tilde{\beta} \nabla \mathbf{C}\mathbf{v} = 0 \tag{9.25}$$

Substituting *Equation 9.16* into the equation above we can get the expression

$$(\mathbf{C}(\mathbf{x}_i) + \nabla \mathbf{C} \Delta \mathbf{x}) + \tilde{\alpha}(\lambda + \Delta\lambda) + \tilde{\alpha}\tilde{\beta} \nabla \mathbf{C} \left(\frac{\mathbf{x}_i + \Delta \mathbf{x} - \mathbf{x}_k}{\Delta t} \right) = 0 \tag{9.26}$$

Where $\mathbf{x_k}$ is the position at the beginning of the frame, $\mathbf{x_i}$ is the position at the current gauss seidel iteration i. In this equation, we have also replaced $\mathbf{v}$ with an expression based on position $\mathbf{v} \approx \frac{\mathbf{x_{k+1}} - \mathbf{x_k}}{\Delta t}$. We can then substitute *Equation 9.17* to get

$$[(1 + \frac{\tilde{\alpha}\tilde{\beta}}{\Delta t^2}) \nabla \mathbf{C} M^{-1} \nabla \mathbf{C}^T + \tilde{\alpha}] \Delta\lambda = -\mathbf{h}(\mathbf{x_i}, \lambda_\mathbf{i}) \tag{9.27}$$

Let $\gamma = \frac{\tilde{\alpha}\tilde{\beta}}{\Delta t^2}$ and solving the constraint C_j for each iteration i in the gauss seidel loop we get

$$\Delta\lambda_j \left(\nabla C_j M^{-1} \nabla C_j^T (1+\gamma_j) + \tilde{\alpha}_j\right) = -(C_j(\mathbf{x}_i) + \tilde{\alpha}_j \lambda_{ij} + \gamma_j \nabla C_j(\mathbf{x}_i - \mathbf{x}_k)) \tag{9.28}$$

And so we can finally derive the final equation for $\Delta\lambda$ for constraint C_j at iteration i as

$$\Delta\lambda_j = \frac{-C_j(\mathbf{x}_i) - \tilde{\alpha}_j \lambda_{ij} - \gamma_j \nabla C_j(\mathbf{x}_i - \mathbf{x}_k)}{(1+\gamma_j)\nabla C_j M^{-1} \nabla C_j^T + \tilde{\alpha}_j} \tag{9.29}$$

For the simulation loop, we iterate over each point mass over multiple sub steps, calculating $\Delta\lambda$ and $\Delta\mathbf{x}$. In each sub step, we also solve constraints multiple times. At the end of each loop, we calculate velocity of each point mass based on the change in position. The simulation algorithm goes like this:

- Loop n times where n is the number of sub steps, calculate $\Delta t_{sub} = \frac{\Delta t}{n}$
- Integrate all soft body point masses position from velocity and external forces (gravity, drag,..)
- Loop m times where m is the position integration steps, each step calculate $\Delta\lambda$ for each constraint and integrate it with the point masses position
- Calculate velocity based on change in position for point masses $\mathbf{v} = \frac{\mathbf{x} - \mathbf{x}_{prev}}{\Delta t_{sub}}$

9.4 XPBD distance constraint and area constraint

The first constraint we will be looking at is the distance constraint which will be replacing the soft distance rigid body constraint we used for the mass spring system. From the final $\Delta\lambda$ equation we derived in the last section, it can be seen the things we need to calculate are C_j, M^{-1} and ∇C_j. The constraint equation is

$$C(\mathbf{x_A}, \mathbf{x_B}) = ||\mathbf{x_B} - \mathbf{x_A}|| - L = 0 \tag{9.30}$$

Where L is the rest length, $\mathbf{x_A}$,$\mathbf{x_B}$ is the position of the point mass A and B involved in the constraint. Differentiating this we can find the value of the gradient of the constraint

$$\begin{aligned} \nabla C(\mathbf{x_A}, \mathbf{x_B}) &= \begin{bmatrix} \frac{\partial C}{\partial x_A} & \frac{\partial C}{\partial y_A} & \frac{\partial C}{\partial x_B} & \frac{\partial C}{\partial y_B} \end{bmatrix} \\ &= \begin{bmatrix} n_x & n_y & -n_x & -n_y \end{bmatrix} \end{aligned} \tag{9.31}$$

Where $\mathbf{n} = \begin{bmatrix} n_x & n_y \end{bmatrix} = \begin{bmatrix} \frac{x_B - x_A}{||x_B - x_A||} & \frac{y_B - y_A}{||y_B - y_A||} \end{bmatrix}$ is the normalize direction vector between A and B. The inverse effective mass of the system is

$$M^{-1} = M_A^{-1} + M_B^{-1} \tag{9.32}$$

From here we can calculate $\Delta\lambda$ based on the equation in the last section. Compliance and damping are parameters that can be entered depending on the desired behaviour of the spring. After each gauss seidel iteration we can use $\Delta\lambda$ to calculate $\Delta\mathbf{x}$ and then add it to λ for the next iteration.

$$\begin{aligned}\lambda_{new} &= \lambda + \Delta\lambda \\ \mathbf{x'_A} &= \mathbf{x_A} + M_A^{-1}\nabla_{\mathbf{x_A}} C\Delta\lambda \\ \mathbf{x'_B} &= \mathbf{x_B} + M_B^{-1}\nabla_{\mathbf{x_B}} C\Delta\lambda\end{aligned} \tag{9.33}$$

Now we can replace all the soft distance constraint with the new XPBD distance constraint. This will solve most of the stability problems that plague the old system that can cause the simulation to explode. However, one problem still has not been fixed yet and that is the problem of spring system reaching an undesirable stable configuration. To combat this, I use a constraint that affect the entire mass aggregate and force the object to maintain a certain area. The first step is to calculate the area of the soft body which I calculate using the formula

$$A = \frac{1}{2}\sum_i (x_i y_{i+1} - x_{i+1} y_i) \tag{9.34}$$

Where $\mathbf{p_i}(x_i, y_i)$ and $\mathbf{p_{i+1}}(x_{i+1}, y_{i+1})$ is global position of the i and $i+1$ vertex. This algorithm assume that the vertices wrap around the object and create a closed loop.

An area is calculated when the object is initialize or when the shape of the object is modify. I will refer to this area as A_0. The constraint equation basically states that the difference between the current area and the initial area must be zero

$$C(\mathbf{x}) = A - A_0 = 0 \tag{9.35}$$

Where $\mathbf{x} = \begin{bmatrix} x_0 & y_0 & x_1 & y_1 & \dots & x_n & y_n \end{bmatrix}$ are the positions of the point masses and A is the current area. We can once again differentiate this to find the gradient of the constraint

$$\nabla C(\mathbf{x}) = \begin{bmatrix} \frac{\partial C}{\partial x_1} & \frac{\partial C}{\partial y_1} & \frac{\partial C}{\partial x_2} & \frac{\partial C}{\partial y_2} & \dots & \frac{\partial C}{\partial x_n} & \frac{\partial C}{\partial y_n} \end{bmatrix} \tag{9.36}$$

When calculating the partial derivative of the constraint equation with respect to a point mass $\mathbf{p_i}(x_i, y_i)$, we only need to consider its contribution to the area equation since all other points are constant. For a point mass $\mathbf{p_i}$ the contribution of that point mass to the area is

$$A_i = \frac{1}{2}(x_i y_{i+1} - x_{i+1} y_i + x_{i-1} y_i - x_i y_{i-1}) \tag{9.37}$$

From there we can derive an expression for the partial derivative of C with respect to $\mathbf{p_i}$

$$\nabla_{\mathbf{p_i}} C = \frac{1}{2}\begin{bmatrix} y_{i+1} - y_{i-1} \\ x_{i-1} - x_{i+1} \end{bmatrix} \tag{9.38}$$

The inverse effective mass of the system is simply the sum of all inverse masses

$$M^{-1} = \sum_i M_i^{-1} \tag{9.39}$$

For the area constraint, we will not be using damping so the equation for $\Delta\lambda$ will be *Equation 9.19*. From there we can easily find the new lambda and new position

$$\begin{aligned}\lambda_{new} &= \lambda + \Delta\lambda \\ \mathbf{x'_i} &= \mathbf{x_i} + M_i^{-1}\nabla_{\mathbf{x_i}} C\Delta\lambda\end{aligned} \tag{9.40}$$

9.5 Inflatable objects simulation with ideal gas law

In the previous section, we have discussed a method to prevent springs of reaching an undesirable stable configuration and applying an area constraint, forcing the object to maintain an area. Another possible approach which we will discussed in this section is to make use of pressure which can be calculated using the ideal gas law. This method is not necessarily better than the other approach, in my engine I used a mix approach. The area constraint approach is good for general soft body while the pressure approach performs very well when simulating inflatable objects like balloons or tires.

This method makes use of the ideal gas law which states that

$$PV = nRT \tag{9.41}$$

where P is the pressure, V is the volume, n is the number of moles of gas, $R = 8.314$ is the ideal gas constant and T is the temperature. Because we are only working with 2D objects that have no volume, we can replace V with area A calculated by *Equation 9.34*. While it is possible to have separate parameters for n and T since we are not simulating thermal dynamics, combing nRT into a single value κ representing the total amount of gas inside of the object. From there we can calculate the pressure P as

$$P = \frac{\kappa}{A} \tag{9.42}$$

The pressure force direction is the normal of each edge of the soft body. For each edge with end points $\mathbf{A}(x_A, y_A)$ and $\mathbf{B}(x_B, y_B)$, the direction vector $\mathbf{d}(x_d, y_d) = \mathbf{B} - \mathbf{A}$, the normal is $\mathbf{n}(-y_d, x_d)$. The magnitude of the pressure force is scaled proportionally with the length of the edge

$$F_{pressure} = P||\mathbf{d}|| \tag{9.43}$$

The force is then split equally between each point mass $\mathbf{A}$ and $\mathbf{B}$ and applied to the point mass acceleration

$$\begin{aligned} \mathbf{a_A} &= \frac{F_{pressure} M_A^{-1} \mathbf{n}}{2} \\ \mathbf{a_B} &= \frac{F_{pressure} M_B^{-1} \mathbf{n}}{2} \end{aligned} \tag{9.44}$$

The pressure force is applied for each edge in the soft body. One problem with this approach that can be come worse with higher κ is that the object may expand a lot which would make it deform and unable to maintain its original shape. This is shown in *Figure 9.2* which shows one objects before and after the gasses are applied. It can be seen that the gas made the object expand a lot and the springs to be stretch (stretch springs are colored red) which could lead to instability in the simulation

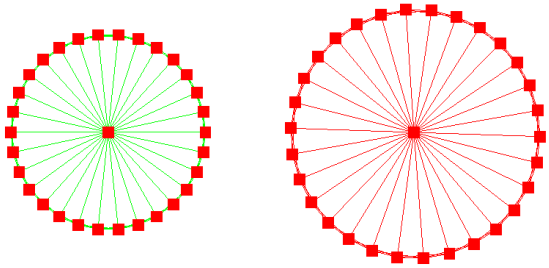

Figure 9.2: Over expansion of objects

The pressure force is processed inside of the XPBD loop, before the position integration step. With the inclusion of the pressure calculation, the simulation loop becomes:

- Loop n times where n is the number of sub steps, calculate $\Delta t_{sub} = \frac{\Delta t}{n}$
- Apply gas pressure to soft body if enabled
- Integrate all soft body point masses position from velocity and external forces (gravity, drag,..)
- Loop m times where m is the position integration steps, each step calculate $\Delta\lambda$ for each constraint and integrate it with the point masses position
- Calculate velocity based on change in position for point masses $\mathbf{v} = \frac{\mathbf{x}-\mathbf{x}_{prev}}{\Delta t_{sub}}$

Chapter 10

Combining two different solvers: PGS and XPBD

10.1 Overview

In the previous chapter we have created a robust system to simulate soft bodies using XPBD solver, however we have only looked at this as a standalone system. In this chapter we will look into interactions between soft and rigid bodies as well as methods to bridge the gap between PGS and XPBD solver. The main difference between these two methods is that they operate in different resolution space, PGS solves constraints in terms of velocity while XPBD solves constraints in terms of position.

10.2 XPBD based soft body collision

Up till now we've used PGS contact constraint for rigid-rigid body collision, soft - soft body collision, rigid - soft body collision. This works fine when we use PGS constraints but now that we've switched to XPBD distance constraints for soft bodies, handling collision with PGS contact constraints can lead to objects behaving incorrectly. Because XPBD solver directly change the object position and also run multiple integration substeps each frame, the soft body may appear to sink into other objects during collision.

For soft - soft body collision, the best option is to create a custom XPBD contact constraint since this will remove the need to bridge between solvers, everything will be handled in a centralized XPBD solver. For the other case, soft - rigid body collision, it will require a bridge between the solver.

First we will look at the XPBD contact constraint, the calculation is similar to what we've done with the distance and area constraint. The XPBD contact constraint is much simpler than the PGS constraint simply because we are working with purely point mass and point mass interaction where rotation are not considered. Similar to the PGS contact constraint, it is also split into two parts, the normal constraint and the friction (or tangent) constraint. The normal constraint equation is

$$C_n(\mathbf{x_A}, \mathbf{x_B}) = \mathbf{n} \cdot (\mathbf{x_B} - \mathbf{x_A}) - s \geq 0 \tag{10.1}$$

Where $\mathbf{n}$ is the collision normal, $\mathbf{x_A}$, $\mathbf{x_B}$ are the position of the two point masses involved in the collision and s is the separation which is the point mass radius. The gradient of the constraint is calculated as

$$\nabla C_n = \begin{bmatrix} \nabla_{\mathbf{x_A}} C_n & \nabla_{\mathbf{x_B}} C_n \end{bmatrix} = \begin{bmatrix} \mathbf{n} & -\mathbf{n} \end{bmatrix} \tag{10.2}$$

The inverse effective mass is

$$M^{-1} = M_A^{-1} + M_B^{-1} \tag{10.3}$$

After that we can calculate $\Delta\lambda$ and then integrate it with the objects position the same way we did for the previous constraints. However, the normal constraint is different because it is an inequality constraint. The way I handle these constraint is to simply clamp the λ

$$\lambda_n^{new} = \begin{cases} 0 & \text{if } C(\mathbf{x_A}, \mathbf{x_B}) \geq 0 \\ \max(\lambda_n + \Delta\lambda, 0) & \text{if } C(\mathbf{x_A}, \mathbf{x_B}) < 0 \end{cases} \tag{10.4}$$

For the friction constraint, we do almost the same thing but along the tangent direction. The friction constraint force is scaled with the change in position of the point masses. The constraint equation is

$$C_t(\Delta\mathbf{x_A}, \Delta\mathbf{x_B}) = \mathbf{t} \cdot (\Delta\mathbf{x_B} - \Delta\mathbf{x_A}) = 0 \tag{10.5}$$

Where $\Delta\mathbf{x_A} = \mathbf{x_A} - \mathbf{x_{A,prev}}$, $\Delta\mathbf{x_B} = \mathbf{x_B} - \mathbf{x_{B,prev}}$ are the change in position of the point masses and $\mathbf{t} = \begin{bmatrix} -n_y & n_x \end{bmatrix}$ is the collision tangent. Similar to the normal constraint, we can differentiate this to find the gradient

$$\nabla C_t = \begin{bmatrix} \nabla_{\mathbf{x_A}} C_t & \nabla_{\mathbf{x_B}} C_t \end{bmatrix} = \begin{bmatrix} \mathbf{t} & -\mathbf{t} \end{bmatrix} \tag{10.6}$$

The inverse effective mass is the same as the normal constraint. For the normal constraint, we clamped the λ based on the normal force λ_n

$$\begin{aligned} &\text{if } |\lambda_t^{new}| \leq \mu_s \lambda_n \implies \lambda_t = \lambda_t^{new} \\ &\text{if } |\lambda_t^{new}| > \mu_s \lambda_n \implies \lambda_t = \text{clamp}(\lambda_t^{new}, -\mu_k \lambda_n, \mu_k \lambda_n) \end{aligned} \tag{10.7}$$

Where μ_s is the coefficient of static friction and μ_k is the coefficient of kinetic (dynamic) friction. After that we can use the $\Delta\lambda$ and gradient of constraint to integrate the position of the two point masses.

With this we can safely replace the PGS contact constraint we've been using for soft - soft body collision and replace it with the XPBD contact constraint and it will behave correctly. The next step is to do rigid - soft body collision. As I've said before, we still have to used PGS contact constraint for this since the XPBD constraint only work for point mass and point mass collision.

The underlying cause of soft body sinking into rigid body is that the rigid body's position and velocity is integrated only once per frame while soft body position is integrated during each XPBD loop which can run multiple times each frame. This means that gravity is integrated multiple times every frame and even though we can divide Δt with the number of iteration and use that for calculation instead, this still does not prevent floating points error which can stack up overtime which will cause objects to seemingly sink into one another. The way we fix this is to warm start

the XPBD solver with the information from the PGS solver. The way we do this is pretty simple, just add position based on the penetration depth

$$\begin{aligned} \mathbf{x_A}' &= \mathbf{x_A} + p\mathbf{n} \\ \mathbf{x_B}' &= \mathbf{x_B} - p\mathbf{n} \end{aligned} \tag{10.8}$$

Where p is the penetration depth and $\mathbf{n}$ is the collision normal

10.3 Joining soft bodies with rigid joints

In past chapters, we have created the revolute, weld, prismatic constraints to join rigid bodies together, it make sense that we should be able to reuse them for soft bodies. While I could make a separate XPBD revolute, weld and prismatic constraint to join soft bodies, I opt for creating a bridge to allow the constraint system to support soft bodies. The main reason I do it is because it is easier to use existing systems and structure instead of making an entirely new one, furthermore, joining soft and rigid body together still would've required a bridge anyway.

The method I use is to create a proxy point which is a fully rotatable rigid body that lives within the soft body. This proxy point is linked with the point masses that are closest to it using a special proxy point constraint which is an XPBD constraint. This way the PGS solver only needs to interact with rigid bodies and does not have to concern itself with point masses while translating rotation, position data of proxy point to the mass aggregate is the job of the XPBD solver which it is suited for. *Figure 10.1* displays a rigid body (shown in black) linked to a soft body where the red corners are point masses and the point in the middle is the proxy point.

Figure 10.1: Soft body and rigid body joined with proxy point

The main component of this approach is the proxy point constraint which takes the rotation, position and velocities of the proxy point and translate it to the affected point masses. Unlike the other XPBD constraints which does not consider rotation, this constraint will have to consider the proxy point rotation so the constraint equation will be a bit different. The constraint equation is basically the distance constraint but with rotation included

$$C(\mathbf{x_{pm}}, \mathbf{x_{proxy}}, \theta) = \left\| \mathbf{x_{pm}} - \left(\mathbf{x_{proxy}} + \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} r_x \\ r_y \end{bmatrix} \right) \right\| = 0 \tag{10.9}$$

Where $\mathbf{x_{pm}}$ is the position of the linked point mass, $\mathbf{x_{proxy}}$ is the position of the proxy point, θ is the rotation of the proxy point. $\mathbf{r} = \begin{bmatrix} r_x & r_y \end{bmatrix}$ is the local offset from the soft body's center of mass at rest (when the soft body is not deformed).

Differentiating the constraint equation, we can once again find the gradient as

$$\begin{aligned} \nabla_{\mathbf{x_{pm}}} C &= \mathbf{n} \\ \nabla_{\mathbf{x_{proxy}}} C &= -\mathbf{n} \\ \nabla_\theta C &= -\mathbf{n} \cdot \begin{bmatrix} -r_x \sin\theta - r_y \cos\theta \\ r_x \cos\theta - r_y \sin\theta \end{bmatrix} \end{aligned} \tag{10.10}$$

Let $\mathbf{d} = \mathbf{x_{pm}} - (\mathbf{x_{proxy}} + \mathbf{R}(\theta)\mathbf{r})$ where $\mathbf{R}(\theta)$ is the rotation matrix of θ. Then the normalize direction vector is $\mathbf{n} = \frac{\mathbf{d}}{||\mathbf{d}||}$. The inverse effective mass of the constraint is

$$M^{-1} = M_{pm}^{-1} + M_{proxy}^{-1} + I_{proxy}^{-1} \tag{10.11}$$

Where M_{pm}^{-1}, M_{proxy}^{-1} is the inverse mass of the point mass and proxy point, I_{proxy}^{-1} is the inverse inertia of the proxy point. For the proxy point constraint, we will be going back to the XPBD formula with damping instead of the one with only compliance to calculate $\Delta\lambda$. After that the new position of the point mass and the proxy point is calculated as

$$\begin{aligned} \mathbf{x'_{pm}} &= \mathbf{x_{pm}} + M_{pm}^{-1}\Delta\lambda\mathbf{n} \\ \mathbf{x'_{proxy}} &= \mathbf{x_{proxy}} - M_{proxy}^{-1}\Delta\lambda\mathbf{n} \\ \theta' &= \theta - I_{proxy}^{-1}\Delta\lambda\mathbf{n} \cdot \begin{bmatrix} -r_x \sin\theta - r_y \cos\theta \\ r_x \cos\theta - r_y \sin\theta \end{bmatrix} \end{aligned} \tag{10.12}$$

The proxy can not be integrated the same way as rigid body but instead integrated separately in the XPBD loop, this will enable the proxy point's position to be integrated in the same way as the soft body point mass which will make things easier. Another method I use is to have the PGS loop additional run inside of the XPBD loop, recalculating all the constraints that are linked to a soft body. This will make it so the PGS constraints are in sync with the XPBD constraint. With that the final XPBD loop will be

- Loop n times where n is the number of substeps, calculate $\Delta t_{sub} = \frac{\Delta t}{n}$
- Apply gas pressure to soft body if enabled
- Integrate all soft body point masses and proxy points
- Loop m times where m is the position integration steps, each step calculate $\Delta\lambda$ for each constraint and integrate it with the point masses / proxy points position
- Recalculate PGS constraints, using Δt_{sub}
- Calculate velocity based on change in position for point masses and proxy points $\mathbf{v} = \frac{\mathbf{x} - \mathbf{x}_{prev}}{\Delta t_{sub}}$
- Calculate angular velocity of proxy points based on change in rotation $\omega = \frac{\theta - \theta_{prev}}{\Delta t_{sub}}$

Chapter 11

Fracture physics for rigid body

11.1 Overview

In the past few chapters we have looked at creating a fully functional soft body physics system and combining it with rigid body physics. In this chapter however, we will be extending the rigid body physics to support destructible objects by implementing fracture physics.

The general idea of fracture physics is basically when an object collide hard enough, we divide the objects into different pieces where each piece is a new rigid body. The algorithm I use for dividing the object is voronoi cells generation, the cell is constructed based on the contact points. Each piece can also facture an additional time, however this should have a limit since it can get incredibly laggy. *Figure 11.1* shows a rigid body in complete, first stage fracture and second stage fracture form.

Figure 11.1: Stages of object fracture

11.2 Voronoi cells generation

The Voronoi diagram partition a space, in this case the object's area, into a set of cells constructed around specific points called seeds. *Figure 11.2* displays a Voronoi cell diagram, the points are seeds that the cells are constructed from

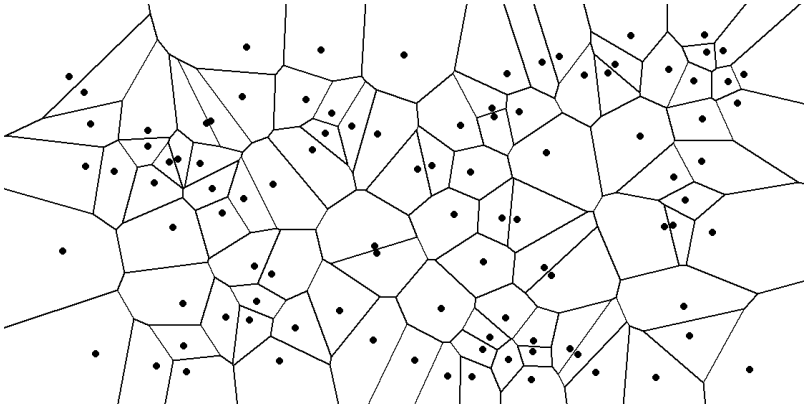

Figure 11.2: Voronoi cells

With that it can be seen that the first step in constructing the Voronoi cells is generating the Voronoi seeds. Because we want to generate the fracture shards from the contact points, I employ a simple algorithm based on that requirement to sample the seeds.

- Generate a random point inside of the object
- The point is a seed if it is at least d_{min} away from the previously generated seed. The first seed is always the contact point.
- The number of seeds generated is the number of shards the object will fracture into.

To check if a point is inside the polygon I simply use a ray cast algorithm similar to the one used in soft body collision detection.

For a specific seed $\mathbf{s_0}$, we can compare it with every other seeds in the object $\mathbf{s}$. For a pair of seed $\mathbf{s_0}$ and $\mathbf{s}$, we calculate the midpoint $\mathbf{m}$ and the normalized direction vector $\mathbf{n}$.

$$\begin{aligned} \mathbf{m} &= \frac{\mathbf{s_0} + \mathbf{s}}{2} \\ \mathbf{n} &= \frac{\mathbf{s} - \mathbf{s_0}}{||\mathbf{s} - \mathbf{s_0}||} \end{aligned} \tag{11.1}$$

From there we can find the distance from the world origin to the cutting half plane as

$$d = \mathbf{n} \cdot \mathbf{m} \tag{11.2}$$

For any point $\mathbf{p}$, it is on the safe side if

$$\mathbf{p} \cdot \mathbf{n} - d \leq 0 \tag{11.3}$$

Figure 11.3 shows the half plane defined by the inequality, by repeating this we can get the entire Voronoi cell as seen in *Figure 11.4* constructed from seed A and referenced the other seeds B, C, D

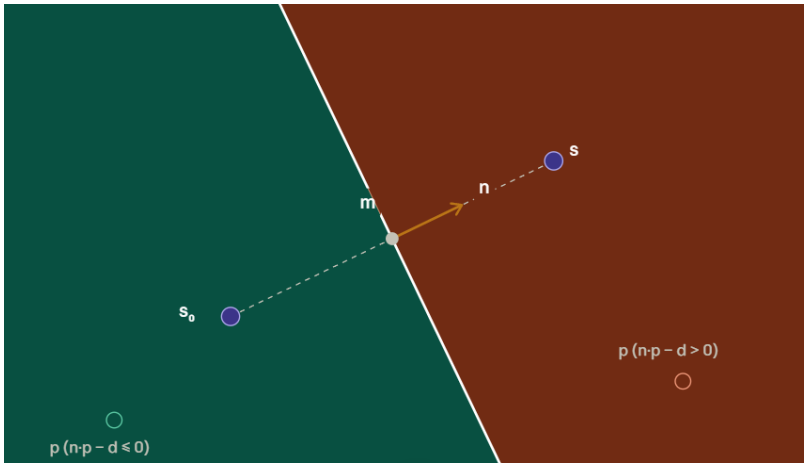

Figure 11.3: Voronoi half plane

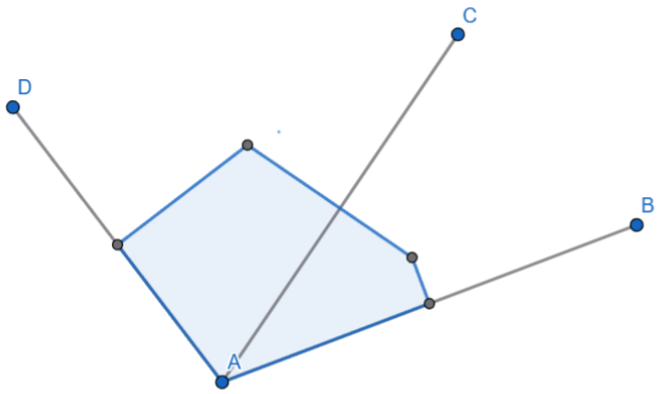

Figure 11.4: Voronoi cell

To clip the lines and obtain the final cell, I use the Sutherland-Hodgman line clipping algorithm which is also the same one used for the contact points generation algorithm in rigid body collision detection. The algorithm basically chops of any part of the polygon that crosses the dividing line.

The algorithm goes like this

- Loop through each edge of the polygon with end points **A**, **B**.
- Calculate the signed distance of **A** and **B** to the cutting plane. For a point **p**, the signed distance to the cutting plane is $d_p = \mathbf{p} \cdot \mathbf{n} - d$
- If both are inside of the plane ($d_p \leq 0$) then keeps both of them. If they are both outside, then discard both

- If one is inside, one is outside, clip the one that is outside by finding the intersection point as

$$t = \frac{d_A}{d_A - d_B}$$

$$\mathbf{p_{intersect}} = \mathbf{A} + t(\mathbf{B} - \mathbf{A})$$

From there we can get a set of vertices for each fragment to define its shape. The mass and velocity of each shard is calculated as

$$\begin{aligned} m_{shard} &= m\frac{A_{shard}}{A} \\ \mathbf{v_{shard}} &= \mathbf{v} + \begin{bmatrix} -r_y\omega \\ r_x\omega \end{bmatrix} \\ \omega_{shard} &= \omega \end{aligned} \tag{11.4}$$

Where m_{shard}, m is the mass of the shard and the original object, A_{shard}, A is the area of shard and the original object. $\mathbf{v_{shard}}$, $\mathbf{v}$, ω_{shard}, ω is the linear and angular velocity of the shard and original object. $\mathbf{r} = \begin{bmatrix} r_x \\ r_y \end{bmatrix} = \mathbf{p_{shard}} - \mathbf{p}$ is the vector between the position of the shard and the position of the original object.

Chapter 12

Fluid simulation with Position Based Fluids (PBF)

12.1 Overview

After the simulating rigid body and soft body physics, the next step is to simulate fluid dynamics. Similar to rigid body and soft body, there are multiple ways one could go about simulating fluids, each with their own pros and cons. All methods goal are similar however, they all aim to solve the Navier-Stokes equation in one way or another. The Navier-Stokes equation is an equation that govern the behaviour of fluids.

The 2 main methods for approximating the Navier-Stokes equation for real time fluid simulation is Smooth Particle Hydrodynamics (SPH) and Position Based Fluids (PBF). For my engine I choose PBF for two main reasons, while SPH can simulate fluids with higher accuracy, PBF is more stable and less prone to explosion than SPH. Furthermore, PBF builds on top of the position based dynamics that we have introduced in soft body physics so it is easier to integrate it with the existing infrastructure. *Figure 12.1* shows a fluid confined in a rigid tank simulated using PBF.

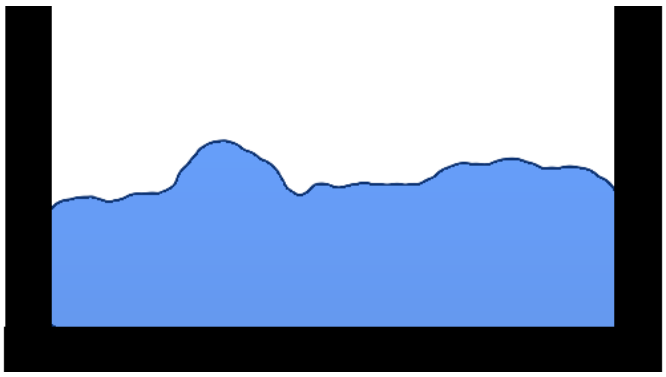

Figure 12.1: 2D fluid confined in a tank

12.2 PBF pressure constraint

In fluid dynamics, the Navier-Stokes equation states that

$$\begin{aligned} \rho\left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u}\cdot\nabla\mathbf{u}\right) &= -\nabla p + \mu\nabla^2\mathbf{u} + \mathbf{F}_{\mathbf{external}} \\ \frac{\partial \rho}{\partial t} + \nabla\cdot(\rho\mathbf{u}) &= 0 \end{aligned} \tag{12.1}$$

Where ρ is the fluid pressure, $\mathbf{u}$ is the fluid velocity, $\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u}\cdot\nabla\mathbf{u}$ is the fluid acceleration, $-\nabla p$ is the pressure gradient force, $\mu\nabla^2\mathbf{u}$ is the viscous force and $\mathbf{F}_{\mathbf{external}}$ is the external forces (such as gravity).

Both SPH and PBF use particles to simulate fluids. Each particle is similar to point masses of soft body, they have linear velocity and position but no rotation and angular velocity. A simulation usually make use of hundreds or thousands of these particles and apply forces or constraints on them to simulate the behaviour of fluids.

In SPH simulation, the Navier-Stokes equation is approximate directly and a pressure force is derived. The pressure force is then used to calculate the acceleration and velocity of each particle. In PBF however, the equation is not necessary solved but replaced with a geometric constraint instead.

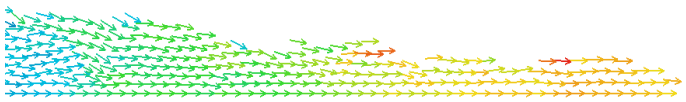

Figure 12.2: Fluid velocity field

PBF aims to enforce the continuity equation $\frac{\partial \rho}{\partial t} + \nabla\cdot(\rho\mathbf{u}) = 0$. Using the dot product rule for vector fields, we can expand this equation as

$$\frac{\partial \rho}{\partial t} + \mathbf{u}\cdot\nabla\rho + \rho(\nabla\cdot\mathbf{u}) = 0 \tag{12.2}$$

It can be seen that the first part of the equation is a material derivative

$$\frac{D\rho}{Dt} = \frac{\partial \rho}{\partial t} + \mathbf{u}\cdot\nabla\rho \tag{12.3}$$

Substituting this back into the equation we get

$$\frac{D\rho}{Dt} + \rho(\nabla\cdot\mathbf{u}) = 0 \tag{12.4}$$

In incompressible fluid flow, the divergence of the velocity field is equal to 0 ($\nabla\cdot\mathbf{u} = 0$). With that the equation becomes simply

$$\frac{D\rho}{Dt} = 0 \tag{12.5}$$

This equation basically state that the density is always constant for every point in the fluid.

The velocity field shown in *Figure 12.2* is a visual representation of the divergence, at any point, the amount of velocity flowing in is equal to the amount of velocity flowing out. Because the density at any point ρ_i is always equal to the rest density ρ_0, we can derive the constraint equation for a particle i as

$$C_i(\mathbf{p_1}, \mathbf{p_2}, \ldots, \mathbf{p_n}) = \frac{\rho_i}{\rho_0} - 1 = 0 \tag{12.6}$$

Where $\mathbf{p_1}, \mathbf{p_2}, \ldots, \mathbf{p_n}$ is the position of the particles that make up the fluid. ρ_i is the pressure of the i^{th} particle, ρ_0 is the rest density of the fluid.

Lets recap some knowledge about position based dynamics. The goal is always to find $\Delta\mathbf{p}$ such that $C(\mathbf{p} + \Delta\mathbf{p}) = 0$. We can approximate the new position from the constraint force as

$$\mathbf{p} + \Delta\mathbf{p} \approx C(\mathbf{p}) + \nabla C^T \nabla C \lambda = 0 \tag{12.7}$$

From there we can see that the goal is to calculate ρ_i, ∇C and λ. The pressure of a particle is calculated through a weighted sum of its neighbours' mass and their distance to the particle.

$$\rho_i = \sum_j m_j W(||\mathbf{p_i} - \mathbf{p_j}||, h) \tag{12.8}$$

Where m_j is the mass of the neighbouring j^{th} particle, $\mathbf{p_i}$ is the position of the particle and $\mathbf{p_j}$ is the position of its neighbouring particle. W is a smoothing kernel function, h is the smoothing radius which is the radius of the area where neighbouring particles are searched and accounted for. In the simulation, it is assumed that all particles have the same mass so we can drop the mass term, turning the equation into

$$\rho_i = \sum_j W(||\mathbf{p_i} - \mathbf{p_j}||, h) \tag{12.9}$$

In the research paper *Position Based Fluids* by *Miles Macklin* and *Matthias Muller*, which my implementation of PBF is based on, the kernel function used is Poly6.

$$W(r, h) = \frac{315}{64\pi h^9}(h^2 - r^2)^3 \quad 0 \leq r \leq h \tag{12.10}$$

From the equation, it can be seen that as r approaches h, W approaches 0. This means that the further away the neighbouring particle is, the less it contributes to the density of the particle and the closer the neighbouring particle is, the more it contribute to the density.

After calculating the pressure, the next thing we need to find is the constraint gradient ∇C. We can simply find it by differentiating the constraint equation with respect to a particle $\mathbf{p_k}$

$$\nabla_{\mathbf{p_k}} C = \frac{1}{\rho_0} \sum_j \nabla_{\mathbf{p_k}} W(\mathbf{p_i} - \mathbf{p_j}, h) \tag{12.11}$$

Where $\nabla_{\mathbf{p_k}} C$ is the gradient smoothing function, for this I use the Spiky kernel function, ρ_0 is the rest density.

The gradient of the constraint is also different whether the particle is the currently addressed particle $k = i$ or it is a neighbouring particle $k = j$. Where

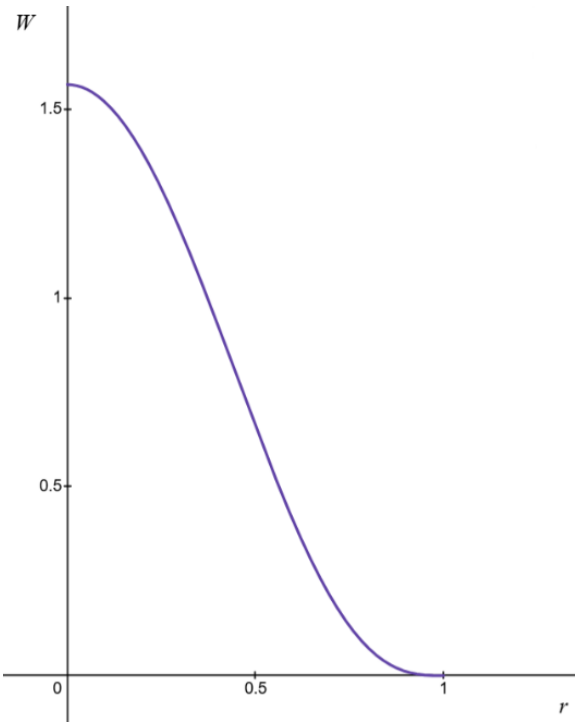

Figure 12.3: Poly 6 kernel: $h = 1$

$$\nabla_{\mathbf{p_k}} C = \frac{1}{\rho} \begin{cases} \sum_j \nabla_{\mathbf{p_k}} W(\mathbf{p_i} - \mathbf{p_j}, h) & \text{if } k = i \\ -\nabla_{\mathbf{p_k}} W(\mathbf{p_i} - \mathbf{p_j}, h) & \text{if } k = j \end{cases} \tag{12.12}$$

The Spiky gradient kernel function $\nabla_{\mathbf{p_k}} W$ used for calculating the constraint gradient is

$$\nabla_{\mathbf{p_k}} W(\mathbf{r}, h) = -\frac{45}{\pi h^6} \frac{(h - ||\mathbf{r}||)^2}{||\mathbf{r}||} \mathbf{r} \tag{12.13}$$

Finally, we can calculate λ of a particle as

$$\lambda_i = -\frac{C_i(\mathbf{p_1}, \mathbf{p_2}, \ldots, \mathbf{p_n})}{\sum_k |\nabla_{\mathbf{p_i}} C_i|^2} \tag{12.14}$$

Where $|\nabla_{\mathbf{p_i}} C_i|^2 = \nabla_{\mathbf{p_i}} C_i \cdot \nabla_{\mathbf{p_i}} C_i$ is the squared gradient, λ_i is the Lagrange multiplier of the i^{th} particle and C_i is the value of the constraint equation at $\mathbf{p_i}$.

From there, the change in position of the particle can be calculated as

$$\Delta \mathbf{p_i} = \frac{1}{\rho_0} \sum_j (\lambda_i + \lambda_j) \nabla W(\mathbf{p_i} - \mathbf{p_j}, h) \tag{12.15}$$

Where ∇W is the Spiky kernel gradient function, λ_i is the multiplier of the currently addressed particle, and λ_j is the multiplier of its neighbours.

The PBF simulation loop is quite similar to the XPBD simulation loop but with some key differences. The algorithm goes like this

- For each particle $\mathbf{p_i}$, integrate the external forces (gravity) to velocity $\mathbf{v_i} = \mathbf{v_i} + \Delta t \mathbf{F_{external}}$. After that calculate the predicted position before the constraint are applied $\mathbf{p_i'} = \mathbf{p_i} + \Delta t \mathbf{v_i}$.
- For each particle, find its neighbouring particle, the search range is the smoothing radius h.
- As with XPBD and PGS, λ is converge over multiple solve iteration. For each solve iteration do
  - Calculate λ_i for each particle
  - Calculate $\Delta \mathbf{p_i}$ for each particle
  - Update predicted position $\mathbf{p_i'} = \mathbf{p_i}' + \Delta \mathbf{p_i}$
- Finally for all particles, update velocity based on the change in position $\mathbf{v_i} = \frac{\mathbf{p_i'} - \mathbf{p_i}}{\Delta t}$. Then finally apply the predicted position to the particle position $\mathbf{p_i} = \mathbf{p_i}'$

12.3 Neighbours query using Spatial Hash Grid

As it can be seen in the PBF equations, one of the most important part of simulating fluid is finding the neighbours of particle. While we can simply check every single particle and compare the distances, however this is incredibly inefficient and runs in $O(n(n+1))$ time where n is the number of particles.

The solution I use is to divide the space into a grid of cells where the position of each particle is mapped to one cell of the grid. Instead of having to check every particle around the currently addressed particle, we only need to get the list of particles stored in the surrounding cell. The size of each cell is the smoothing radius.

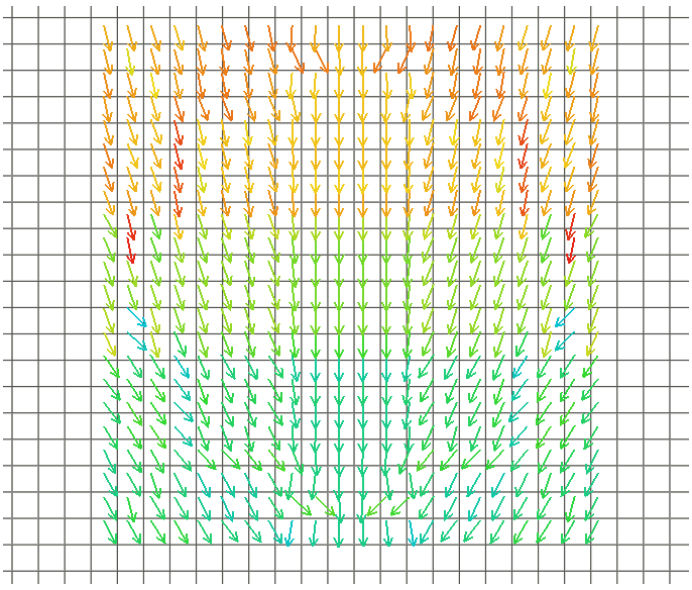

Figure 12.4: Spatial hash grid

The reason this algorithm is called a spatial hash grid is because a hash map is used to map each particle to each grid cell which allow $O(1)$ retrieval time.

The formula for translating from world coordinates to cell coordinates is very simple. For a particle with coordinates $\mathbf{p}(x, y)$ and the grid with cell size h, the cell coordinate $\mathbf{c}(x_c, y_c)$ is

$$\begin{aligned} x_c &= \text{floor}(\frac{x}{h}) \\ y_c &= \text{floor}(\frac{y}{h}) \end{aligned} \tag{12.16}$$

We also need to create a hashing algorithm to register the particle and cell coordinates to the hash map. The hashing algorithm I use this

$$\text{Key}(c_x, c_y) = (c_x \ll 32) \oplus c_y \tag{12.17}$$

Where $c_x \ll 32$ is left shift 32 bits, and $\oplus$ is XOR, each cell store an array of pointers to the fluid particles. At the build grid step, each particle are hashed to their respective grid cell.

After building the grid, retrieving data from the grid is quite simple. For a particle, its neighbours lies in the cell that the particle is in as well as the 8 surrounding cells. *Figure 12.5* shows the area that is searched in green while the area in red is the outside of the search bound

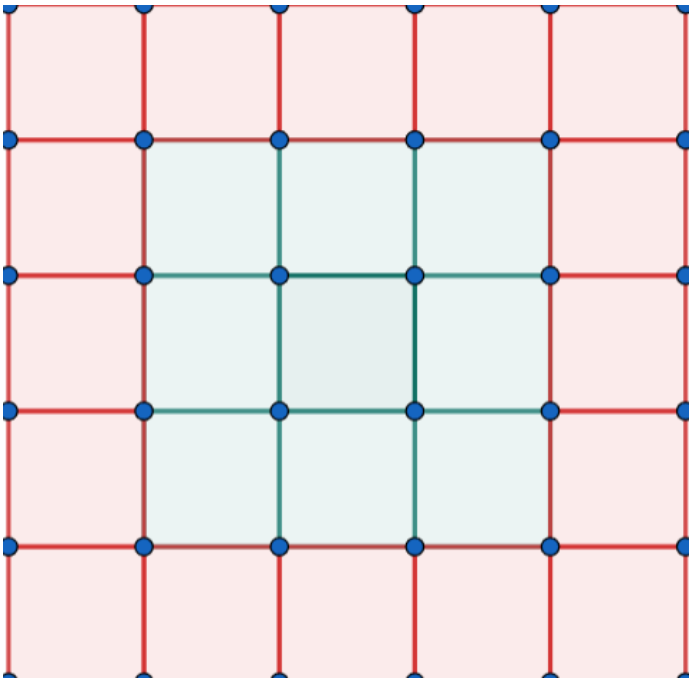

Figure 12.5: Neighbours search area

However h is the smoothing radius so we actually want to compute the distance between the point and its neighbours to filter out the ones with distance higher than h. After that, append all the fluid particles pointers into an array and return it for processing in the PBF solve loop.

12.4 Viscosity and vorticity confinement

Before adding viscosity and vorticity confinement, a problem needs to be addressed first. Since the constraint equation is non-linear, there may be instability when the particles are close the separating. My solution for this is to allow some wiggle room by using constraint force mixing (CFM). This is also the same method I used in constructing soft constraints for rigid bodies, the same logic is applied here, we are trying to make the density constraint a bit softer. The softness CFM ϵ is applied directly to λ calculation.

$$\lambda_i = -\frac{C_i(\mathbf{p_1}, \mathbf{p_2}, \ldots, \mathbf{p_n})}{\sum_k |\nabla_{\mathbf{p_i}} C_i|^2 + \epsilon} \tag{12.18}$$

Unlike SPH, PBF does not solve viscosity directly, this is because it does not solves the momentum equation (which has viscosity baked in) but only apply a geometric constraint to the particles. The solution to work around this is to create a small XSPH viscosity solve loop as post processing after the PBF solve loop. Viscosity is basically a damping force acted on the fluid particles to limit its velocity, *Figure 12.6* shows the difference between low viscosity fluid (left side) and high viscosity fluid (right side). The low viscosity one splatter high into the air before resting while the high viscosity fluid instantly rest and does not splatter.

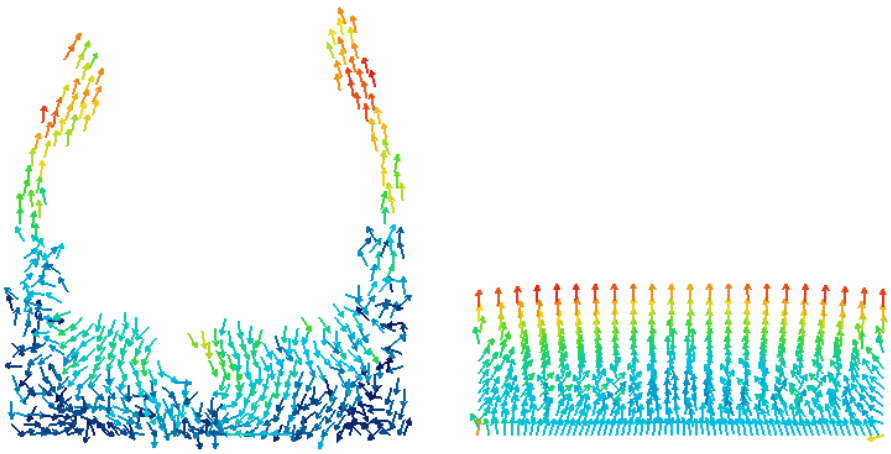

Figure 12.6: Fluid viscosity

The viscosity correction term is actually quite simple, it is calculated based on the velocities of neighbouring particles.

$$\mathbf{v_i} = \mathbf{v_i} + c \sum_j m_j (\frac{\mathbf{v_j} - \mathbf{v_i}}{\rho_j}) W(||\mathbf{p_i} - \mathbf{p_j}||, h) \tag{12.19}$$

Where $\mathbf{v_i}$, $\mathbf{v_j}$, $\mathbf{p_i}$, $\mathbf{p_j}$ is the velocities and positions of the currently addressed particle and its neighbouring particles. ρ_j is the density of the neighbouring particle, c is the coefficient of viscosity, m_j is the mass of the neighbouring particle, the smoothing kernel W used here is the poly6 kernel. Viscosity is applied after velocity correction are applied at the end of the PBF solve loop.

Due to numerical drift of resolving the density constraints, artificial damping (dissipation) can be unwantedly injected into the system. This can heavily damp rotational motion or vortices of the fluid, making behaviour like swirls, vortices and turbulence quickly dissipate overtime which can make the fluid looks sluggish and viscous. While this can be fixed by simply increasing the smoothing radius os the substeps this can quickly cause performance issues.

Vorticity confinement fixes this by detecting to dissipation of rotational motion and apply a corrective force to the fluid particle, injecting lost energy back into the system. This is shown in *Figure 12.7* where the fluid on the left has high vorticity and it can be seen that waves and vortices are forming while the fluid on the left has low vorticity and feel more viscous.

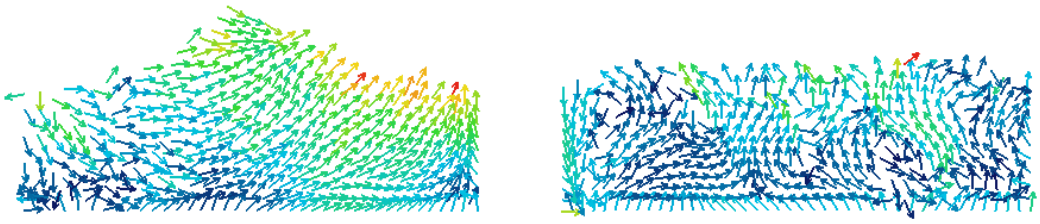

Figure 12.7: Fluid vorticity confinement

The first thing we need to do is to find the vorticity $\boldsymbol{\omega}$ or rotational motion of the fluids. This is calculated by taking the curl of the velocity field

$$\boldsymbol{\omega} = \nabla \times \mathbf{u} \tag{12.20}$$

The vorticity of a point $\mathbf{p}_i$ is thus calculated as

$$\boldsymbol{\omega}_i = \sum_j \mathbf{v}_{ij} \times \nabla_{\mathbf{p}_j} W(\mathbf{p}_i - \mathbf{p}_j, h) \tag{12.21}$$

Where $\mathbf{v}_{i,j} = \mathbf{v}_j - \mathbf{v}_i$ is the relative velocity between the particle and its neighbours. The smoothing gradient kernel function used here is the Spiky kernel function.

Next, we want to find where the vorticity magnitude increases the fastest, this is calculated by finding the gradient of the vorticity

$$\boldsymbol{\eta}_i = \nabla ||\boldsymbol{\omega}||_i = \sum_j ||\boldsymbol{\omega}_j|| \nabla_{\mathbf{p}_j} W(\mathbf{p}_i - \mathbf{p}_j, h) \tag{12.22}$$

By normalizing $\boldsymbol{\eta}_i$ we get a direction vector pointing to the center of the vortex. This can then be used to find the vorticity force

$$\begin{aligned} \mathbf{N}_i &= \frac{\boldsymbol{\eta}_i}{||\boldsymbol{\eta}_i||} \\ \mathbf{F_i^{vorticity}} &= \epsilon(\mathbf{N}_i \times \boldsymbol{\omega}_i) \\ \mathbf{v}_i &= \mathbf{v}_i + \Delta t \mathbf{F_i^{vorticity}} \end{aligned} \tag{12.23}$$

The viscosity and vorticity confinement is calculated as post processing after the PBF loop. With these added, the PBF loop becomes.

- For each particle $\mathbf{p_i}$, integrate the external forces (gravity) to velocity $\mathbf{v_i} = \mathbf{v_i} + \Delta t \mathbf{F_{external}}$. After that calculate the predicted position before the constraint are applied $\mathbf{p_i'} = \mathbf{p_i} + \Delta t \mathbf{v_i}$.
- For each particle, find its neighbouring particle, the search range is the smoothing radius h.
- As with XPBD and PGS, λ is converge over multiple solve iteration. For each solve iteration do
  - Calculate λ_i for each particle
  - Calculate $\Delta \mathbf{p_i}$ for each particle
  - Update predicted position $\mathbf{p_i'} = \mathbf{p_i}' + \Delta \mathbf{p_i}$
- For all particles, update velocity based on the change in position $\mathbf{v_i} = \frac{\mathbf{p_i'} - \mathbf{p_i}}{\Delta t}$. Then finally apply the predicted position to the particle position $\mathbf{p_i} = \mathbf{p_i}'$
- Compute and apply vorticity
- Compute and apply viscosity

Chapter 13

Unifying three solvers: PGS - XPBD - PBF

13.1 Overview

In the previous chapter, we have constructed a system to simulate fluid dynamics using Position Based Fluids method. In this chapter, we will follow the same path as when implementing soft body physics, combining the different solvers together.

With the soft body physics, the only type of collision between different solvers is rigid body with soft body. For fluids there are two additional collision: rigid body and fluid, soft body and fluid.

The method I used for having fluid interact with rigid and soft bodies is that I construct a virtual boundary around the object. The rigid, soft bodies are treated as static objects during the collision with the fluid particle and appropriate correction is applied to the fluid particles. After the particles are corrected, the impulses are calculated and apply into the rigid, soft bodies using a gauss seidel loop.

13.2 Fluid - Rigid collision detection and handling

The first step in fluid - rigid collision detection is computing the boundary of the rigid bodies. The algorithm basically just loop through each rigid body and get the edges of the rigid bodies, this is the object's boundary. The boundary is then used to detect collision in for rigid bodies. The rigid - fluid collision detection algorithm is quite similar to the rigid - soft body collision detection algorithm. This is because we can treat fluid particles in a similar way to soft body's point masses since they are both circles with small radius and no rotation. Unlike rigid - soft body's collision detection however, we do not have to divide the detection process in two phases, we just need to check the rigid body's edges with the fluid particles, since fluid have no edges. Collision detection for fluid - rigid body is performed by checking every particle in the fluids with the rigid body.

The first step, similar to soft - rigid body collision detection is to check if the particle is inside of the rigid body's boundary or not. The algorithm we used for this is the ray casting algorithm which was used in rigid - soft body collision detection. If the particle is inside of the rigid body then a collision is detected and needs to be handled.

After that, the contact point, collision normal and penetration depth needs to be found. This process is pretty simple, we simply find the closest edge of the rigid body to the particle. The collision normal is the normal of the edge, the contact point is the position of the particle projected along the normal onto the edge and the penetration depth is the distance from the particle to the contact point. This process is visually highlighted in *Figure 13.1*

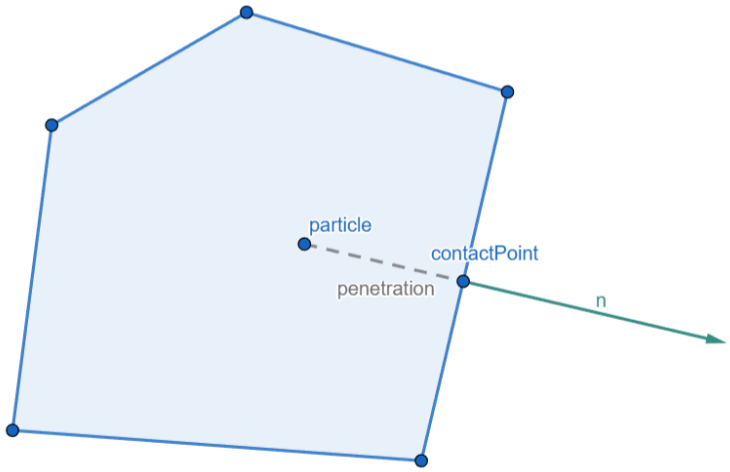

Figure 13.1: Fluid - Rigid collision detection

The next part is to resolve the collision, this is where the difference between rigid - soft collision and fluid - rigid collision begins to appear. As I've said before, we first treat the rigid body as static object and resolving only the fluid particles. This will happen after the PBF solve loop and after predicted position is calculated but before the new velocity and position are applied. Additionally, this will also only be a partial correction, only a fraction of the positional correction is applied, the rest of the correction will be done in another step later on.

One fluid particle can be in contact with many rigid bodies at once, so when resolving the collision we loop through the contact and find the best one, which is the one with the highest penetration depth. After that the predicted position of the fluid particle is corrected using the equation

$$\mathbf{p_i}' = \mathbf{p_i}' + \mathbf{n}_{best} p_{best} \epsilon_{correction} \tag{13.1}$$

Where $\mathbf{p_i}'$ is the predicted position of the fluid particle, $\mathbf{n}_{best}$, p_{best} is the collision normal and penetration depth of the best contact. $\epsilon_{correction}$ is a hyperparameter representing how much correction is applied in the PBF solve loop, for my engine, I set $\epsilon_{correction} = 0.2$

The next part is applying the impulse to the rigid body and additional correction to the fluid particle. The impulse is solved iteratively over multiple sub steps and velocity of both the rigid body and fluid particle is corrected.

First we need to find the velocity at contact, this is calculated as

$$\mathbf{v}_c = \mathbf{v}_{rb} + \omega_{rb} \times \mathbf{r} \tag{13.2}$$

Where $\mathbf{v}_c$ is the velocity at contact, $\mathbf{v}_{rb}$ is the velocity of the rigid body, ω_{rb} is the angular velocity of the rigid body. $\mathbf{r} = \mathbf{p}_{fluid} - \mathbf{p}_{rb}$ is a vector point from the center of the rigid body to the fluid particle. Next we calculate the contact velocity along the normal direction as

$$v_n = \mathbf{n} \cdot (\mathbf{v}_p - \mathbf{v}_c) \tag{13.3}$$

Where $\mathbf{n}$ is the collision normal, $\mathbf{v}_p$ is the velocity of the fluid particle. Next we need to calculate the Lagrange multiplier for the impulse. This is calculated as

$$\lambda = \frac{B - (1 + c)v_n}{M^{-1}} \tag{13.4}$$

Where c is the coefficient of restitution and $M^{-1} = M_{rb}^{-1} + M_p^{-1}$ is the inverse mass sum. B is the Baumgarte stabilization bias term, which is calculated from the penetration depth p as

$$B = \frac{\beta}{\Delta t_{sub}} p \tag{13.5}$$

After this we apply the impulse to the fluid particle and rigid body as

$$\begin{aligned} \mathbf{v}_p &= \mathbf{v}_p + M_p^{-1}\lambda\mathbf{n} \\ \mathbf{v}_{rb} &= \mathbf{v}_{rb} - M_{rb}^{-1}\lambda\mathbf{n} \\ \omega_{rb} &= \omega_{rb} - I_{rb}^{-1}(\mathbf{r} \times \lambda\mathbf{n}) \end{aligned} \tag{13.6}$$

The collision detection and handling is slot directly into the PBF solve loop. With these new addition, the new solve loop becomes

- For each particle $\mathbf{p_i}$, integrate the external forces (gravity) to velocity $\mathbf{v_i} = \mathbf{v_i} + \Delta t \mathbf{F_{external}}$. After that calculate the predicted position before the constraint are applied $\mathbf{p_i'} = \mathbf{p_i} + \Delta t \mathbf{v_i}$.
- For each particle, find its neighbouring particle, the search range is the smoothing radius h.
- As with XPBD and PGS, λ is converge over multiple solve iteration. For each solve iteration do
  - Calculate λ_i for each particle
  - Calculate $\Delta\mathbf{p_i}$ for each particle
  - Update predicted position $\mathbf{p_i'} = \mathbf{p_i}' + \Delta\mathbf{p_i}$
- Detect rigid body collision and apply partial position correction to fluid particles
- For all particles, update velocity based on the change in position $\mathbf{v_i} = \frac{\mathbf{p_i'} - \mathbf{p_i}}{\Delta t}$. Then finally apply the predicted position to the particle position $\mathbf{p_i} = \mathbf{p_i}'$

- Resolve fluid - rigid collision impulse
- Compute and apply vorticity
- Compute and apply viscosity

13.3 Fluid - Soft collision detection and handling

The next part is detecting and resolving fluid - soft collision, this follows roughly the same process as rigid body, simply applied to point masses instead of rigid body. The collision detection process is exactly the same as fluid - rigid collision detection. We simply replace the rigid body edges with the edges of the soft body (which can be dynamically calculated from the point masses), the algorithm for finding the contact point, penetration depth and collision normal is exactly the same.

The partial positional correction for fluid particles also remained the same, simply swapping rigid points with soft body point masses, rigid edges with soft body edges.

The main difference lies in the impulse correction for the soft body. While there are a lot of things similar to fluid - rigid collision, there are also some key differences. One of the things that has stayed the same is the overall process, we still first calculate the velocity at contact, then velocity along the normal, Lagrange multiplier, impulses then finally applying velocity correction.

The velocity at contact is calculated as

$$\mathbf{v}_c = \mathbf{v}_p - (w_A \mathbf{v}_{pmA} + w_B \mathbf{v}_{pmB}) \tag{13.7}$$

Where $\mathbf{v}_{pmA}$, $\mathbf{v}_{pmB}$ is the velocity of two point masses that make up the soft body edge. w_A, w_B are weights factor, calculated based on how close the contact point is compared to the two soft body point masses (0 = furthest, 1 = closest).

The velocity along the normal is thus calculated as

$$v_n = \mathbf{n} \cdot (\mathbf{v}_p - \mathbf{v}_c) \tag{13.8}$$

After that, we can calculate the Lagrange multiplier as

$$\lambda = \frac{B - (1 + c)v_n}{M^{-1}} \tag{13.9}$$

Where $M^{-1} = M_p^{-1} + M_{pmA}^{-1} w_A^2 + M_{pmB}^{-1} w_B^2$ is the inverse mass sum, c is the coefficient of restitution, B is the Bamguarte stabilization bias. The bias factor calculation remains the same as from the fluid - rigid body impulse resolution. Finally, the velocity is corrected as

$$\begin{aligned} \mathbf{v}_p &= \mathbf{v}_p + M_p^{-1} \lambda \mathbf{n} \\ \mathbf{v}_{pmA} &= \mathbf{v}_{pmA} - M_{pmA}^{-1} w_A \lambda \mathbf{n} \\ \mathbf{v}_{pmB} &= \mathbf{v}_{pmB} - M_{pmB}^{-1} w_B \lambda \mathbf{n} \end{aligned} \tag{13.10}$$

The fluid-soft collision detection and resolution is simply added into the same place alongside fluid-rigid collision handling in the PBF solve loop.

- For each particle $\mathbf{p_i}$, integrate the external forces (gravity) to velocity $\mathbf{v_i} = \mathbf{v_i} + \Delta t \mathbf{F_{external}}$. After that calculate the predicted position before the constraint are applied $\mathbf{p_i'} = \mathbf{p_i} + \Delta t \mathbf{v_i}$.
- For each particle, find its neighbouring particle, the search range is the smoothing radius h.
- As with XPBD and PGS, λ is converge over multiple solve iteration. For each solve iteration do
  - Calculate λ_i for each particle
  - Calculate $\Delta \mathbf{p_i}$ for each particle
  - Update predicted position $\mathbf{p_i'} = \mathbf{p_i}' + \Delta \mathbf{p_i}$
- Detect rigid body collision and apply partial position correction to fluid particles
- Detect soft body collision and apply partial position correction to fluid particles
- For all particles, update velocity based on the change in position $\mathbf{v_i} = \frac{\mathbf{p_i'} - \mathbf{p_i}}{\Delta t}$. Then finally apply the predicted position to the particle position $\mathbf{p_i} = \mathbf{p_i}'$
- Resolve fluid - rigid collision impulse
- Resolve fluid - soft collision impulse
- Compute and apply vorticity
- Compute and apply viscosity

13.4 Buoyancy

Normally in fluid simulation, buoyancy is baked into the simulation, however with how collision is currently handled, this won't be possible. The reason is because the object only interacts with the fluid particles closest to it, during collision resolution, when the object is submerged, the particles pushing down will cancel out withe particles below pushing up, the same goes for the sides, this will cause the object to be stuck in place. *Figure 13.2* shows this exact problem, the fluid's velocity field expand uniformly outward from the object, keeping it stuck in place.

The option I used is to apply buoyancy as an external force into the fluid particles, similar to the vorticity confinement and viscosity. The force is calculated based on the submerged area using the archimedes principle. The force is calculated as

$$\mathbf{F}_b = \rho V \mathbf{g} \tag{13.11}$$

Where $\mathbf{F}_b$ is the buoyant force, ρ is the fluid pressure, V is the submerged volume, $\mathbf{g}$ is the acceleration due to gravity. However since we are doing 2D, the volume can be replaced by area A

$$\mathbf{F}_b = \rho A \mathbf{g} \tag{13.12}$$

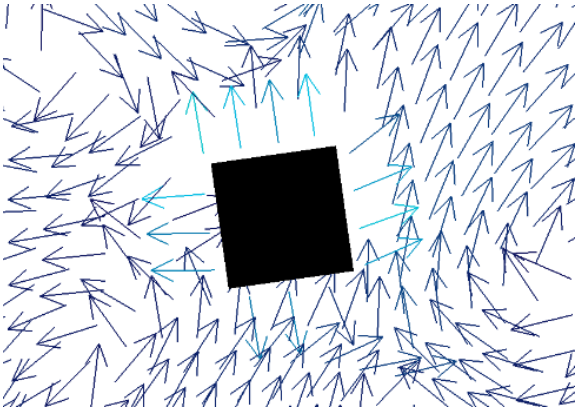

Figure 13.2: Buoyancy force canceled out during collision

The first thing and most difficult thing to calculate is the object's submerged area. The general idea of the algorithm is to find the particles in direct contact with the object. The particle with the maximum y value is considered the fluid surface boundary line, the area of the object below this line is the submerged area.

$$y_{surface} = \max_i(y_i^p) \tag{13.13}$$

Where $y = y_{surface}$ is the boundary surface line, y_i^p is the y value of a fluid particle $\mathbf{p}_i$. *Figure 13.3* displays the boundary line, the submerged area (shown in yellow), the area that is discarded (shown in red). It can be seen that this method is not entirely accurate, the submerged are is actually slightly lower than the computed one, however for our purposes, this is mostly accurate enough.

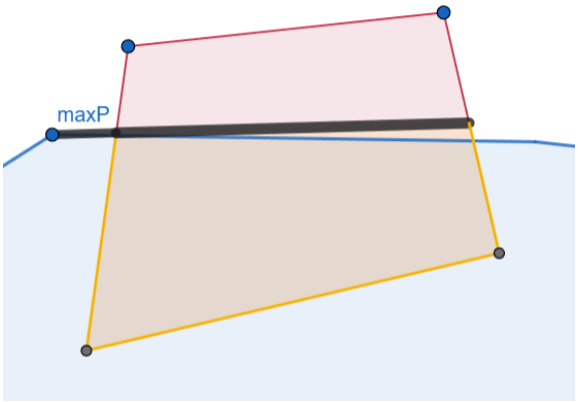

Figure 13.3: Object submerged region

After determining $y_{surface}$, we first clip the line with the polygon edges to find the intersection points to construct the submerged polygon. We loop through each edge of the rigid or soft body to calculate the intersection points. For an edge with end points $\mathbf{A}(x_A, y_A)$ and $\mathbf{B}(x_B, y_B)$. If the edge is entirely above the line, then both point is discarded, if the edge is entirely below the line, both point is kept, if one point is above and one is below, an intersection is calculated as

$$\begin{aligned} t &= \frac{y_{surface} - y_A}{y_B - y_A} \\ x_{int} &= x_A + t \cdot (x_B - x_A) \\ \mathbf{p}_{int} &= \begin{bmatrix} x_{int} & y_{surface} \end{bmatrix} \end{aligned} \tag{13.14}$$

A new polygon is constructed from the new filtered list of points, the area of that polygon is the submerged area. The area of the polygon is calculated similar to how we calculate area in the previous chapters.

$$A = \frac{1}{2} \sum_i (x_i y_{i+1} - x_{i+1} y_i) \tag{13.15}$$

We can't use the object's centroid to apply the buoyancy force to since we are only applying it to the section that is submerged, so we need to compute a new centroid as

$$\begin{aligned} c_x &= \frac{1}{6A} \sum_i (x_i + x_{i+1})(x_i y_{i+1} - x_{i+1} y_i) \\ c_y &= \frac{1}{6A} \sum_i (y_i + y_{i+1})(x_i y_{i+1} - x_{i+1} y_i) \\ \mathbf{c} &= \begin{bmatrix} c_x & c_y \end{bmatrix} \end{aligned} \tag{13.16}$$

Now that we have the submerged area, we need to find the fluid density. This is calculated by simply taking the average density of the fluid particles surrounding the object

$$\rho = \frac{1}{N} \sum_k \rho_k \tag{13.17}$$

Where ρ_k is the density of the fluid particle $\mathbf{p}_k$. After that we can apply the buoyancy formula to calculate the buoyancy force

$$\mathbf{F}_b = \rho A \mathbf{g} \tag{13.18}$$

where $\mathbf{g} = \begin{bmatrix} 0 & -9.8 \end{bmatrix}$ is the acceleration due to gravity. We will first talk about rigid body buoyancy and move onto discussing soft body buoyancy later since there are some differences. For rigid body, after finding the force, we need to find the torque, this is calculated as

$$\begin{aligned} \mathbf{r} &= \mathbf{c} - \mathbf{c}_{rb} \\ \tau &= r_x F_{b,y} - r_y F_{b,x} \end{aligned} \tag{13.19}$$

Where $\mathbf{c}_{rb}$ is the center of the original rigid body before it is clipped by the boundary line.

When resurfacing, the object might experience a jolt of velocity due to there being no fluid particles to dampen the velocity. To fix this, we can apply a damping force to the buoyant force to stop the object from experiencing high buoyancy force when it resurfaced.

$$\begin{aligned}\mathbf{F}_{drag} &= -k_{linear}\mathbf{v}_{rb}\frac{A}{A_{total}} \\ \tau_{drag} &= -k_{angular}\omega_{rb}\frac{A}{A_{total}}\end{aligned} \tag{13.20}$$

Where $-k_{linear}$ and $-k_{angular}$ is the linear and angular drag coefficient, $\mathbf{v}_{rb}$, ω_{rb} is the linear and angular velocity of the rigid body, A_{total} is the area of the original rigid body before it is clipped. After that we can apply the force and torque to the velocity as

$$\begin{aligned}\mathbf{v}_{rb} &= \mathbf{v}_{rb} + M_{rb}^{-1}\mathbf{F}_b\Delta t_{sub} \\ \omega_{rb} &= \omega_{rb} + I_{rb}^{-1}\tau_b\Delta t_{sub} \\ \mathbf{v}_{rb} &= \mathbf{v}_{rb} + M_{rb}^{-1}\mathbf{F}_{drag}\Delta t_{sub} \\ \omega_{rb} &= \omega_{rb} + I_{rb}^{-1}\tau_{drag}\Delta t_{sub}\end{aligned} \tag{13.21}$$

For soft body buoyancy we simply split the force between the point masses. The force is distributed based on the point mass depth relative to the fluid surface, the depth of a point mass $\mathbf{p}_{pm,i}$ is

$$d_i = \max(0, y_{surface} - y_{pm,i}) \tag{13.22}$$

from there we can find the weight factor of each point mass as

$$w_i = \frac{d_i}{\sum_j d_j} \tag{13.23}$$

After that, we can find the force applied to each point mass based on their weight factor

$$\begin{aligned}\mathbf{F}_{i,b} &= w_i\mathbf{F}_b \\ \mathbf{F}_{i,drag} &= -k_{linear}\mathbf{v}_i\frac{A}{A_{total}}\end{aligned} \tag{13.24}$$

Where $\mathbf{F}_b$ is the buoyancy force calculated using the same formula as rigid body, $\mathbf{v}_i$ is the velocity of the point mass, k_{linear} is the coefficient of linear drag. Finally we can apply the force to each point mass's acceleration as

$$\mathbf{a}_i = \mathbf{a}_i + M_i^{-1}(\mathbf{F}_{i,b} + \mathbf{F}_{i,drag}) \tag{13.25}$$

After implementing buoyancy, the full PBF loop becomes:

- For each particle $\mathbf{p_i}$, integrate the external forces (gravity) to velocity $\mathbf{v_i} = \mathbf{v_i} + \Delta t\mathbf{F_{external}}$. After that calculate the predicted position before the constraint are applied $\mathbf{p'_i} = \mathbf{p_i} + \Delta t\mathbf{v_i}$.
- Calculate submerged area of rigid and soft bodies
- Apply rigid and soft buoyancy
- For each particle, find its neighbouring particle, the search range is the smoothing radius h.
- As with XPBD and PGS, λ is converge over multiple solve iteration. For each solve iteration do

  - Calculate λ_i for each particle
  - Calculate $\Delta \mathbf{p_i}$ for each particle
  - Update predicted position $\mathbf{p_i'} = \mathbf{p_i}' + \Delta \mathbf{p_i}$
- Detect rigid body collision and apply partial position correction to fluid particles
- Detect soft body collision and apply partial position correction to fluid particles
- For all particles, update velocity based on the change in position $\mathbf{v_i} = \frac{\mathbf{p_i'} - \mathbf{p_i}}{\Delta t}$. Then finally apply the predicted position to the particle position $\mathbf{p_i} = \mathbf{p_i}'$
- Resolve fluid - rigid collision impulse
- Resolve fluid - soft collision impulse
- Compute and apply vorticity
- Compute and apply viscosity

13.5 Improving rigid - soft collision handling

Currently, for rigid - soft collision resolution, we are using the PGS contact constraint, then using the penetration depth to warm start the XPBD solver. This works decently well however is very unstable for soft body with a lot of point masses or low compliance springs. Currently, with fluid added, this instability has become even more prevalent, in cases where rigid box rest on top of soft box which is floating on a body of fluid, the simulation can become unstable quickly.

The new improved system for rigid - soft follows the same process as fluid - rigid and fluid - soft process. This also makes it so the system for interacting across solvers is consistent across the entire engine. For the collision resolution, we can treat soft body point masses like fluid particles, applying a partial positional correction to them. After that we fully resolves the collision by applying the impulse to the rigid body and soft body point masses.

For the collision detection, we still use the ray casting algorithm to detect if the point mass is inside of the polygon, after that use the precomputed collision normal to avoid cases where the objects can sink into each other. The main difference therefore is in how we handle the collision. Since the rigid - soft collision process is divided into two phases, rigid edges against point masses, rigid vertices against soft edges, we divide the new collision detection and resolution into the same two phases.

For rigid edges against point masses, the positional correction is applied to the point mass position $\mathbf{p}_i$ as

$$\mathbf{p}_i = \mathbf{p}_i + \mathbf{n} p \epsilon \tag{13.26}$$

Where $\mathbf{n}$ is the collision normal, p is the penetration depth, ϵ is the position correction factor. The value for ϵ is the same as for fluid but it can be different if needed. For rigid vertices against soft edges, the positional correction is divided between the two soft body point masses in the soft edge. For a soft edge with end points $\mathbf{p}_A$ and $\mathbf{p}_B$, the correction is

$$\begin{aligned}\mathbf{p}_A &= \mathbf{p}_A - \mathbf{n}pw_A\epsilon \\ \mathbf{p}_B &= \mathbf{p}_B - \mathbf{n}pw_B\epsilon\end{aligned} \tag{13.27}$$

Where w_A and w_B is the weight of the point mass A and B, this is calculated based on how close the point mass is to the contact point.

After the position correction, the next step is to apply the full impulse the rigid body and point mass. This process is also split into two parts similar to the positional correction.

For the point mass against rigid edges case, we first calculate the velocity at contact as

$$\mathbf{v}_c = \mathbf{v}_{rb} + \omega \times \mathbf{r} \tag{13.28}$$

Where $\mathbf{r} = \mathbf{p}_c - \mathbf{p}_{rb}$ is the relative position between the contact point and the center of the rigid body. After that the velocity along the normal is

$$v_n = (\mathbf{v}_{pm} - \mathbf{v}_c) \cdot \mathbf{n} \tag{13.29}$$

After that we can calculate the effective mass as

$$M^{-1} = M_{pm}^{-1} + M_{rb}^{-1} + I_{rb}^{-1}(\mathbf{r} \times \mathbf{n})^2 \tag{13.30}$$

After that we can calculate the stabilization bias and Lagrange multiplier as

$$\begin{aligned}B &= \frac{\beta}{\Delta t_{sub}}p \\ \lambda_n &= \frac{B - (1+c)v_n}{M^{-1}}\end{aligned} \tag{13.31}$$

Finally the velocity of the point mass and the rigid body can be calculated as

$$\begin{aligned}\mathbf{v}_{pm} &= \mathbf{v}_{pm} + M_{pm}^{-1}\lambda_n\mathbf{n} \\ \mathbf{v}_{rb} &= \mathbf{v}_{rb} - M_{rb}^{-1}\lambda_n\mathbf{n} \\ \omega_{rb} &= \omega_{rb} - I_{rb}^{-1}(\mathbf{r} \times \lambda_n\mathbf{n})\end{aligned} \tag{13.32}$$

For the rigid vertex against soft edges case we also do the same thing but with slight differences. The velocity at contact is calculated exactly the same however the velocity along the normal is now calculated as

$$v_n = \mathbf{n} \cdot (\mathbf{v}_c - \mathbf{v}_A w_A - \mathbf{v}_B w_B) \tag{13.33}$$

Where $\mathbf{v}_A$, $\mathbf{v}_B$ is the velocity of the point mass A and B of the soft edge. w_A, w_B is the weight of point mass A and B. The effective mass is now calculated as

$$M^{-1} = M_{rb}^{-1} + I_{rb}^{-1}(\mathbf{r} \times \mathbf{n})^2 + M_A^{-1}w_A^2 + M_B^{-1}w_B^2 \tag{13.34}$$

After that the impulse is calculated the same way as before, the velocity is applied to the rigid body and point masses as

$$\begin{aligned}
\mathbf{v}_A &= \mathbf{v}_A - M_A^{-1} w_A \lambda_n \mathbf{n} \\
\mathbf{v}_B &= \mathbf{v}_B - M_B^{-1} w_B \lambda_n \mathbf{n} \\
\mathbf{v}_{rb} &= \mathbf{v}_{rb} + M_{rb}^{-1} \lambda_n \mathbf{n} \\
\omega_{rb} &= \omega_{rb} + I_{rb}^{-1} (\mathbf{r} \times \lambda_n \mathbf{n})
\end{aligned} \tag{13.35}$$

Currently we have only apply the correction along the normal direction, this means that currently no friction is applied to the object. To introduce friction, we simply apply an additional impulse along the tangent direction. This is also divided into two phases similar other processes.

For the point mass against rigid edges case, we first calculate the velocity along the tangent as

$$v_t = (\mathbf{v}_{pm} - \mathbf{v}_c) \cdot \mathbf{t} \tag{13.36}$$

The friction impulse is calculated from the impulse along the normal based on the coefficient of static and dynamic friction

$$\lambda_t = \begin{cases} \lambda_{t,trial} & \text{if } |\lambda_{t,trial}| \leq \mu_s \lambda_n \\ \mu_d \lambda_n & \text{if } \lambda_{t,trial} > \mu_s \lambda_n \\ -\mu_d \lambda_n & \text{if } \lambda_{t,trial} < -\mu_s \lambda_n \end{cases} \tag{13.37}$$

Where $\lambda_{t,trial} = -\frac{v_t}{M^{-1}}$, μ_s is the coefficient of static friction, μ_d is the coefficient of dynamic friction. After that we can apply the friction impulse to the velocity similar to the normal impulse. For the other case, rigid vertices against soft edges, we simply do the same thing but split the impulse and velocity by weights w_A, w_B similar to how we did it for normal impulse.

Finally, we can integrate the new system into the XPBD solve loop directly, this will make the new XPBD solve loop will be

- Loop n times where n is the number of substeps, calculate $\Delta t_{sub} = \frac{\Delta t}{n}$
- Apply gas pressure to soft body if enabled
- Integrate all soft body point masses and proxy points
- Loop m times where m is the position integration steps, each step calculate $\Delta\lambda$ for each constraint and integrate it with the point masses / proxy points position
- Detect rigid - soft body contacts and apply partial positional correction
- Recalculate PGS constraints, using Δt_{sub}
- Calculate velocity based on change in position for point masses and proxy points $\mathbf{v} = \frac{\mathbf{x} - \mathbf{x}_{prev}}{\Delta t_{sub}}$
- Calculate angular velocity of proxy points based on change in rotation $\omega = \frac{\theta - \theta_{prev}}{\Delta t_{sub}}$
- Apply collision impulses for point masses - rigid edges case and rigid vertices - soft edges case

Chapter 14

Python scripting

14.1 Script component

After the previous chapter, the physics section is complete, this is the beginning of the scripting and AI integration chapter where I will go over how the scripting systems work as well as reinforcement training environment integration.

For scripting, the user can create a script which will can be attached to an object similar to how components are added to the object. Every script is actually just a script component that links to a specific python script file, this means that the script will inherit functionality of base component which is useful.

The script component first task is to load the script from the linked python file. First the script class is imported from the python file as details in *Figure 14.1*

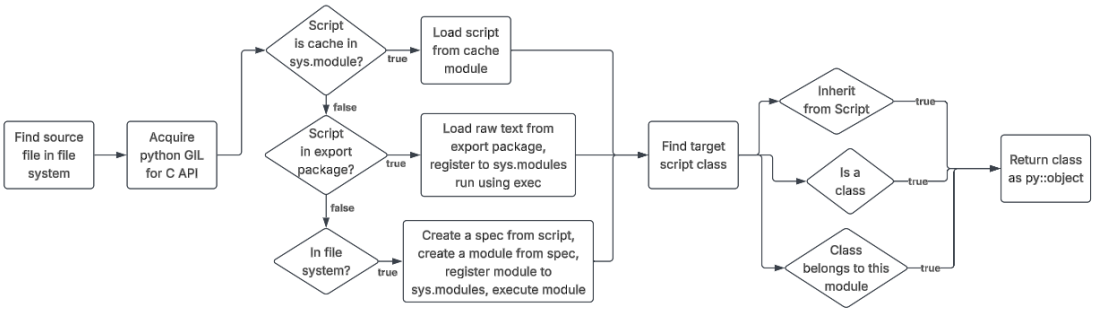

Figure 14.1: Import script class

It can be seen that we first load the script from the file system which has 3 cases, the first case is that the python file is already cache in sys.modules so there's no need to reload the script. The second case is that the engine is in player (exported) mode and the data of the scripts are encrypted so we need to decrypt the script to get the raw text, register it to the cache and execute the the

script. Finally for the normal editor case, we load the script from the file, register it to the cache and execute the script.

The reason why we need to execute the script even when loading it and not only when starting to run the game is because python execute line by line and only knows about a class inside of the script when it execute the line that define that class. So, in order to get the desired class (class that inherit from fusion.Script), we need to first execute the python script so that we can find out which class exists in the script and select the right one. Since ideally, the script will only contain the desired class and no other code outside of the class itself, no functionality within the class will be run on load and will only be ran when it is called. Importing the script is only part of the loading process though, the entire process is displayed in *Figure 14.2*

Figure 14.2: Load script process

As I said above, when loading the script, none of the start up code would run. Like Godot and Unity, my engine's script component also defines two basic functions that user created scripts can override and use. The first is the **OnStart()** function which run once at the start of the game and the second is **Process(delta)** which runs every frame. These functions are called for every script component using attr() function in pybind11. When creating a script, a template script will automatically be generated as

```
1 from fusion import *
2
3 class NewScript(Script):
4         def __init__(self):
5                 super().__init__()
6         def OnStart(self):
7                 pass
8         def Process(self, delta: float):
9                 pass
```

14.2 Component registry

In the C++ code of my engine, get component, add component are merely C++ function that can easily be used. However, this does not transfer directly to python because in python the typing is not specified, everything is just a python object. The syntax for get, has, add, remove components is like this

```
tc : TransformComponent = self.get_component(TransformComponent)
```

Since the parameter is a class and not a reference or instance, we need to identify which component type to get base on the class. To do this I basically register each component to a hash map (component registry) where each python class object maps to a function that perform either get, add, remove or has component function in C++. Other than the component type T, the mapped function also takes in a pointer to an object basically to specify which object the component is getting from. The object pointer parameter is automatically passed, the value is the object that the get, add, remove or has component function is called from, so in the case above the object is self (owner of the script). The component registry for get component is detailed in *Figure 14.3*

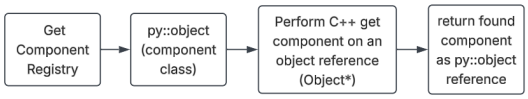

Figure 14.3: Get component registry

For each component registered to python bindings we need to enable get, add, remove and has component on it which basically allow the component to perform get, add, remove and has component, for example

```
tc : TransformComponent = self.get_component(TransformComponent)
rc : RenderComponent = tc.get_component(RenderComponent)
```

All in all, when registering a component to python bindings we do

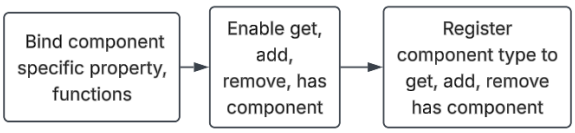

Figure 14.4: Register component bindings

This process works for engine component like transform component, render component,.. however for user created scripts which in my engine are also treated as components the current system here will not work. The reason is because the component registry is build and compiled from the engine C++ code which never knows about user created scripts and only know the base component. For user created scripts, we need a way to register them dynamically when they created to the component registry to be able to perform get, add, remove or has component on the scripts. The way I do this is detailed below in *Figure 14.5*

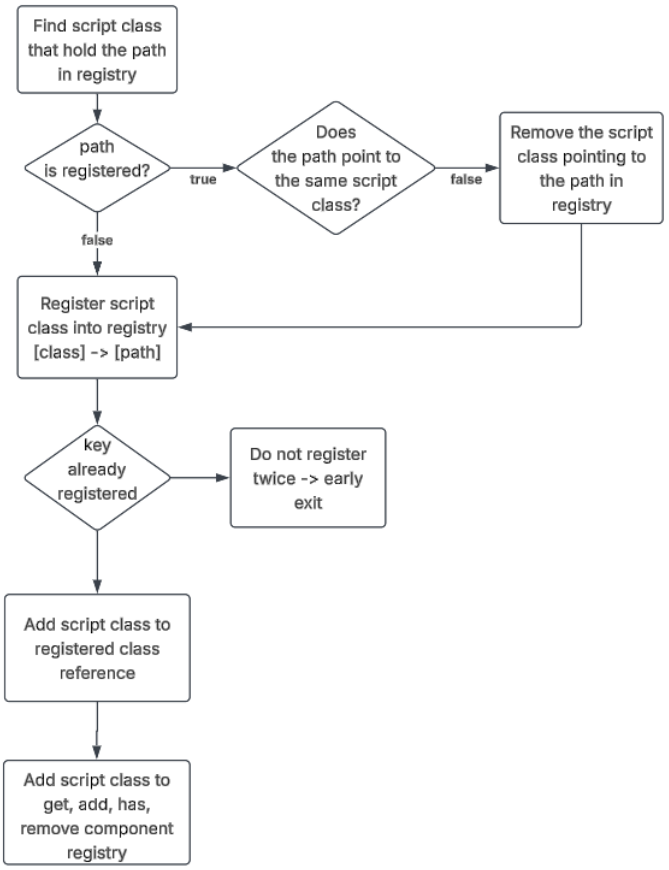

Figure 14.5: Register script as component

There are 2 new registry here, the first is the script class registry which is a hash map with key being a python object (script class reference) and the value is the virtual path that point to the script class. The second is the class reference registry, the main purpose of this registry is mainly just to keep the class object alive by holding a reference in memory (avoid the object from being discarded or override by the garbage collector). At the end of the import script class function, the script class will also be passed into the register script as component function in order for it to be able to be accessed via get, add, remove and has component.

Unlike normal engine components, user created scripts are all stored internally as script component so there would be no way to differentiate between scripts. As I said above, each script component is differentiated only by the script path that the script component points to and the way get, has, add, remove component for script class works in a similar way. However, I still use the same registry for both engine component and script class, so the key is still a python class object (in this case it is the script class) and the value is still a function. The get, has, add, remove function value for the registry is

- get component: Loop through each script component, if the script component script instance class name is the same as the python class key then return the instance
- remove component: Does the same checking as get component but delete the component from the object instead of returning the instance
- has component: Call get component, if it returns a valid instance, return true, else return false
- add component: Create a new script component, point the script component to the python script path and add it to the object

The syntax for get component is displayed below, the other script class must be imported to access the actual class. Additionally scripts are reloaded (performing import script class, register script as component) when it is edited. To check this I just check if the previous edit time has been changed.

```
1 from enemy import Enemy
2 e = self.get_component(Enemy)
```

However because a component is only registered and loaded if it is attached to an object in the editor currently. If for example, we create an object at run time and try to attach a script to it using add component, because the script was never added to any object before, the script would just throw an error. To fix this, I basically have another function, auto register script class which basically try to register a script when it is created or added to the project file not when it is attached to an object. The process is showed in *Figure 14.6*

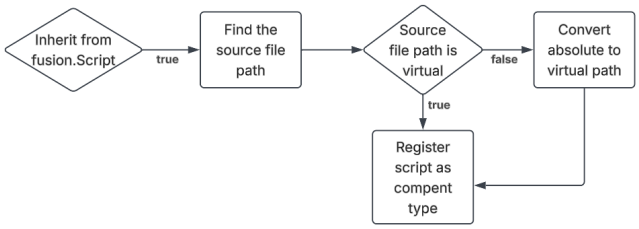

Figure 14.6: Auto register class

14.3 Export properties

Sometimes it is desirable to be able to edit variables in the editor. This is a feature present in almost all game engine and is also present in mine. The way to export a property is to called an export function when declaring the property. So for example

```
count : int = export(0)
name : str = export("")
```

The export function return an Export Marker which marks the property for export. The export marker contains a python object which represent the value that the exported property is storing. Other than that, the export marker also contains data about the export type, the range of the value and other data.

The export type dictates how the property get displayed in the inspector, as well as the data that is stored with the property. For example, export type range will export the property with a min, max, stepping value and display it as a slider. Each export type is associated with its own function and can work with different data types, for example export color edit can only work with vector 3 or vector 4, export range can only work with float or int, passing in a wrong data type will result in an error. This is showed in the code block below

```
speed : float = export_range(20, min=5, max=10, slider=True, suffix=" m/s")
color : Vector4 = export_color_edit(Vector4(0))
color_invalid : float = export_color_edit(0.5) #invalid
```

Since the export properties are meant to be declared outside of the init or start function, when loading the script, the engine will recognizes all the export properties as of type Export Marker. This is useful in differentiating between exported and normal properties. After the script is loaded, the attributes will automatically be replaced with real values. When the script is first loaded, after import script class and registering script as component type, we will scan for exported properties. The logic for the scan is detailed in *Figure 14.7* below

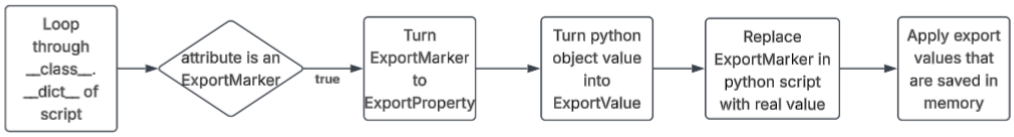

Figure 14.7: Scan exported properties

All declared properties of a class is stored in __dict__ so we can scan it to find one that is of type Export Marker. Export Property is a C++ struct that handles things in the C++ engine side, it

basically stores the same info as Export Property but instead of a python object representing the stored value of a property it is a std::variant Export Value. The Export Value variant just makes it easier to determine which type the exported property represent and make it easy for syncing between the engine and python script.

The process the displaying the properties is showed in *Figure 14.8*. Refresh exported property loop through every stored properties and update its value based on what is set in the python script. After that depending on the type of the export property, we process the property inspector display differently. If the property is edited in any way, we find the property inside of the python script and update its value based on the entered value. Due to the properties of variant type in C++, it is easy to cast from Export Type to the appropriate float, string, int,.. data type for each Export Property while avoiding conversion errors.

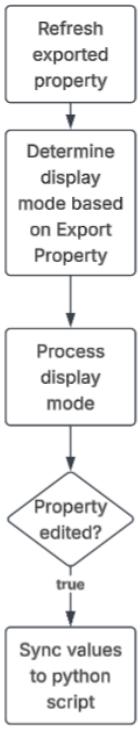

Figure 14.8: Display exported property

Additionally, the exported values is also serialize into the scene file and deserialize on load so the values won't be loss upon reloading the scene. It might also be desirable to export an Object and select an object from the scene, this will construct an Object reference using a stored pointer and the object's id. A deletion callback will also be linked so that in the case where the object is removed, the object reference will automatically be set to a null pointer with an object id of -1 instead of storing a stale pointer which could cause crashes. On serializing, the engine does not store the pointer but the object id then find the appropriate object on deserialization and relink the reference.

Chapter 15

Reinforcement learning integration

15.1 Package management

In my engine, instead of having all features included in the base engine which could make it very bloated, I get features behind packages which the user can quickly install or uninstall from their projects. This keeps the core engine concise and only have the most basic features while being easy to extend, include in new features.

On the project selection screen, the user can select configure project and manage their installed packages. Reinforcement learning package is one of the packages and will be discussed in this chapter. A package usually contain python libraries (numpy, pytorch, gymnasium,..) and additional engine features (custom python API, new editor windows,..). The selected packages is stored in a small file which store details about the package id, requirements (python packages to be installed when selecting this package), is the package selected and is the package installed. There is a difference between if a package is selected and installed because if it is already installed then the installation code will not run for the package saving load time, if the package is only selected then the installation code will run when the project is opened. When opening a project with packages, a background thread is performed to check if any packages has been added or removed and to check if all the packages are up to date. If not, a syncing process is performed to install and delete necessary python packages or engine features. The syncing process is detailed in *15.1*

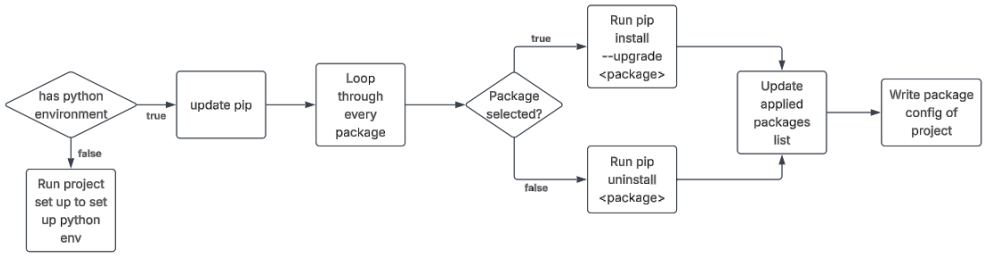

Figure 15.1: Sync project packages

The syncing code will only install or uninstall python packages, however for engine python bindings it will be sync in another sub process. For example the reinforcement relearning package have its own fusionRL python API that allow you to access the engine features for reinforcement learning via the logic in *Figure 15.2*

```
1 import fusionRL
```

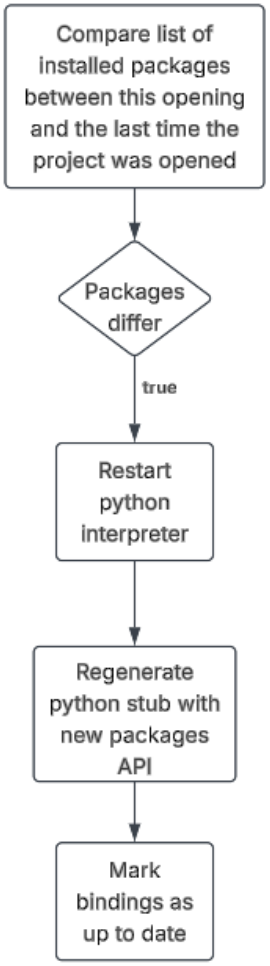

Figure 15.2: Sync python bindings

15.2 Agent component

When including the reinforcement learning package, certain aspect of the engine will be unlocked, the first part is the agent component which is attach to an object so the engine knows that the object is the agent that needs to be trained.

By itself, the agent component does not do anything, it is only when combined with an additional script on the object that the agent component does anything. The agent component allow one to add observation, reward, action,.. to the agent through both the python API and the inspector.

At its core, the reinforcement learning loop is simple, at each time steps, the environment gives the agent an observation, the agent responds with the an action, the environment punishes or rewards the agent based on the action, the agent changes its action to optimize for reward. This process is detailed in *Figure 15.3*, I will touch on the training loop more in subsequent sections when talking about environment and training.

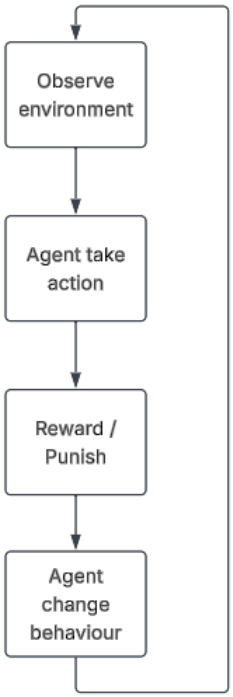

Figure 15.3: Reinforcement training loop

The agent component allows the user to configure the observation and action space based on gymnasium library spaces. The type of spaces are detailed below

- Discrete: the most basic space type, it represent a single scalar integer out of a set of possible values. For example Discrete(4) represent a single choice out of $\begin{bmatrix}0 & 1 & 2 & 3\end{bmatrix}$. This is most commonly used for action spaces, for example 0 could be mapped to doing nothing, 1 could be

mapped to moving jump, 2 to moving left, 3 to moving right and the agent at each timestep select one of those action.

- Multi Discrete: Sometimes however, we want multiple separate actions to be executed simultaneously. This is what multi discrete is, basically an array of discrete space. For example, a game controller have a D pad with 4 directions and 3 different buttons for each action. During gameplay, we might want to have the player move and shoot at the same time and Multi Discrete serves this exact purpose.
- Box: This is most commonly used as the observation space and for good reasons, the box space is basically an N-dimensional array of real values within a range. The values can be position, rotation, forces,..
- Multi Binary: Represent an N-dimensional array of binary values (each value is strictly 0 or 1). This is used to represent binary state like a light switch being on or off.

Although the observation space can be set in the inspector or through code at the start, since the majority of environment are Box space, I simplify the process so that it is default to Box space. Through code the user can add data to the observation and the space will automatically expand to fit all the data.

15.3 Headless running

While it is possible to train the agent on the engine as is with everything rendered, it is of course not very efficient. Most of the time the agent does not need to actually see the game and only needs to receive data on it, rendering the entire game in full resolution while the agent is running is just pointless. Moreover, OpenGL capped processing at 60 fps and does not allow the game to be ran at a higher fps, this basically is putting a cap on training speed which is not what we want.

The fix for this is to disable rendering and the editor while training which is also often called headless running. When entering training mode, the engine also switch to headless mode, the process is detailed in *Figure 15.4* below

Figure 15.4: Enter headless mode

The reason training will be ran in a thread is because the engine editor won't actually disappear during training but switch to a training monitor screen where you can monitor the training stats as well as have the option to watch agent train in real time.

When the engine switch to headless mode, the main function for the engine perform a few changes to rendering. During headless mode the engine skips all rendering and physics processing,

only the training monitor gets processed. However processing the training monitor every single frame would be inefficient so the monitor only refresh at a fixed 30 Hertz to save on processing time. Sometime during training, certain function might want to run on main thread, if so they are placed in a queue and at the start of each refresh those function are executed.

When training stop, the headless mode need to be exited and normal editing mode needs to be restored. First the training is set to false and training thread is stopped if it is still running. After that, headless is set to false, then the stored editing scene is loaded and restored. In the main function, all rendering and physics processing is restored as well.

Since the training runs on different thread however the main thread still needs to access info on training status safely, I use the C++ mutex type to store data safely. A mutex type is like a lock which only allows one thread to hold its value at a time, protecting it from being read and written to by multiple threads at once. The way mutex are used in my engine to communicate data between main thread and training thread is shown in *Figure 15.5*

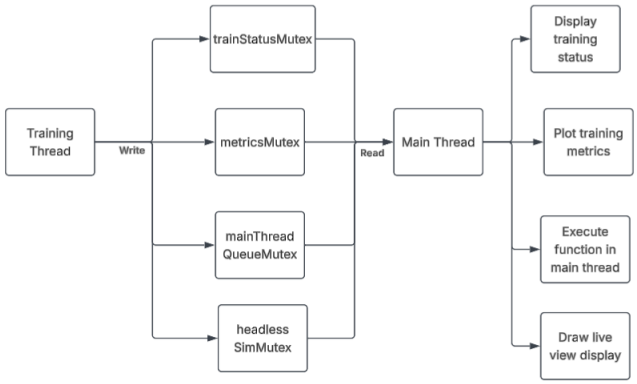

Figure 15.5: Communication between headless and main thread

While it is desirable to always run in headless mode during training, sometimes it is also desirable to be able to see how the agent is behaving while it is training. This is the purpose of the live view display, which act as a temporary display to render the game. *Figure 15.6* shows what happen when the live view tab is selected. It also should be noted that live view tab can slow down training and also may sometime affect training results.

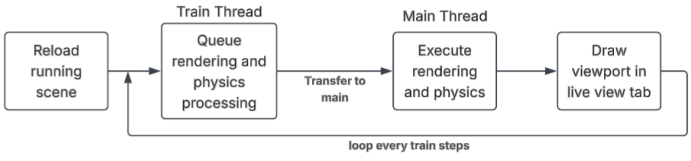

Figure 15.6: Live train view

15.4 Environment

One of the most important aspect of training a reinforcement learning agent is the environment. Any game created in my engine can act as an environment and have agent train on it. While the core for the engine is written in C++, the environment itself is actually written in python but is included as text inside of the C++ python bindings script and get compiled with the engine.

A gymnasium environment contains a few core function that is also implemented in my engine. The first is the init function which basically find the agent component and then takes the action and observation space defined in the component and apply it to the training environment. Next is the step function, this basically process the game, add up the reward and get the observation. Finally the reset function basically reload the entire scene. Stepping is the main logic of the environment since it is the process that actually runs the environment and allow the agent to train. The logic for stepping is detailed below in *Figure 15.7*

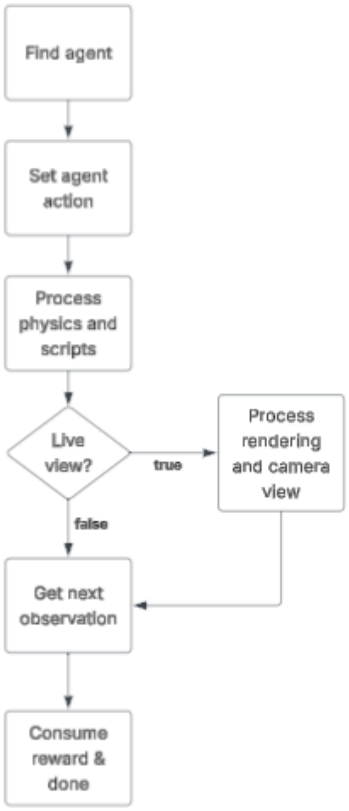

Figure 15.7: Environment stepping

The reward is added during processing script and consumed at the end. If the player die, loose or need a reset, done is set to true and consumed at the end to call reset for the environment.

Currently we have a way to set up the training environment however we also need to run actual model training on this and stable baseline library in python already provided a quick and easy way to train RL models. Before training, the user can pick and choose training algorithm, policy, the number of time steps to train for, where to save the model, whether to start from an existing model and some advance settings as well. The algorithm supported in my engine are

- A2C (Advantage Actor-Critic): As the name implies, this algorithm uses a metric called advantage, this metric basically evaluate whether an action is more advantageous than other action. The algorithm works for both discrete and continuous (Box) action space. It is an on-policy algorithm which means it learns strictly from data generated by its current policy, we will talk more about policy down below.
- PPO (Proximal Policy Optimization): One of the most commonly used algorithm for reinforcement learning and is an extension of A2C. It adds a clipping mechanism that limits how much the policy can change within a single training step which can prevent performance collapse during training, making it more stable.
- DDPG (Deep Deterministic Policy Gradient): Designed for continuous action space and outputs a deterministic action instead of a probability distribution. DDPG is off-policy which means it samples from a buffer of past experiences making it highly sample-efficient, though it can suffer from instability.
- TD3 (Twin Delayed DDPG): A direct upgrade to DDPG which was designed to fix the instability of DDPG. It uses two separate critic network and update the actor network less frequently than the critics, ensuring policy on updates based on accurate value estimates.
- SAC (Soft Actor-Critic): An algorithm that modifies the standard RL objective by adding entropy maximization. Basically the agent try to do things as randomly as possible while still completing the task, this basically encourages exploration instead of converging to a sub-optimal strategy.

Beside training algorithm, there is also training policy, the three training policy supported in my engine are MLP, CNN and Multi Input policy.

- MLP policy: The input is a 1D array of numerical data like positions, velocities, health,.. The policy usually uses standard multi layer perceptron (MLP) architecture for deep learning. This works for most common task and does not require much processing power, usually CPU is enough.
- CNN policy: The input is a 2D or 3D array usually representing images. The policy uses a convolutional neural network to analyze and process the inputted images. Since the observation contain much more data than MLP policy, it requires a lot more processing power and it is recommended that the model is trained on the GPU instead of CPU.
- Multi Input policy: This basically combines MLP and CNN policy together for a multi-model approach, the model gets both the raw data of MLP with positions, velocities,.. as well as the visual input of CNN. The model is basically a combination of MLP layer and CNN layer together and because of this it is also resource intensive to run and require a GPU for training to progress at a reasonable rate.

15.5 Threading and engine snapshot